ipfx Documentation

Release 1.0.8

Allen Institute for Brain Science

Jun 29, 2023

Contents

The easiest way to install `ipfx` is via pip:

```
pip install ipfx
```

We suggest installing `ipfx` into a managed Python environment, such as those provided by anaconda or venv. This avoids conflicts between different Python and package versions.

CHAPTER 1

Installing for development

If you wish to make contributions to `ipfx` (thank you!), you must clone the git repository. You will need to install git-lfs before cloning (we store some test data in lfs). Once you have cloned `ipfx`, simply pip install it into your environment:

```
pip install -e path/to/your/ipfx/clone
```

(-e installs in editable mode, so that you can use your changes right off the bat).

See contributing for more information.

CHAPTER 2

Quick Start

To get started, install IPFX via pip into a fresh conda evironment. For details see the Installation guide *Installation*.

Next, example datasets can be downloaded according to the instructions in the Data Access *Data Access*.

To process a dataset saved in the nwb file, run:

```
$ python -m ipfx.bin.run_pipeline_from_nwb_file <input_nwb_file> <output_dir>
```

Input:

- input_nwb_file: a full path to the NWB file with cell ephys recordings
- output_dir: an output directory to save the results

Output:

- input.json: input parameters
- output.json: output including cell features
- output.nwb: NWB file including spike times
- log.txt: run log
- qc_figs: index.html includes cell figures and feature table and sweep.html includes sweep figures

CHAPTER 3

Tutorial

This guide will walk you through the functionality of IPFX starting with the simplest examples and build on itself towards increasing complexity of the scope of the analysis and the size of data to be analyzed.

3.1 Detect Action Potentials

To detect action potentials, you must provide three 1D numpy arrays: stimulus current `i` in pA, response voltage `v` in mV, and timestamps `t` in seconds. With these three arrays, you can then do the following:

```
from ipfx.feature_extractor import SpikeFeatureExtractor

ext = SpikeFeatureExtractor()
spikes = ext.process(t, v, i)
```

The `spikes` object is a `pandas` `DataFrame` where rows are action potentials and columns are features.

3.2 Extract Spike Train Features

Given a spike train `DataFrame`, you can then compute a number of other features(e.g. adaptation and latency:

```
from ipfx.feature_extractor import SpikeTrainFeatureExtractor

ext = SpikeTrainFeatureExtractor()
features = ext.process(t, v, i, spikes) # re-using spikes from above
```

3.3 Stimulus-specific Analysis

To analyze all of the sweeps with a particular stimulus type (say, long square pulses), you'll first need to create a *SweepSet* object. This object provides utilities for accessing properties of a group of *Sweep* objects:

```
from ipfx.sweep import Sweep, SweepSet

sweep_set = SweepSet([ Sweep(t=t0, v=v0, i=i0),
                       Sweep(t=t1, v=v1, i=i1),
                       Sweep(t=t2, v=v2, i=i2) ])
```

Sweeps corresponding to the same stimulus type may have different stimulus start time. This happens because sweeps may have different duration of the test pulse epoch preceding the stimulus epoch. To perform the analysis, we must align a sweep set to have the same stimulus start times.

Here, we will align sweeps to the beginning of the experimental epoch, that includes stimulus epoch padded by the pre-stimulus and post-stimulus time intervals. With sweeps aligned, we can obtain common to all sweeps `start_time` and `end_time` time of the stimulus:

```
sweep_set.align_to_start_of_epoch("experiment")

sweep = sweep_set[0]
t = sweep.t
start_idx, end_idx = sweep.epochs["stim"] # choose stimulus epoch
start_time, end_time = t[start_idx], t[end_idx]
```

Now that we have this object, we can hand it to one of the stimulus-specific analysis classes. You first need to configure a *SpikeFeatureExtractor* and *SpikeTrainFeatureExtractor*:

```
from ipfx.feature_extractor import SpikeFeatureExtractor, SpikeTrainFeatureExtractor
from ipfx.stimulus_protocol_analysis import LongSquareAnalysis

spx = SpikeFeatureExtractor(start=start_time, end=end_time)
spfx = SpikeTrainFeatureExtractor(start=start_time, end=end_time)

analysis = LongSquareAnalysis(spx, spfx)
results = analysis.process(sweep_set)
```

At this point `results` contains whatever features/objects the analysis instance wants to return.

3.4 Analyze a Data Set

The *extract_data_set_features()* function allows you to calculate all available features for a given dataset in one call. IPFX supports datasets stored in Neurodata Without Borders 2.0 (NWB) format via a *EphysDataSet* class, which provides a well-known interface to all of the data in an experiment. The data released by the Allen Institute is hosted on the DANDI public archive in the NWB format. Refer to *Data Access* page for the instructions on downloading the data files.

To create an instance of the *EphysDataSet*:

```
from ipfx.dataset.create import create_ephys_data_set

dataset = create_ephys_data_set(nwb_file="path/to/experiment.nwb")
long_squares = dataset.filtered_sweep_table(stimuli=ds.ontology.long_square_names) #␣
↪more on this next!
sweep_set = dataset.sweep_set(long_squares.sweep_number)
```

where `path/to/experiment.nwb` is a local path to the nwb2 file that you have downloaded from the public archive.

With an instance of the *EphysDataSet* available you can easily obtain: a *Sweep* for a given sweep number:

```
sweep = ds.sweep(sweep_number)
t = sweep.t
v = sweep.v
i = sweep.i
```

with the corresponding `t`, `v`, and `i` arrays.

You may also obtain a *SweepSet*, for a particular grouping of sweeps by filtering the `sweep_table`:

```
long_squares = dataset.filtered_sweep_table(stimuli=dataset.ontology.long_square_
↪names) # more on this next!
sweep_set = dataset.sweep_set(long_squares.sweep_number)
```

where `dataset.ontology` includes references to the names of all stimuli types known to `IPFX`. See *Stimulus Types* for details.

Finally, you can run end-to-end analyses on an NWB file:

to calculate the QC features:

```
from ipfx.qc_feature_extractor import sweep_qc_features, cell_qc_features

dataset = create_ephys_data_set(nwb_file="path/to/experiment.nwb")
sweep_qc_features = sweep_qc_features(dataset)
cell_features, cell_tags = cell_qc_features(dataset)
```

and to calculate the analysis features:

```
from ipfx.data_set_features import extract_data_set_features

drop_failed_sweeps(data_set)     # sweeps with incomplete recording or failing QC␣
↪criteria
(cell_features, sweep_features,
 cell_record, sweep_records,
 cell_state, feature_states) = dsft.extract_data_set_features(data_set)
```

this code block does the following:

1. Creates a dataset
2. Drops failed sweeps that cannot be used for feature extraction
3. Computes features for the Long Square, Short Square and Ramp sweeps

CHAPTER 4

Data Access

The electrophysiology data files for the PatchSeq experiments released by the Allen Institute are stored in Neurodata Without Borders 2.0 (NWB) format. The files are hosted on the Distributed Archives for Neurophysiology Data Integration (DANDI).

The PatchSeq data release is composed of two archives:

Mouse data archive (114 GB): https://dandiarchive.org/dandiset/000020

Human data archive (12 GB): https://dandiarchive.org/dandiset/000023

Each archive is accompanied by the corresponding file manifest and the experiment metadata tables.

The file manifest table contains information about the files included in the archive, their location ("archive_uri" column) and the corresponding cell("cell_specimen_id" column). The file manifest combines information about several different data modalities (see the "technique" column) recorded from each cell. The files with the intracellular electrophysiological recordings stored on DANDI are denoted as "technique" = intracellular_electrophysiology.

In turn, the experiment metadata table includes information about the experimental conditions for each cell("specimen_id" column). This table could be used to select the desired cells satisfying particular experimental conditions. Then, given the desired "specimen_ids", you can find the corresponding DANDI urls of these data from the file manifest.

IPFX includes a utility that provides file manifest and experiment data of the published archives.

For example, to obtain detailed information about Human data archive:

```
from ipfx.data_access import get_archive_info
archive_url, file_manifest, experiment_metadata = get_archive_info(dataset="human")
```

where `archive_uri` is the DANDI URL for the Human data, `file_manifest` is a pandas.DataFrame of file manifest and `experiment_metadata` is a pandas.DataFrame of experiment metadata. To obtain the same information for the Mouse data, change to *dataset="mouse"* in the function argument.

You can download data files by directly entering the DANDI's archive_uri in your browser. Alternatively, a more powerful option is to install DANDI's command line client:

```
pip install dandi
```

With client installed, you can easily download individual files or an entire archive as:

```
dandi download --output-dir <DIRECTORY> <URL>
```

where <DIRECTORY> is the existing directory on your file system and <URL> is the url of a file or an archive.

CHAPTER 5

Stimulus Types

Higher-level analyses available in `IPFX` are critically-dependent on stimulus type and naming conventions. Unless overwritten, all *`EphysDataSet`* instances create a default *`StimulusOntology`* based on stimulus_ontology.json. The instance of this class provides mechanisms for searching for stimuli that have been tagged with standardized names.

For example, in the default ontology stimulus_ontology.json we find one of the entries:

```
[
    [
        "core",
        "Core 1"
    ],
    [
        "resolution",
        "Coarse"
    ],
    [
        "name",
        "Short Square"
    ],
    [
        "code",
        "C1SSCOARSE",
        "C1SSCOARSE150112"
    ]
],
```

where: code: "`C1SSCOARSE150112`" and ""`C1SSCOARSE`"" are a stimulus codes found in the dataset core: "`Core 1`" (part of the basic protocol used for all data sets), name: "`Short Square`" (3ms square pulse), resolution: "`Coarse`" (large jumps between amplitudes while searching for an action potential).

Tagging provides a mechanism for mapping stimulus codes to stimulus types and enables filtering of the sweeps based on stimulus types.

For example, Short Square stimuli are identified by the following name tags:

```
self.short_square_names = ( "Short Square",
                            "Short Square Threshold",
                            "Short Square - Hold -60mV",
                            "Short Square - Hold -70mV",
                            "Short Square - Hold -80mV" )
```

that allows mapping the sweep with the stimulus code "C1SSCOARSE150112" to the Short Square stimuli 'self.short_square_names'.

With the ontology defined, you can now filter *EphysDataSet* sweeps by the stimulus type:

```
short_square_table = data_set.filtered_sweep_table(
    stimuli=data_set.ontology.long_square_names
)
```

that returns a table of metadata for the sweeps matching the `self.short_square_names` tags.

CHAPTER 6

Gallery of Examples

6.1 Stimulus-specific Analysis

6.1.1 All Analysis

Run analysis for all stimulus types on data in the NWB file

Out:

```
/home/docs/checkouts/readthedocs.org/user_builds/ipfx/envs/latest/lib/python3.6/site-
↪packages/hdmf/spec/namespace.py:485: UserWarning: Ignoring cached namespace 'hdmf-
↪common' version 1.1.0 because version 1.3.0 is already loaded.
  % (ns['name'], ns['version'], self.__namespaces.get(ns['name'])['version']))
/home/docs/checkouts/readthedocs.org/user_builds/ipfx/envs/latest/lib/python3.6/site-
↪packages/hdmf/spec/namespace.py:485: UserWarning: Ignoring cached namespace 'core'␣
↪version 2.2.0 because version 2.2.5 is already loaded.
  % (ns['name'], ns['version'], self.__namespaces.get(ns['name'])['version']))
WARNING:root:sweep 61: ['Recording stopped before completing the experiment epoch']
WARNING:root:sweep 62: ['Recording stopped before completing the experiment epoch']
WARNING:root:sweep 63: ['Recording stopped before completing the experiment epoch']
WARNING:root:sweep 64: ['Recording stopped before completing the experiment epoch']
WARNING:root:sweep 65: ['Recording stopped before completing the experiment epoch']
WARNING:root:sweep 66: ['Recording stopped before completing the experiment epoch']
WARNING:root:sweep 67: ['Recording stopped before completing the experiment epoch']
WARNING:root:sweep 68: ['Recording stopped before completing the experiment epoch']
WARNING:root:sweep 69: ['Recording stopped before completing the experiment epoch']
WARNING:root:sweep 70: ['Recording stopped before completing the experiment epoch']
WARNING:root:Warning: stimulus_name Noise 1 not found.
STIMULUS_TYPE_NAME_MAPPING: {<StimulusType.RAMP: 'ramp'>: {'Ramp'}, <StimulusType.
↪LONG_SQUARE: 'long_square'>: {'Long Square Threshold', 'Long Square SupraThreshold',
↪ 'Long Square SubThreshold', 'Long Square'}, <StimulusType.COARSE_LONG_SQUARE:
↪'coarse_long_square'>: {'C1LSCOARSE'}, <StimulusType.SHORT_SQUARE_TRIPLE: 'short_
↪square_triple'>: {'Short Square - Triple'}, <StimulusType.SHORT_SQUARE: 'short_
↪square'>: {'Short Square - Hold -80mV', 'Short Square - Hold -70mV', 'Short Square -
↪ Hold -60mV', 'Short Square Threshold', 'Short Square'}, <StimulusType.CHIRP: 'chirp
↪'>: {'Chirp C - Hold -60mV', 'Chirp A Threshold', 'Chirp', 'Chirp D - Hold -55mV',
↪'Chirp B - Hold -65mV'}, <StimulusType.SEARCH: 'search'>: {'Search'}, <StimulusType.
↪TEST: 'test'>: {'Test'}, <StimulusType.BLOWOUT: 'blowout'>: {'EXTPBLWOUT'},
↪<StimulusType.BATH: 'bath'>: {'EXTPINBATH'}, <StimulusType.SEAL: 'seal'>: {
↪'EXTPCllATT'}, <StimulusType.BREAKIN: 'breakin'>: {'EXTPBREAKN'}, <StimulusType.
↪EXTP: 'extp'>: {'EXTP'}}
```

(continues on next page)

(continued from previous page)

```
Using default detection parameters
WARNING:root:Warning: stimulus_name Noise 2 not found.
STIMULUS_TYPE_NAME_MAPPING: {<StimulusType.RAMP: 'ramp'>: {'Ramp'}, <StimulusType.
↪LONG_SQUARE: 'long_square'>: {'Long Square Threshold', 'Long Square SupraThreshold',
↪ 'Long Square SubThreshold', 'Long Square'}, <StimulusType.COARSE_LONG_SQUARE:
↪'coarse_long_square'>: {'C1LSCOARSE'}, <StimulusType.SHORT_SQUARE_TRIPLE: 'short_
↪square_triple'>: {'Short Square - Triple'}, <StimulusType.SHORT_SQUARE: 'short_
↪square'>: {'Short Square - Hold -80mV', 'Short Square - Hold -70mV', 'Short Square -
↪ Hold -60mV', 'Short Square Threshold', 'Short Square'}, <StimulusType.CHIRP: 'chirp
↪'>: {'Chirp C - Hold -60mV', 'Chirp A Threshold', 'Chirp', 'Chirp D - Hold -55mV',
↪'Chirp B - Hold -65mV'}, <StimulusType.SEARCH: 'search'>: {'Search'}, <StimulusType.
↪TEST: 'test'>: {'Test'}, <StimulusType.BLOWOUT: 'blowout'>: {'EXTPBLWOUT'},
↪<StimulusType.BATH: 'bath'>: {'EXTPINBATH'}, <StimulusType.SEAL: 'seal'>: {
↪'EXTPCllATT'}, <StimulusType.BREAKIN: 'breakin'>: {'EXTPBREAKN'}, <StimulusType.
↪EXTP: 'extp'>: {'EXTP'}}
Using default detection parameters
{'rheobase_sweep_num': 34, 'thumbnail_sweep_num': 36, 'vrest': -67.84906877790179, 'ri
↪': 161.2471640110016, 'sag': 0.1976964920759201, 'tau': 16.20768335673351, 'vm_for_
↪sag': -86.21875, 'f_i_curve_slope': 0.3530456831503806, 'adaptation': 0.
↪05018917956237554, 'latency': 0.028179999999999872, 'avg_isi': 44.4504761904762,
↪'upstroke_downstroke_ratio_long_square': 2.7872165672263765, 'peak_v_long_square':␣
↪38.28125, 'peak_t_long_square': 0.56728, 'trough_v_long_square': -52.
↪343753814697266, 'trough_t_long_square': 0.5685800000000001, 'fast_trough_v_long_
↪square': -52.3125, 'fast_trough_t_long_square': 0.5685199999999999, 'slow_trough_v_
↪long_square': None, 'slow_trough_t_long_square': None, 'threshold_v_long_square': -
↪38.312503814697266, 'threshold_i_long_square': 70, 'threshold_t_long_square': 0.
↪56688, 'upstroke_downstroke_ratio_ramp': 2.8088185451956593, 'peak_v_ramp': 39.
↪395832, 'peak_t_ramp': 5.09762, 'trough_v_ramp': -54.96875, 'trough_t_ramp': 5.
↪099006666666667, 'fast_trough_v_ramp': -54.9375, 'fast_trough_t_ramp': 5.
↪098966666666667, 'slow_trough_v_ramp': -52.645835876464844, 'slow_trough_t_ramp': 5.
↪123726666666667, 'threshold_v_ramp': -38.875, 'threshold_i_ramp': 115.0, 'threshold_
↪t_ramp': 5.097200000000001, 'upstroke_downstroke_ratio_short_square': 2.
↪8238712481895005, 'peak_v_short_square': 40.04018, 'peak_t_short_square': 0.
↪5033514285714286, 'trough_v_short_square': -69.11607, 'trough_t_short_square': 0.
↪6062599999999999, 'fast_trough_v_short_square': -52.54018, 'fast_trough_t_short_
↪square': 0.5044228571428572, 'slow_trough_v_short_square': None, 'slow_trough_t_
↪short_square': None, 'threshold_v_short_square': -43.339287, 'threshold_i_short_
↪square': 540.0, 'threshold_t_short_square': 0.5029971428571428}
```

```
import os

from ipfx.dataset.create import create_ephys_data_set
from ipfx.data_set_features import extract_data_set_features
from ipfx.utilities import drop_failed_sweeps

# Download and access the experimental data from DANDI archive per instructions in␣
↪the documentation
# Example below will use an nwb file provided with the package

nwb_file = os.path.join(
    os.path.dirname(os.getcwd()),
```

(continues on next page)

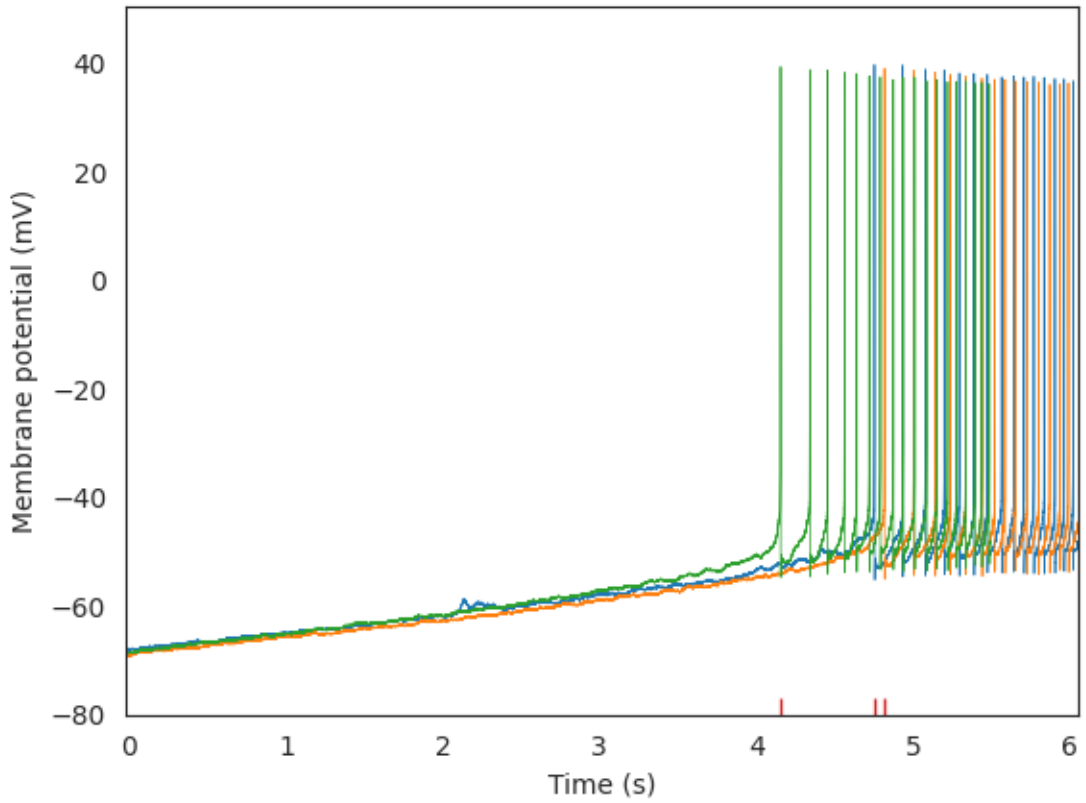
(continued from previous page)

```
    "data",
    "nwb2_H17.03.008.11.03.05.nwb"
)

# Create Ephys Data Set
data_set = create_ephys_data_set(nwb_file=nwb_file)

# Drop failed sweeps: sweeps with incomplete recording or failing QC criteria
drop_failed_sweeps(data_set)

# Calculate ephys features
cell_features, sweep_features, cell_record, sweep_records, _, _ = \
    extract_data_set_features(data_set, subthresh_min_amp=-100.0)

print(cell_record)
```

Total running time of the script: (0 minutes 19.470 seconds)

6.1.2 Ramp Analysis

Calculate features of Ramp sweeps



Out:

```
/home/docs/checkouts/readthedocs.org/user_builds/ipfx/envs/latest/lib/python3.6/site-
↪packages/hdmf/spec/namespace.py:485: UserWarning: Ignoring cached namespace 'hdmf-
↪common' version 1.1.0 because version 1.3.0 is already loaded.
  % (ns['name'], ns['version'], self.__namespaces.get(ns['name'])['version']))
/home/docs/checkouts/readthedocs.org/user_builds/ipfx/envs/latest/lib/python3.6/site-
↪packages/hdmf/spec/namespace.py:485: UserWarning: Ignoring cached namespace 'core'␣
↪version 2.2.0 because version 2.2.5 is already loaded.
  % (ns['name'], ns['version'], self.__namespaces.get(ns['name'])['version']))
```

```
import os
import matplotlib.pyplot as plt
import seaborn as sns
from ipfx.epochs import get_stim_epoch
from ipfx.dataset.create import create_ephys_data_set
from ipfx.utilities import drop_failed_sweeps

from ipfx.feature_extractor import (
    SpikeFeatureExtractor, SpikeTrainFeatureExtractor
)
from ipfx.stimulus_protocol_analysis import RampAnalysis

# Download and access the experimental data from DANDI archive per instructions in␣
↪the documentation
# Example below will use an nwb file provided with the package

nwb_file = os.path.join(
    os.path.dirname(os.getcwd()),
    "data",
    "nwb2_H17.03.008.11.03.05.nwb"
)
# Create Ephys Data Set
data_set = create_ephys_data_set(nwb_file=nwb_file)

# Drop failed sweeps: sweeps with incomplete recording or failing QC criteria
drop_failed_sweeps(data_set)

# get sweep table of Ramp sweeps
ramp_table = data_set.filtered_sweep_table(
    stimuli=data_set.ontology.ramp_names
)
ramp_sweeps = data_set.sweep_set(ramp_table.sweep_number)

# Select epoch corresponding to the actual recording from the sweeps
# and align sweeps so that the experiment would start at the same time
ramp_sweeps.select_epoch("recording")
ramp_sweeps.align_to_start_of_epoch("experiment")

# Select epoch corresponding to the actual recording from the sweeps
# and align sweeps so that the experiment would start at the same time
ramp_sweeps.select_epoch("recording")
ramp_sweeps.align_to_start_of_epoch("experiment")
```

(continues on next page)

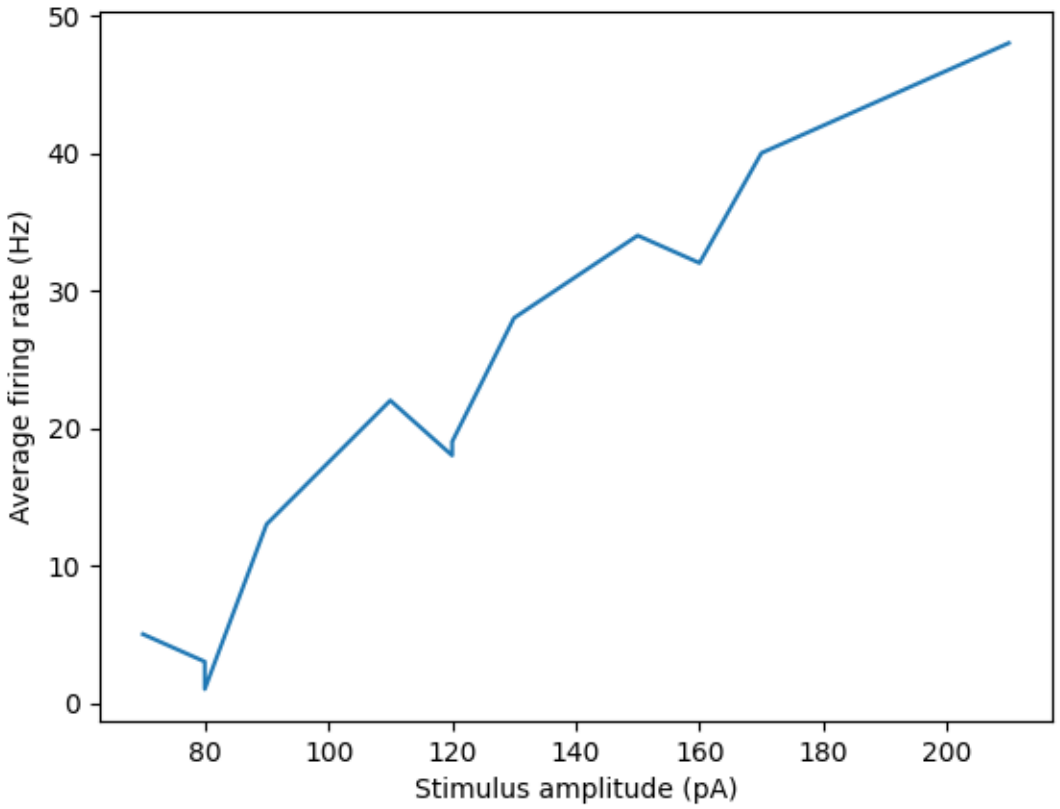
(continued from previous page)

```
# find the start and end time of the stimulus
# (treating the first sweep as representative)
stim_start_index, stim_end_index = get_stim_epoch(ramp_sweeps.i[0])
stim_start_time = ramp_sweeps.t[0][stim_start_index]
stim_end_time = ramp_sweeps.t[0][stim_end_index]

spx = SpikeFeatureExtractor(start=stim_start_time, end=None)
sptfx = SpikeTrainFeatureExtractor(start=stim_start_time, end=None)

# Run the analysis
ramp_analysis = RampAnalysis(spx, sptfx)
results = ramp_analysis.analyze(ramp_sweeps)

# Plot the sweeps and the latency to the first spike of each
sns.set_style("white")
for swp in ramp_sweeps.sweeps:
    plt.plot(swp.t, swp.v, linewidth=0.5)
sns.rugplot(results["spiking_sweeps"]["latency"].values + stim_start_time)

# Set the plot limits to highlight where spikes are and axis labels
plt.xlim(stim_start_time, stim_end_time)
plt.xlabel("Time (s)")
plt.ylabel("Membrane potential (mV)")

plt.show()
```

Total running time of the script: (0 minutes 9.156 seconds)

6.1.3 Long Square Analysis

Calculate Features of Long Square sweeps



Out:

```
/home/docs/checkouts/readthedocs.org/user_builds/ipfx/envs/latest/lib/python3.6/site-
↪packages/hdmf/spec/namespace.py:485: UserWarning: Ignoring cached namespace 'hdmf-
↪common' version 1.1.0 because version 1.3.0 is already loaded.
  % (ns['name'], ns['version'], self.__namespaces.get(ns['name'])['version']))
/home/docs/checkouts/readthedocs.org/user_builds/ipfx/envs/latest/lib/python3.6/site-
↪packages/hdmf/spec/namespace.py:485: UserWarning: Ignoring cached namespace 'core'␣
↪version 2.2.0 because version 2.2.5 is already loaded.
  % (ns['name'], ns['version'], self.__namespaces.get(ns['name'])['version']))
tau: 0.01620768335673351
v_baseline: -67.84906877790179
input_resistance: 161.2471640110016
vm_for_sag: -86.21875
fi_fit_slope: 0.35305274420526456
sag: 0.1976930797100067
rheobase_i: 70.0
```

```
from ipfx.feature_extractor import (
    SpikeFeatureExtractor, SpikeTrainFeatureExtractor
```

(continues on next page)

(continued from previous page)

```
)
import ipfx.stimulus_protocol_analysis as spa
from ipfx.epochs import get_stim_epoch
from ipfx.dataset.create import create_ephys_data_set
from ipfx.utilities import drop_failed_sweeps

import os
import matplotlib.pyplot as plt

# Download and access the experimental data from DANDI archive per instructions in␣
↪the documentation
# Example below will use an nwb file provided with the package

nwb_file = os.path.join(
    os.path.dirname(os.getcwd()),
    "data",
    "nwb2_H17.03.008.11.03.05.nwb"
)
# Create Ephys Data Set
data_set = create_ephys_data_set(nwb_file=nwb_file)

# Drop failed sweeps: sweeps with incomplete recording or failing QC criteria
drop_failed_sweeps(data_set)

# get sweep table of Long Square sweeps
long_square_table = data_set.filtered_sweep_table(
    stimuli=data_set.ontology.long_square_names
)
long_square_sweeps = data_set.sweep_set(long_square_table.sweep_number)

# Select epoch corresponding to the actual recording from the sweeps
# and align sweeps so that the experiment would start at the same time
long_square_sweeps.select_epoch("recording")
long_square_sweeps.align_to_start_of_epoch("experiment")

# find the start and end time of the stimulus
# (treating the first sweep as representative)
stim_start_index, stim_end_index = get_stim_epoch(long_square_sweeps.i[0])
stim_start_time = long_square_sweeps.t[0][stim_start_index]
stim_end_time = long_square_sweeps.t[0][stim_end_index]

# build the extractors
spfx = SpikeFeatureExtractor(start=stim_start_time, end=stim_end_time)
sptfx = SpikeTrainFeatureExtractor(start=stim_start_time, end=stim_end_time)

# run the analysis and print out a few of the features
long_square_analysis = spa.LongSquareAnalysis(spfx, sptfx, subthresh_min_amp=-100.0)
data = long_square_analysis.analyze(long_square_sweeps)

fields_to_print = [
    'tau',
    'v_baseline',
    'input_resistance',
    'vm_for_sag',
    'fi_fit_slope',
    'sag',
    'rheobase_i'
```

(continues on next page)

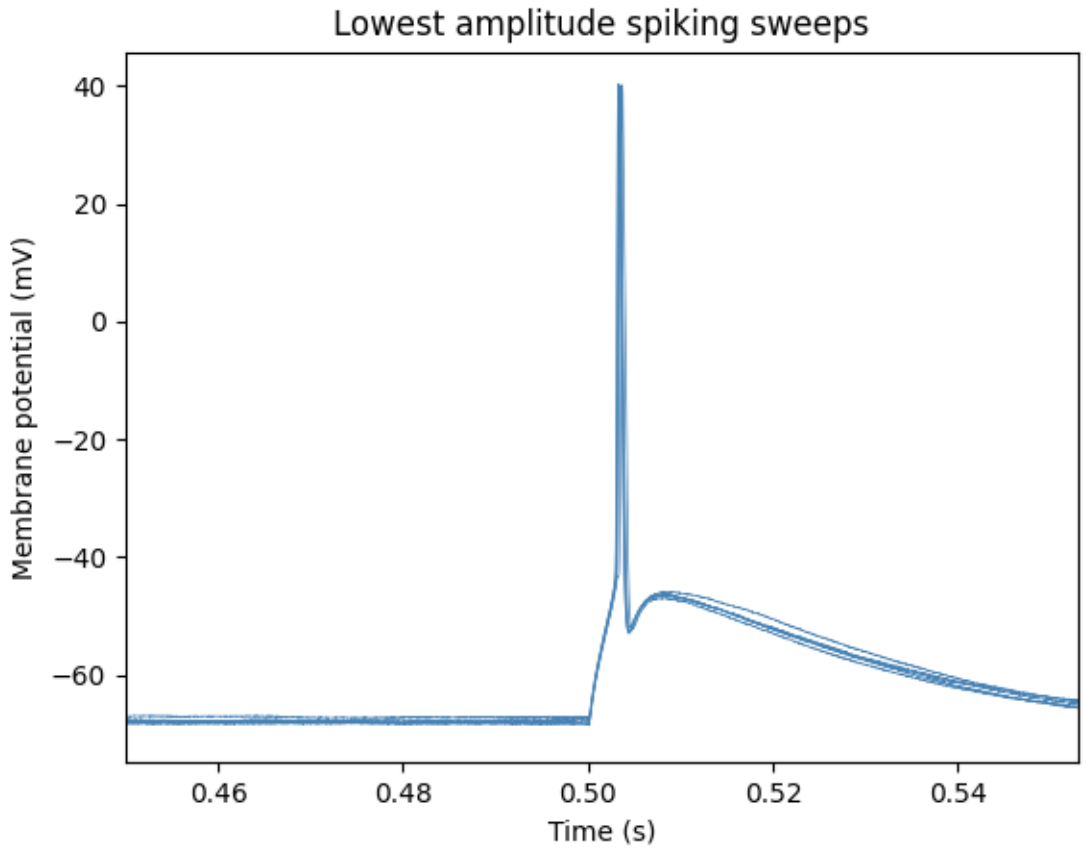
(continued from previous page)

```
]

for field in fields_to_print:
    print("%s: %s" % (field, str(data[field])))

# Plot stimulus amplitude vs. firing rate
spiking_sweeps = data['spiking_sweeps'].sort_values(by='stim_amp')
plt.plot(spiking_sweeps.stim_amp,
         spiking_sweeps.avg_rate)
plt.xlabel('Stimulus amplitude (pA)')
plt.ylabel('Average firing rate (Hz)')

plt.show()
```

Total running time of the script: (0 minutes 11.469 seconds)

6.1.4 Short Square Analysis

Calculate features of Short Square sweeps



Out:

```
/home/docs/checkouts/readthedocs.org/user_builds/ipfx/envs/latest/lib/python3.6/site-
↪packages/hdmf/spec/namespace.py:485: UserWarning: Ignoring cached namespace 'hdmf-
↪common' version 1.1.0 because version 1.3.0 is already loaded.
  % (ns['name'], ns['version'], self.__namespaces.get(ns['name'])['version']))
/home/docs/checkouts/readthedocs.org/user_builds/ipfx/envs/latest/lib/python3.6/site-
↪packages/hdmf/spec/namespace.py:485: UserWarning: Ignoring cached namespace 'core'␣
↪version 2.2.0 because version 2.2.5 is already loaded.
  % (ns['name'], ns['version'], self.__namespaces.get(ns['name'])['version']))
```

```
import os
import matplotlib.pyplot as plt
from ipfx.dataset.create import create_ephys_data_set
from ipfx.feature_extractor import (
    SpikeFeatureExtractor, SpikeTrainFeatureExtractor
)
from ipfx.stimulus_protocol_analysis import ShortSquareAnalysis
from ipfx.spike_features import estimate_adjusted_detection_parameters
from ipfx.epochs import get_stim_epoch
from ipfx.utilities import drop_failed_sweeps

# Download and access the experimental data from DANDI archive per instructions in␣
↪the documentation
# Example below will use an nwb file provided with the package

nwb_file = os.path.join(
    os.path.dirname(os.getcwd()),
    "data",
    "nwb2_H17.03.008.11.03.05.nwb"
)
# Create Ephys Data Set
data_set = create_ephys_data_set(nwb_file=nwb_file)

# Drop failed sweeps: sweeps with incomplete recording or failing QC criteria
drop_failed_sweeps(data_set)

short_square_table = data_set.filtered_sweep_table(
    stimuli=data_set.ontology.short_square_names
)
short_square_sweeps = data_set.sweep_set(short_square_table.sweep_number)

# Select epoch corresponding to the actual recording from the sweeps
# and align sweeps so that the experiment would start at the same time
short_square_sweeps.select_epoch("recording")
short_square_sweeps.align_to_start_of_epoch("experiment")

# find the start and end time of the stimulus
# (treating the first sweep as representative)
stim_start_index, stim_end_index = get_stim_epoch(short_square_sweeps.i[0])
stim_start_time = short_square_sweeps.t[0][stim_start_index]
stim_end_time = short_square_sweeps.t[0][stim_end_index]

# Estimate the dv cutoff and threshold fraction
```

(continues on next page)

(continued from previous page)

```
dv_cutoff, thresh_frac = estimate_adjusted_detection_parameters(
    short_square_sweeps.v,
    short_square_sweeps.t,
    stim_start_time,
    stim_start_time + 0.001
)

# Build the extractors
spfx = SpikeFeatureExtractor(
    start=stim_start_time, dv_cutoff=dv_cutoff, thresh_frac=thresh_frac
)
sptfx= SpikeTrainFeatureExtractor(start=stim_start_time, end=None)

# Run the analysis
short_square_analysis = ShortSquareAnalysis(spfx, sptfx)
results = short_square_analysis.analyze(short_square_sweeps)

# Plot the sweeps at the lowest amplitude that evoked the most spikes
for i, swp in enumerate(short_square_sweeps.sweeps):
    if i in results["common_amp_sweeps"].index:
        plt.plot(swp.t, swp.v, linewidth=0.5, color="steelblue")

# Set the plot limits to highlight where spikes are and axis labels
plt.xlim(stim_start_time - 0.05, stim_end_time + 0.05)
plt.xlabel("Time (s)")
plt.ylabel("Membrane potential (mV)")
plt.title("Lowest amplitude spiking sweeps")

plt.show()
```

Total running time of the script: (0 minutes 9.096 seconds)

6.2 Quality Control

6.2.1 Sweep QC Features

Estimate sweep QC features

Out:

```
/home/docs/checkouts/readthedocs.org/user_builds/ipfx/envs/latest/lib/python3.6/site-
↪packages/hdmf/spec/namespace.py:485: UserWarning: Ignoring cached namespace 'hdmf-
↪common' version 1.1.0 because version 1.3.0 is already loaded.
  % (ns['name'], ns['version'], self.__namespaces.get(ns['name'])['version']))
/home/docs/checkouts/readthedocs.org/user_builds/ipfx/envs/latest/lib/python3.6/site-
↪packages/hdmf/spec/namespace.py:485: UserWarning: Ignoring cached namespace 'core'␣
↪version 2.2.0 because version 2.2.5 is already loaded.
  % (ns['name'], ns['version'], self.__namespaces.get(ns['name'])['version']))
   sweep_number stimulus_units  ...  slow_noise_rms_mv  vm_delta_mv
0             5           Amps  ...           0.090198          NaN
1             6           Amps  ...           0.128340          NaN
2             7           Amps  ...           0.121554          NaN
3             8           Amps  ...           0.139625     0.029572
4             9           Amps  ...           0.349430     0.223572
```

(continues on next page)

(continued from previous page)

```
[5 rows x 20 columns]
[{'sweep_number': 5, 'passed': True, 'reasons': []}, {'sweep_number': 6, 'passed':␣
↪True, 'reasons': []}, {'sweep_number': 7, 'passed': True, 'reasons': []}, {'sweep_
↪number': 8, 'passed': True, 'reasons': []}, {'sweep_number': 9, 'passed': True,
↪'reasons': []}]
```

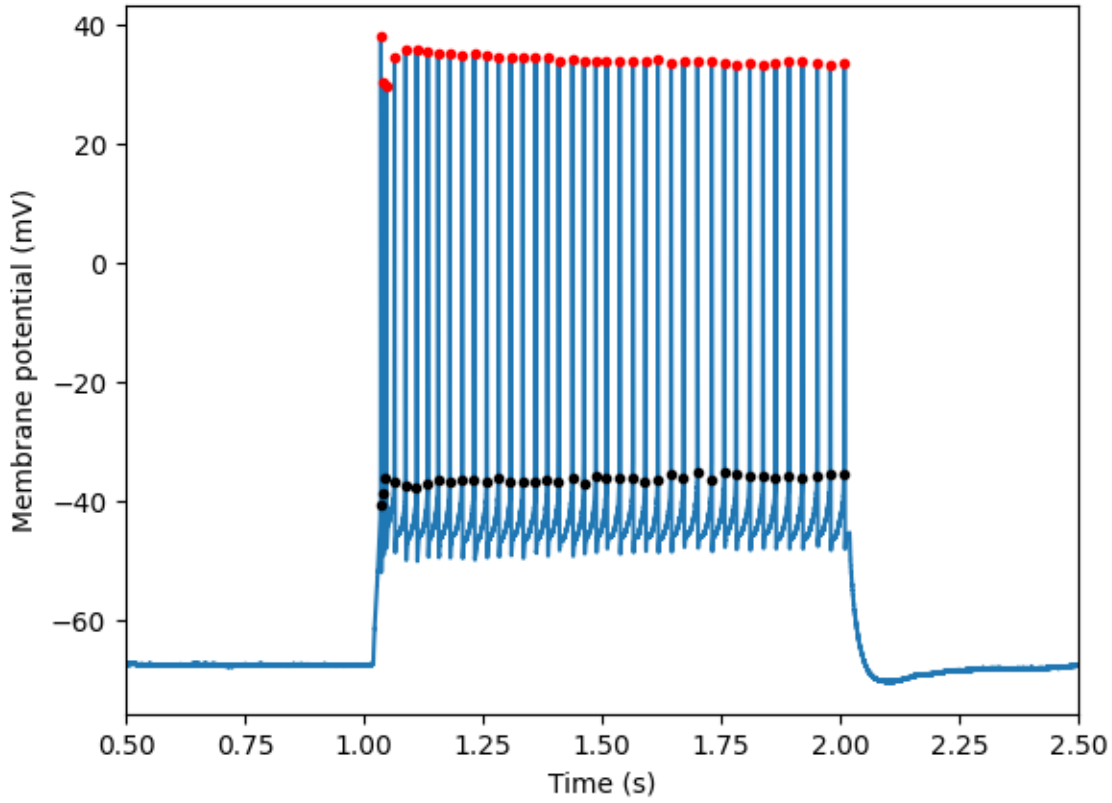
```
import os
import pandas as pd
from ipfx.dataset.create import create_ephys_data_set
from ipfx.qc_feature_extractor import sweep_qc_features

import ipfx.sweep_props as sweep_props
import ipfx.qc_feature_evaluator as qcp
from ipfx.stimulus import StimulusOntology


# Download and access the experimental data from DANDI archive per instructions in␣
↪the documentation
# Example below will use an nwb file provided with the package

nwb_file = os.path.join(
    os.path.dirname(os.getcwd()),
    "data",
    "nwb2_H17.03.008.11.03.05.nwb"
)
data_set = create_ephys_data_set(nwb_file=nwb_file)

# Compute sweep QC features
sweep_features = sweep_qc_features(data_set)

# Drop sweeps that failed to compute QC criteria
sweep_props.drop_tagged_sweeps(sweep_features)
sweep_props.remove_sweep_feature("tags", sweep_features)

stimulus_ontology = StimulusOntology.default()
qc_criteria = qcp.load_default_qc_criteria()

sweep_states = qcp.qc_sweeps(
    stimulus_ontology, sweep_features, qc_criteria
)

# print a few sweeps and states
print(pd.DataFrame(sweep_features).head())
print(sweep_states[0:len(pd.DataFrame(sweep_features).head())])
```

Total running time of the script: (0 minutes 8.234 seconds)

6.3 Spike Detection

6.3.1 Single sweep detection

Detect spikes for a single sweep



Out:

```
/home/docs/checkouts/readthedocs.org/user_builds/ipfx/envs/latest/lib/python3.6/site-
↪packages/hdmf/spec/namespace.py:485: UserWarning: Ignoring cached namespace 'hdmf-
↪common' version 1.1.0 because version 1.3.0 is already loaded.
  % (ns['name'], ns['version'], self.__namespaces.get(ns['name'])['version']))
/home/docs/checkouts/readthedocs.org/user_builds/ipfx/envs/latest/lib/python3.6/site-
↪packages/hdmf/spec/namespace.py:485: UserWarning: Ignoring cached namespace 'core'␣
↪version 2.2.0 because version 2.2.5 is already loaded.
  % (ns['name'], ns['version'], self.__namespaces.get(ns['name'])['version']))
```

```
import os
import matplotlib.pyplot as plt
```

(continues on next page)

(continued from previous page)

```
from ipfx.dataset.create import create_ephys_data_set
from ipfx.feature_extractor import SpikeFeatureExtractor

# Download and access the experimental data from DANDI archive per instructions in␣
↪the documentation
# Example below will use an nwb file provided with the package

nwb_file = os.path.join(
    os.path.dirname(os.getcwd()),
    "data",
    "nwb2_H17.03.008.11.03.05.nwb"
)

# Create data set from the nwb file and choose a sweeep
dataset = create_ephys_data_set(nwb_file=nwb_file)
sweep = dataset.sweep(sweep_number=39)

# Extract information about the spikes
ext = SpikeFeatureExtractor()
results = ext.process(t=sweep.t, v=sweep.v, i=sweep.i)

# Plot the results, showing two features of the detected spikes
plt.plot(sweep.t, sweep.v)
plt.plot(results["peak_t"], results["peak_v"], 'r.')
plt.plot(results["threshold_t"], results["threshold_v"], 'k.')

# Set the plot limits to highlight where spikes are and set axis labels
plt.xlim(0.5, 2.5)
plt.xlabel("Time (s)")
plt.ylabel("Membrane potential (mV)")

plt.show()
```

Total running time of the script: (0 minutes 1.398 seconds)

6.3.2 Spike Train Features

Detect spike train features

Out:

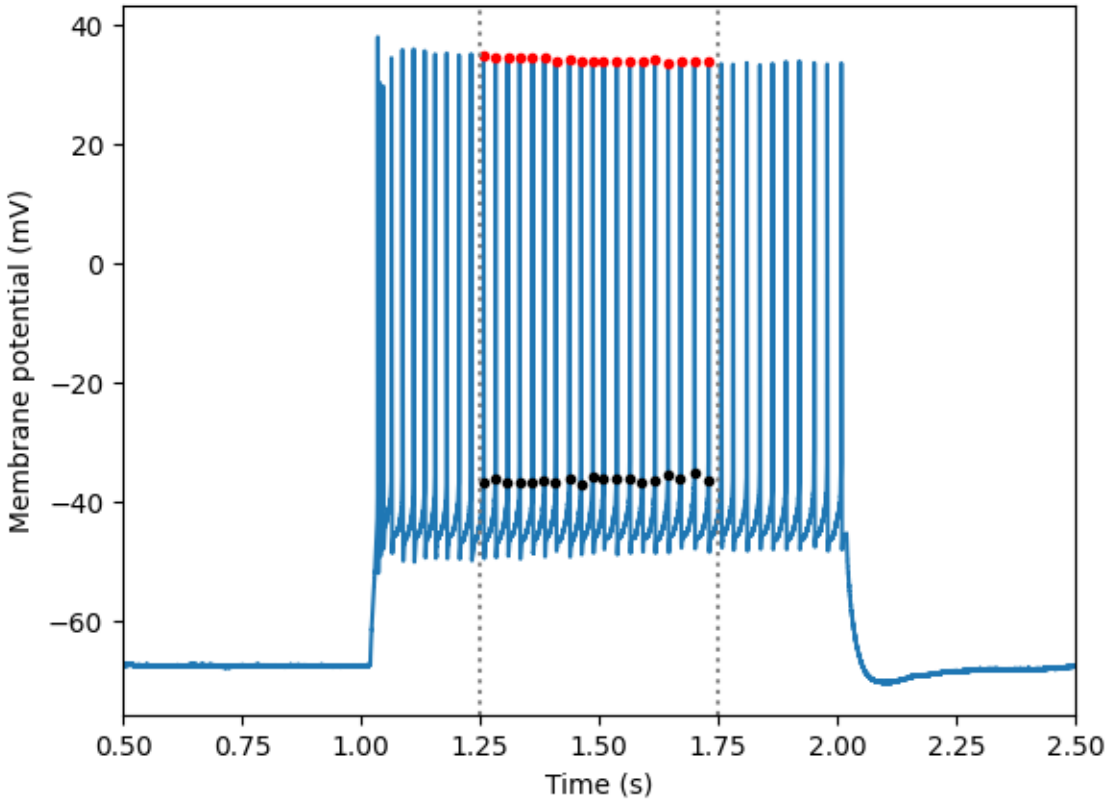
```
/home/docs/checkouts/readthedocs.org/user_builds/ipfx/envs/latest/lib/python3.6/site-
↪packages/hdmf/spec/namespace.py:485: UserWarning: Ignoring cached namespace 'hdmf-
↪common' version 1.1.0 because version 1.3.0 is already loaded.
  % (ns['name'], ns['version'], self.__namespaces.get(ns['name'])['version']))
/home/docs/checkouts/readthedocs.org/user_builds/ipfx/envs/latest/lib/python3.6/site-
↪packages/hdmf/spec/namespace.py:485: UserWarning: Ignoring cached namespace 'core'␣
↪version 2.2.0 because version 2.2.5 is already loaded.
  % (ns['name'], ns['version'], self.__namespaces.get(ns['name'])['version']))
{'adapt': 0.02422398922048236, 'latency': 0.01540000000000008, 'isi_cv': 0.
↪20699907608695922, 'mean_isi': 0.024953333333333327, 'median_isi': 0.
↪026100000000000012, 'first_isi': 0.0043400000000000105, 'avg_rate': 40.0}
```

```
import os
from ipfx.dataset.create import create_ephys_data_set
from ipfx.feature_extractor import (
    SpikeFeatureExtractor, SpikeTrainFeatureExtractor
)

# Download and access the experimental data from DANDI archive per instructions in␣
↪the documentation
# Example below will use an nwb file provided with the package

nwb_file = os.path.join(
    os.path.dirname(os.getcwd()),
    "data",
    "nwb2_H17.03.008.11.03.05.nwb"
)

# Create data set from the nwb file and choose a sweeep
dataset = create_ephys_data_set(nwb_file=nwb_file)
sweep = dataset.sweep(sweep_number=39)

# Instantiate feature extractor for spikes
start, end = 1.02, 2.02
sfx = SpikeFeatureExtractor(start=start, end=end)

# Run feature extractor returning a table of spikes and their features
spikes_df = sfx.process(t=sweep.t, v=sweep.v, i=sweep.i)

# Instantiate Spike Train feature extractor
stfx = SpikeTrainFeatureExtractor(start=start, end=end)

# Run to produce features of a spike train
spike_train_results = stfx.process(
    t=sweep.t,
    v=sweep.v,
    i=sweep.i,
    spikes_df=spikes_df
)

print(spike_train_results)
```

Total running time of the script: (0 minutes 1.257 seconds)

6.3.3 Single sweep detection with analysis window

Detect spikes for a single sweep in a specified time window



Out:

```
/home/docs/checkouts/readthedocs.org/user_builds/ipfx/envs/latest/lib/python3.6/site-
↪packages/hdmf/spec/namespace.py:485: UserWarning: Ignoring cached namespace 'hdmf-
↪common' version 1.1.0 because version 1.3.0 is already loaded.
  % (ns['name'], ns['version'], self.__namespaces.get(ns['name'])['version']))
/home/docs/checkouts/readthedocs.org/user_builds/ipfx/envs/latest/lib/python3.6/site-
↪packages/hdmf/spec/namespace.py:485: UserWarning: Ignoring cached namespace 'core'␣
↪version 2.2.0 because version 2.2.5 is already loaded.
  % (ns['name'], ns['version'], self.__namespaces.get(ns['name'])['version']))
```

```
import os
import matplotlib.pyplot as plt
from ipfx.dataset.create import create_ephys_data_set
from ipfx.feature_extractor import SpikeFeatureExtractor

# Download and access the experimental data from DANDI archive per instructions in␣
↪the documentation
# Example below will use an nwb file provided with the package
```

(continues on next page)

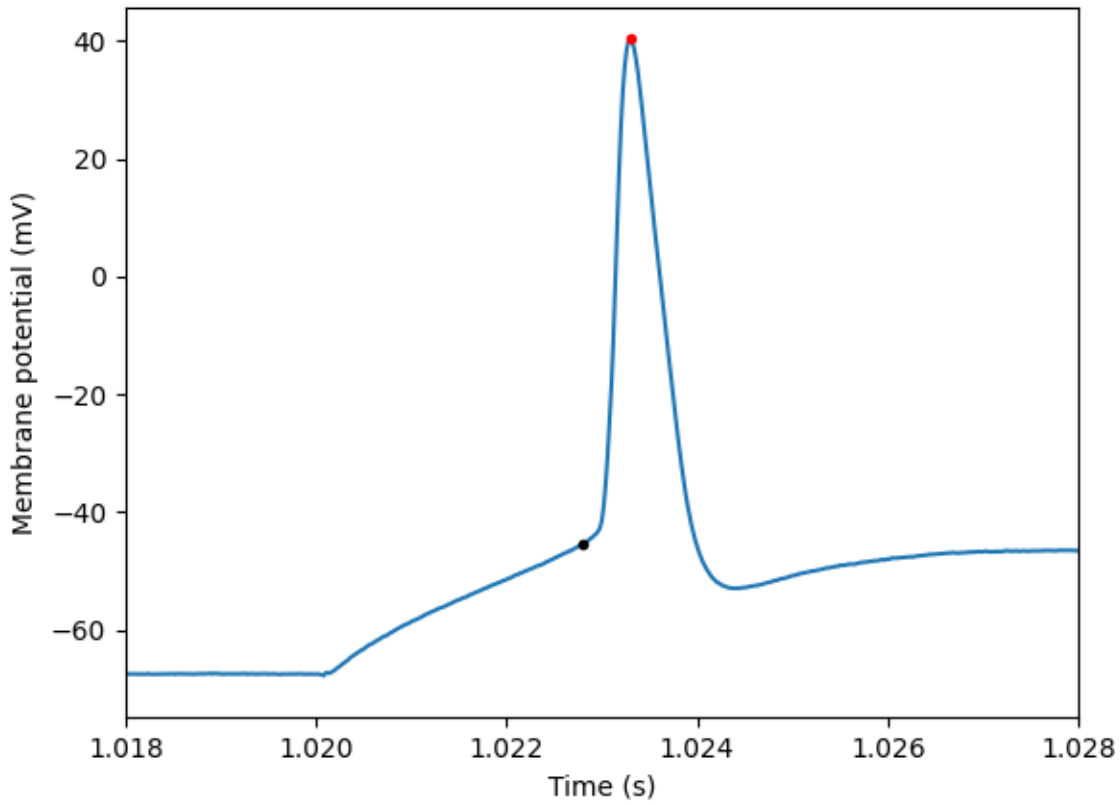
(continued from previous page)

```
nwb_file = os.path.join(
    os.path.dirname(os.getcwd()),
    "data",
    "nwb2_H17.03.008.11.03.05.nwb"
)

# Create data set from the nwb file and choose a sweeep
dataset = create_ephys_data_set(nwb_file=nwb_file)
sweep = dataset.sweep(sweep_number=39)

# Configure the extractor to just detect spikes in the middle of the step
ext = SpikeFeatureExtractor(start=1.25, end=1.75)
results = ext.process(t=sweep.t, v=sweep.v, i=sweep.i)

# Plot the results, showing two features of the detected spikes
plt.plot(sweep.t, sweep.v)
plt.plot(results["peak_t"], results["peak_v"], 'r.')
plt.plot(results["threshold_t"], results["threshold_v"], 'k.')

# Set the plot limits to highlight where spikes are and axis labels
plt.xlim(0.5, 2.5)
plt.xlabel("Time (s)")
plt.ylabel("Membrane potential (mV)")

# Show the analysis window on the plot
plt.axvline(1.25, linestyle="dotted", color="gray")
plt.axvline(1.75, linestyle="dotted", color="gray")

plt.show()
```

Total running time of the script: (0 minutes 1.450 seconds)

6.3.4 Estimate Spike Detection Parameters

Estimate spike detection parameters



Out:

```
/home/docs/checkouts/readthedocs.org/user_builds/ipfx/checkouts/latest/docs/gallery/
↪spikes_examples/estimate_params.py:25: VisibleDeprecationWarning: Function create_
↪ephys_data_set is deprecated. Instead of using ipfx.data_set_utils.create_data_set,␣
↪use ipfx.dataset.create.create_ephys_data_set
  data_set = create_data_set(nwb_file=nwb_file)
/home/docs/checkouts/readthedocs.org/user_builds/ipfx/envs/latest/lib/python3.6/site-
↪packages/hdmf/spec/namespace.py:485: UserWarning: Ignoring cached namespace 'hdmf-
↪common' version 1.1.0 because version 1.3.0 is already loaded.
  % (ns['name'], ns['version'], self.__namespaces.get(ns['name'])['version']))
/home/docs/checkouts/readthedocs.org/user_builds/ipfx/envs/latest/lib/python3.6/site-
↪packages/hdmf/spec/namespace.py:485: UserWarning: Ignoring cached namespace 'core'␣
↪version 2.2.0 because version 2.2.5 is already loaded.
  % (ns['name'], ns['version'], self.__namespaces.get(ns['name'])['version']))
```

```
import os
from ipfx.data_set_utils import create_data_set
from ipfx.spike_features import estimate_adjusted_detection_parameters
from ipfx.feature_extractor import SpikeFeatureExtractor
```

(continues on next page)

(continued from previous page)

```
from ipfx.utilities import drop_failed_sweeps
import matplotlib.pyplot as plt

# Download and access the experimental data from DANDI archive per instructions in␣
↪the documentation
# Example below will use an nwb file provided with the package

nwb_file = os.path.join(
    os.path.dirname(os.getcwd()),
    "data",
    "nwb2_H17.03.008.11.03.05.nwb"
)

# Create data set from the nwb file and find the short squares
data_set = create_data_set(nwb_file=nwb_file)

# Drop failed sweeps: sweeps with incomplete recording or failing QC criteria
drop_failed_sweeps(data_set)

short_square_table = data_set.filtered_sweep_table(stimuli=["Short Square"])
ssq_set = data_set.sweep_set(short_square_table.sweep_number)

# estimate the dv cutoff and threshold fraction
dv_cutoff, thresh_frac = estimate_adjusted_detection_parameters(
    ssq_set.v, ssq_set.t, 1.02, 1.021
)

# detect spikes  in a given sweep number
sweep_number = 16
sweep = data_set.sweep(sweep_number)
ext = SpikeFeatureExtractor(dv_cutoff=dv_cutoff, thresh_frac=thresh_frac)
spikes = ext.process(t=sweep.t, v=sweep.v, i=sweep.i)

# and plot them
plt.plot(sweep.t, sweep.v)
plt.plot(spikes["peak_t"], spikes["peak_v"], 'r.')
plt.plot(spikes["threshold_t"], spikes["threshold_v"], 'k.')
plt.xlim(1.018, 1.028)
plt.xlabel('Time (s)')
plt.ylabel('Membrane potential (mV)')
plt.show()
```

Total running time of the script: (0 minutes 8.732 seconds)

CHAPTER 7

QC and Analysis Pipeline

The IPFX package is utilized in the Allen Institute's electrophysiology processing pipeline that is being used to generate publicly available data.

Given the nwb2 file with electrphysiological recording for a single cell the pipeline will perform the following steps (run with the corresponding executables):

1. **Compute QC features (`ipfx.bin.run_sweep_extraction`)** Extract sweeps and their metadata from the nwb file Computed QC features for the cell as a whole and also for each individual sweeps
2. **Perform QC checks (`ipfx.bin.run_qc`)** Check QC criteria on QC features at the level of the entire cell and for individual sweeps Tag cell, sweeps failing QC criteria
3. **Compute intrinsic features (`ipfx.bin.run_feature_extraction`)** Compute features of spikes, spike trains, as well as stimulus specific features
4. **Attach metadata (`ipfx.attach_metadata`)** Add ancillary information about an experiment to the output nwb2 file

Each executable defines a schema for the input parameters specified in the <input.json> and can be invoked as:

```
python -m <executable> --input_json <path/to/input.json> --output_json <path/to/
↪output.json>
```

where <executable> stands for the executables listed in the above pipeline steps.

Running the pipeline requires two additional pieces of information:

1. Stimulus ontology that maps names of sweeps in the input nwb2 file to the stimulus types known to `ipfx`
2. QC criteria that specify the acceptable values for the computed QC features

If not explicitly provided, the pipeline will invoke the default values from the ipfx.defaults folder

CHAPTER 8

ipfx package

8.1 Subpackages

8.1.1 ipfx.attach_metadata package

Subpackages

ipfx.attach_metadata.sink package

Submodules

ipfx.attach_metadata.sink.dandi_yaml_sink module

Sink for metadata intended to be uploaded to DANDI

class ipfx.attach_metadata.sink.dandi_yaml_sink.**DandiYamlSink**

Bases: *ipfx.attach_metadata.sink.metadata_sink.MetadataSink*

Sink specialized for writing data to a DANDI-compatible YAML file.

Attributes

supported_cell_fields The names of each cell-level field supported by this sink.

supported_sweep_fields The names of each sweep-level field supported by this sink.

targets A sequence of preregistered targets.

Methods

toctree references unknown document 'ipfx.attach_metadata.sink.dandi_yaml_sink.DandiYamlSink.register'

toctree references unknown document 'ipfx.attach_metadata.sink.dandi_yaml_sink.DandiYamlSink.register_target'

toctree references unknown document 'ipfx.attach_metadata.sink.dandi_yaml_sink.DandiYamlSink.register_targets'

toctree references unknown document 'ipfx.attach_metadata.sink.dandi_yaml_sink.DandiYamlSink.serialize'

register(self, name, value, sweep_id, ...)	Attaches a named piece of metadata to this sink's internal store.
register_target(self, target, Any])	Preregister a single target specification on this sink.
register_targets(self, targets, Any], ...)	Configures targets for this sink.
serialize(self, targets, Any], ...)	Writes this sink's data to an external target or targets.

register (*self*, *name:str*, *value:Any*, *sweep_id:Union[int*, *NoneType]=None*)
Attaches a named piece of metadata to this sink's internal store. Should dispatch to a protected method which carries out appropriate validations and transformations.

Parameters

name [the well-known name of the metadata]

value [the value of the metadata (before any required transformations)]

sweep_id [If provided, this will be interpreted as sweep-level] metadata and sweep_id will be used to identify the sweep to which value ought to be attached. If None, this will be interpreted as cell-level metadata

Raises

ValueError [An argued piece of metadata is not supported by this sink]

serialize (*self, targets:Union[Dict[str, Any], Collection[Dict[str, Any]], NoneType]=None*)
Writes this sink's data to an external target or targets. Does not modify this sink.

Parameters

targets [If provided, these targets will be written to. Otherwise,] write to targets previously defined by register_target.

supported_cell_fields
The names of each cell-level field supported by this sink.

supported_sweep_fields
The names of each sweep-level field supported by this sink.

targets
A sequence of preregistered targets. Calling serialize with no arguments will write to these.

ipfx.attach_metadata.sink.metadata_sink module

Base class for metadata sinks, which serve as a staging ground for metadata attachment.

class ipfx.attach_metadata.sink.metadata_sink.**MetadataSink**
Bases: abc.ABC

Abstract(ish) interface for metadata sinks.

Attributes

supported_cell_fields The names of each cell-level field supported by this sink.

supported_sweep_fields The names of each sweep-level field supported by this sink.

targets A sequence of preregistered targets.

Methods

toctree references unknown document 'ipfx.attach_metadata.sink.metadata_sink.MetadataSink.register'

toctree references unknown document 'ipfx.attach_metadata.sink.metadata_sink.MetadataSink.register_target'

toctree references unknown document 'ipfx.attach_metadata.sink.metadata_sink.MetadataSink.register_targets'

toctree references unknown document 'ipfx.attach_metadata.sink.metadata_sink.MetadataSink.serialize'

register(self, name, value, sweep_id, ...)	Attaches a named piece of metadata to this sink's internal store.
register_target(self, target, Any])	Preregister a single target specification on this sink.
register_targets(self, targets, Any], ...)	Configures targets for this sink.
serialize(self, targets, Any], ...)	Writes this sink's data to an external target or targets.

register (*self*, *name:str*, *value:Any*, *sweep_id:Union[int*, *NoneType]=None*)
Attaches a named piece of metadata to this sink's internal store. Should dispatch to a protected method which carries out appropriate validations and transformations.

Parameters

name [the well-known name of the metadata]

value [the value of the metadata (before any required transformations)]

sweep_id [If provided, this will be interpreted as sweep-level] metadata and sweep_id will be used to identify the sweep to which value ought to be attached. If None, this will be interpreted as cell-level metadata

Raises

ValueError [An argued piece of metadata is not supported by this sink]

register_target (*self, target:Dict[str, Any]*)
Preregister a single target specification on this sink.

Parameters

target [Must provide parameters required for serialization]

register_targets (*self, targets:Union[Dict[str, Any], Collection[Dict[str, Any]]]*)
Configures targets for this sink. Calls to serialize (without new targets supplied) will write to these targets.

Parameters

targets [Configure these targets. Each should be a dictionary which] passes this class's validate_target method

serialize (*self, targets:Union[Dict[str, Any], Collection[Dict[str, Any]], NoneType]=None*)
Writes this sink's data to an external target or targets. Does not modify this sink.

Parameters

targets [If provided, these targets will be written to. Otherwise,] write to targets previously defined by register_target.

supported_cell_fields
The names of each cell-level field supported by this sink.

supported_sweep_fields
The names of each sweep-level field supported by this sink.

targets
A sequence of preregistered targets. Calling serialize with no arguments will write to these.

ipfx.attach_metadata.sink.nwb2_sink module

Sink for appending to an NWBFile (not in place)

class ipfx.attach_metadata.sink.nwb2_sink.**Nwb2Sink**(*nwb_path: Union[str, pathlib.Path, None]*)

Bases: *ipfx.attach_metadata.sink.metadata_sink.MetadataSink*

A metadata sink which modifies an NWBFile (in-memory representation of an NWB 2 file)

Attributes

supported_cell_fields The names of each cell-level field supported by this sink.

supported_sweep_fields The names of each sweep-level field supported by this sink.

targets A sequence of preregistered targets.

Methods

toctree references unknown document 'ipfx.attach_metadata.sink.nwb2_sink.Nwb2Sink.register'

toctree references unknown document 'ipfx.attach_metadata.sink.nwb2_sink.Nwb2Sink.register_target'

toctree references unknown document 'ipfx.attach_metadata.sink.nwb2_sink.Nwb2Sink.register_targets'

toctree references unknown document 'ipfx.attach_metadata.sink.nwb2_sink.Nwb2Sink.serialize'

register(self, name, value, sweep_id, ...)	Attaches a named piece of metadata to this sink's internal store.
register_target(self, target, Any])	Preregister a single target specification on this sink.
register_targets(self, targets, Any], ...)	Configures targets for this sink.
serialize(self, targets, Any], ...)	Writes this sink's data to an external target or targets.

register(*self*, *name:str*, *value:Any*, *sweep_id:Union[int, NoneType]=None*)
Attaches a named piece of metadata to this sink's internal store. Should dispatch to a protected method which carries out appropriate validations and transformations.

Parameters

name [the well-known name of the metadata]

value [the value of the metadata (before any required transformations)]

sweep_id [If provided, this will be interpreted as sweep-level] metadata and sweep_id will be used to identify the sweep to which value ought to be attached. If None, this will be interpreted as cell-level metadata

Raises

ValueError [An argued piece of metadata is not supported by this sink]

serialize(*self, targets:Union[Dict[str, Any], Collection[Dict[str, Any]], NoneType]=None*)
Writes this sink's data to an external target or targets. Does not modify this sink.

Parameters

targets [If provided, these targets will be written to. Otherwise,] write to targets previously defined by register_target.

supported_cell_fields
The names of each cell-level field supported by this sink.

supported_sweep_fields
The names of each sweep-level field supported by this sink.

targets
A sequence of preregistered targets. Calling serialize with no arguments will write to these.

ipfx.attach_metadata.sink.nwb2_sink.**set_container_sources**(*container:hdmf.container.AbstractContainer*, *source:str*)
Traverse an NWBFile starting at a given container, setting the container_source attribute inplace on each container.

Parameters

container [container_source will be set on this object as well as on] each of its applicable children.

source [The new value of container source]

Module contents

Sinks (in-memory staging areas) for metadata

ipfx.attach_metadata.sink.**default_sink_kinds**() → Dict[str, Type[ipfx.attach_metadata.sink.metadata_sink.MetadataSink]]
Maps string names to metadata sink classes

Module contents

8.1.2 ipfx.bin package

Submodules

ipfx.bin.generate_fx_input module

ipfx.bin.generate_fx_input.**generate_fx_input**(*se_input*, *se_output*, *cell_dir*, *plot_figures=False*)

ipfx.bin.generate_fx_input.**main**()
Usage: > python generate_fx_input.py –specimen_id SPECIMEN_ID –cell_dir CELL_DIR > python generate_fx_input.py –input_nwb_file INPUT_NWB_FILE –cell_dir CELL_DIR

ipfx.bin.generate_pipeline_input module

ipfx.bin.generate_pipeline_input.**generate_pipeline_input**(*cell_dir=None*, *specimen_id=None*, *input_nwb_file=None*, *plot_figures=False*, *qc_fig_dirname='qc_figs'*)

ipfx.bin.generate_pipeline_input.**main**()
Usage: > python generate_pipeline_input.py –specimen_id SPECIMEN_ID –cell_dir CELL_DIR > python generate_pipeline_input.py –input_nwb_file INPUT_NWB_FILE –cell_dir CELL_DIR

ipfx.bin.generate_qc_input module

ipfx.bin.generate_qc_input.**generate_qc_input**(*se_input*, *se_output*)

ipfx.bin.generate_qc_input.**main**()
Usage: > python generate_qc_input.py –specimen_id SPECIMEN_ID –cell_dir CELL_DIR > python generate_qc_input.py –input_nwb_file input_nwb_file –cell_dir CELL_DIR

ipfx.bin.generate_se_input module

ipfx.bin.generate_se_input.**generate_se_input**(*cell_dir*, *specimen_id=None*, *input_nwb_file=None*)

ipfx.bin.generate_se_input.**main**()
Usage: > python generate_se_input.py –specimen_id SPECIMEN_ID –cell_dir CELL_DIR > python generate_se_input.py –input_nwb_file input_nwb_file –cell_dir CELL_DIR

ipfx.bin.generate_se_input.**parse_args**()

ipfx.bin.get_fx_output module

ipfx.bin.get_fx_output.**get_fx_output_json**(*specimen_id*)
Find in LIMS the full path to the json output of the feature extraction module If more than one file exists, then chose the latest version

Parameters

specimen_id

Returns

file_name: string

ipfx.bin.get_fx_output.**main**()
Usage: $python get_fx_output.py –input_file IN_FILE –output_file OUT_FILE IN_FILE: name of the input file including a single column with the header ‘specimen_id’ OUT_FILE: name of the output file that includes columns ‘specimen_id’ and ‘fx_output_json’

ipfx.bin.get_fx_output.**parse_args**()

ipfx.bin.make_stimulus_ontology module

ipfx.bin.make_stimulus_ontology.**main**()

ipfx.bin.make_stimulus_ontology.**make_default_stimulus_ontology**()

ipfx.bin.make_stimulus_ontology.**make_stimulus_ontology**(*stims*)

ipfx.bin.make_stimulus_ontology.**make_stimulus_ontology_from_lims**(*file_name*)

ipfx.bin.mcc_get_settings module

Code for interfacing with the Multi Clamp Commander application from Axon/Molecular Devices

class ipfx.bin.mcc_get_settings.**DataGatherer**
Bases: object

Collect data from all available MCC amplifiers

Methods

toctree references unknown document 'ipfx.bin.mcc_get_settings.DataGatherer.getData'

getData(self, mcc)	Return all MCC settings for all amplifiers as dictionary

getData (*self*, *mcc*)
Return all MCC settings for all amplifiers as dictionary

class ipfx.bin.mcc_get_settings.**MultiClampControl** (*dllPath=None*)
Bases: object

Class for interacting with the MultiClamp Commander from Axon/Molecular Devices

Usage:

‘‘‘ Inline literal start-string without end-string.

Inline interpreted text or phrase reference start-string without end-string.

import mcc

m = mcc.MultiClampControl() UIDs = m.getUIDs() # output all found amplifiers m.selectUniqueID(next(iter(UIDs))) # select the first one (implicitly done by __init__) clampMode = m.GetMode() # return the clamp mode if m._handleError()[0]:

Unexpected indentation.

handle error pass

Definition list ends without a blank line; unexpected unindent.

‘‘‘

Inline literal start-string without end-string.

Inline interpreted text or phrase reference start-string without end-string.

Methods

toctree references unknown document 'ipfx.bin.mcc_get_settings.MultiClampControl.CreateObject'

toctree references unknown document 'ipfx.bin.mcc_get_settings.MultiClampControl.DestroyObject'

toctree references unknown document 'ipfx.bin.mcc_get_settings.MultiClampControl.FindFirstMultiClamp'

toctree references unknown document 'ipfx.bin.mcc_get_settings.MultiClampControl.FindNextMultiClamp'

toctree references unknown document 'ipfx.bin.mcc_get_settings.MultiClampControl.getUIDs'

CreateObject(self)	run this first to create self.hMCCmsg
DestroyObject(self)	Do this last if you do it
FindFirstMultiClamp(self)	Find the first available amplifier and return its parameter
FindNextMultiClamp(self)	Find the next available amplifier and return its parameter
getUIDs(self)	Return a list of all amplifier UIDs

AutoBridgeBal	
AutoFastComp	
AutoLeakSub	
AutoOutputZero	
AutoPipetteOffset	
AutoSlowComp	
AutoWholeCellComp	
BuildErrorText	
Buzz	
CheckAPIVersion	
ClearMinus	
ClearPlus	
GetBridgeBalEnable	
GetBridgeBalResist	
GetBuzzDuration	
GetFastCompCap	
GetFastCompTau	
GetHolding	
GetHoldingEnable	
GetLeakSubEnable	
GetLeakSubResist	
GetMeterIrmsEnable	
GetMeterResistEnable	
GetMeterValue	
GetMode	
GetModeSwitchEnable	
GetNeutralizationCap	
GetNeutralizationEnable	
GetOscKillerEnable	
GetOutputZeroAmplitude	
GetOutputZeroEnable	
GetPipetteOffset	
GetPrimarySignal	
GetPrimarySignalGain	
GetPrimarySignalHPF	
GetPrimarySignalLPF	
GetPulseAmplitude	
GetPulseDuration	
GetRsCompBandwidth	
GetRsCompCorrection	
GetRsCompEnable	
GetRsCompPrediction	

Continued on next page

Table 6 – continued from previous page

GetScopeSignalLPF	
GetSecondarySignal	
GetSecondarySignalGain	
GetSecondarySignalLPF	
GetSlowCompCap	
GetSlowCompTau	
GetSlowCompTauX20Enable	
GetSlowCurrentInjEnable	
GetSlowCurrentInjLevel	
GetSlowCurrentInjSettlingTime	
GetTestSignalAmplitude	
GetTestSignalEnable	
GetTestSignalFrequency	
GetWholeCellCompCap	
GetWholeCellCompEnable	
GetWholeCellCompResist	
GetZapDuration	
Pulse	
QuickSelectButton	
Reset	
SelectMultiClamp	
SetBridgeBalEnable	
SetBridgeBalResist	
SetBuzzDuration	
SetFastCompCap	
SetFastCompTau	
SetHolding	
SetHoldingEnable	
SetLeakSubEnable	
SetLeakSubResist	
SetMeterIrmsEnable	
SetMeterResistEnable	
SetMode	
SetModeSwitchEnable	
SetNeutralizationCap	
SetNeutralizationEnable	
SetOscKillerEnable	
SetOutputZeroAmplitude	
SetOutputZeroEnable	
SetPipetteOffset	
SetPrimarySignal	
SetPrimarySignalGain	
SetPrimarySignalHPF	
SetPrimarySignalLPF	
SetPulseAmplitude	
SetPulseDuration	
SetRsCompBandwidth	
SetRsCompCorrection	
SetRsCompEnable	
SetRsCompPrediction	

Continued on next page

Table 6 – continued from previous page

SetScopeSignalLPF	
SetSecondarySignal	
SetSecondarySignalGain	
SetSecondarySignalLPF	
SetSlowCompCap	
SetSlowCompTau	
SetSlowCompTauX20Enable	
SetSlowCurrentInjEnable	
SetSlowCurrentInjLevel	
SetSlowCurrentInjSettlingTime	
SetTestSignalAmplitude	
SetTestSignalEnable	
SetTestSignalFrequency	
SetTimeOut	
SetWholeCellCompCap	
SetWholeCellCompEnable	
SetWholeCellCompResist	
SetZapDuration	
ToggleAlwaysOnTop	
ToggleResize	
Zap	
cleanup	
getChannel	
getSerial	
selectUniqueID	

AutoBridgeBal(*self*)

AutoFastComp(*self*)

AutoLeakSub(*self*)

AutoOutputZero(*self*)

AutoPipetteOffset(*self*)

AutoSlowComp(*self*)

AutoWholeCellComp(*self*)

BuildErrorText(*self*)

Buzz(*self*)

CheckAPIVersion(*self*)

ClearMinus(*self*)

ClearPlus(*self*)

CreateObject(*self*)
run this first to create self.hMCCmsg

DestroyObject(*self*)
Do this last if you do it

FindFirstMultiClamp(*self*)
Find the first available amplifier and return its parameter

FindNextMultiClamp (*self*)
Find the next available amplifier and return its parameter

GetBridgeBalEnable (*self*)

GetBridgeBalResist (*self*)

GetBuzzDuration (*self*)

GetFastCompCap (*self*)

GetFastCompTau (*self*)

GetHolding (*self*)

GetHoldingEnable (*self*)

GetLeakSubEnable (*self*)

GetLeakSubResist (*self*)

GetMeterIrmsEnable (*self*)

GetMeterResistEnable (*self*)

GetMeterValue (*self*, *u*)

GetMode (*self*)

GetModeSwitchEnable (*self*)

GetNeutralizationCap (*self*)

GetNeutralizationEnable (*self*)

GetOscKillerEnable (*self*)

GetOutputZeroAmplitude (*self*)

GetOutputZeroEnable (*self*)

GetPipetteOffset (*self*)

GetPrimarySignal (*self*)

GetPrimarySignalGain (*self*)

GetPrimarySignalHPF (*self*)

GetPrimarySignalLPF (*self*)

GetPulseAmplitude (*self*)

GetPulseDuration (*self*)

GetRsCompBandwidth (*self*)

GetRsCompCorrection (*self*)

GetRsCompEnable (*self*)

GetRsCompPrediction (*self*)

GetScopeSignalLPF (*self*)

GetSecondarySignal (*self*)

GetSecondarySignalGain (*self*)

GetSecondarySignalLPF (*self*)

GetSlowCompCap (*self*)

GetSlowCompTau (*self*)

GetSlowCompTauX20Enable (*self*)

GetSlowCurrentInjEnable (*self*)

GetSlowCurrentInjLevel (*self*)

GetSlowCurrentInjSettlingTime (*self*)

GetTestSignalAmplitude (*self*)

GetTestSignalEnable (*self*)

GetTestSignalFrequency (*self*)

GetWholeCellCompCap (*self*)

GetWholeCellCompEnable (*self*)

GetWholeCellCompResist (*self*)

GetZapDuration (*self*)

Pulse (*self*)

QuickSelectButton (*self*, *u*)

Reset (*self*)

SelectMultiClamp (*self*, *uModel*, *_pszSerialNum*, *uCOMPortID*, *uDeviceID*, *uChannelID*)

SetBridgeBalEnable (*self*, *b*)

SetBridgeBalResist (*self*, *d*)

SetBuzzDuration (*self*, *d*)

SetFastCompCap (*self*, *d*)

SetFastCompTau (*self*, *d*)

SetHolding (*self*, *d*)

SetHoldingEnable (*self*, *b*)

SetLeakSubEnable (*self*, *b*)

SetLeakSubResist (*self*, *d*)

SetMeterIrmsEnable (*self*, *b*)

SetMeterResistEnable (*self*, *b*)

SetMode (*self*, *u*)

SetModeSwitchEnable (*self*, *b*)

SetNeutralizationCap (*self*, *d*)

SetNeutralizationEnable (*self*, *b*)

SetOscKillerEnable (*self*, *b*)

SetOutputZeroAmplitude (*self*, *d*)

SetOutputZeroEnable (*self*, *b*)

SetPipetteOffset (*self*, *d*)

SetPrimarySignal (*self*, *u*)

SetPrimarySignalGain (*self*, *d*)

SetPrimarySignalHPF (*self*, *d*)

SetPrimarySignalLPF (*self*, *d*)

SetPulseAmplitude (*self*, *d*)

SetPulseDuration (*self*, *d*)

SetRsCompBandwidth (*self*, *d*)

SetRsCompCorrection (*self*, *d*)

SetRsCompEnable (*self*, *b*)

SetRsCompPrediction (*self*, *d*)

SetScopeSignalLPF (*self*, *d*)

SetSecondarySignal (*self*, *u*)

SetSecondarySignalGain (*self*, *d*)

SetSecondarySignalLPF (*self*, *d*)

SetSlowCompCap (*self*, *d*)

SetSlowCompTau (*self*, *d*)

SetSlowCompTauX20Enable (*self*, *b*)

SetSlowCurrentInjEnable (*self*, *b*)

SetSlowCurrentInjLevel (*self*, *d*)

SetSlowCurrentInjSettlingTime (*self*, *d*)

SetTestSignalAmplitude (*self*, *d*)

SetTestSignalEnable (*self*, *b*)

SetTestSignalFrequency (*self*, *d*)

SetTimeOut (*self*, *u*)

SetWholeCellCompCap (*self*, *d*)

SetWholeCellCompEnable (*self*, *b*)

SetWholeCellCompResist (*self*, *d*)

SetZapDuration (*self*, *d*)

ToggleAlwaysOnTop (*self*)

ToggleResize (*self*)

Zap (*self*)

cleanup (*self*)

getChannel (*self*)

getSerial (*self*)

getUIDs (*self*)
Return a list of all amplifier UIDs

selectUniqueID(*self*, *uniqueID*)

class ipfx.bin.mcc_get_settings.**SettingsFetcher**(*settingsFile*)
Bases: watchdog.events.RegexMatchingEventHandler

Attributes

case_sensitive (Read-only)

ignore_directories (Read-only)

ignore_regexes (Read-only)

regexes (Read-only)

Methods

toctree references unknown document 'ipfx.bin.mcc_get_settings.SettingsFetcher.dispatch'

toctree references unknown document 'ipfx.bin.mcc_get_settings.SettingsFetcher.on_any_event'

toctree references unknown document 'ipfx.bin.mcc_get_settings.SettingsFetcher.on_closed'

toctree references unknown document 'ipfx.bin.mcc_get_settings.SettingsFetcher.on_created'

toctree references unknown document 'ipfx.bin.mcc_get_settings.SettingsFetcher.on_deleted'

toctree references unknown document 'ipfx.bin.mcc_get_settings.SettingsFetcher.on_modified'

toctree references unknown document 'ipfx.bin.mcc_get_settings.SettingsFetcher.on_moved'

toctree references unknown document 'ipfx.bin.mcc_get_settings.SettingsFetcher.on_opened'

dispatch(self, event)	Dispatches events to the appropriate methods.
on_any_event(self, event)	Catch-all event handler.
on_closed(self, event)	Called when a file opened for writing is closed.
on_created(self, event)	Called when a file or directory is created.
on_deleted(self, event)	Called when a file or directory is deleted.
on_modified(self, event)	Called when a file or directory is modified.
on_moved(self, event)	Called when a file or a directory is moved or renamed.
on_opened(self, event)	Called when a file is opened.

on_created(*self*, *event*)
Called when a file or directory is created.

Parameters **event** (DirCreatedEvent or FileCreatedEvent) – Event representing file/directory creation.

ipfx.bin.mcc_get_settings.**c_bool_p**
alias of ipfx.bin.mcc_get_settings.LP_c_bool

ipfx.bin.mcc_get_settings.**c_double_p**
alias of ipfx.bin.mcc_get_settings.LP_c_double

ipfx.bin.mcc_get_settings.**c_uint_p**
alias of ipfx.bin.mcc_get_settings.LP_c_uint

ipfx.bin.mcc_get_settings.**main**()

ipfx.bin.mcc_get_settings.**parseSettingsFromFile**(*settingsFile*)

ipfx.bin.mcc_get_settings.**val**(*ptr*, *ptype*)

ipfx.bin.mcc_get_settings.**writeSettingsToFile**(*settingsFile*, *filename*)

ipfx.bin.nwb_to_pdf module

class ipfx.bin.nwb_to_pdf.**PatchClampSeriesPlotData**(*pcs*)
Bases: object

Data class for storing plotting information for PatchClampSeries pcs: neurodata patchClampSeries or derived object

Methods

add_annotation	
check_type	
get_annotation	

add_annotation(*self*, *name*, *data*, *unit*)

check_type(*self*, *type_*)

get_annotation(*self*)

class ipfx.bin.nwb_to_pdf.**SingleSweep**(*id*)
Bases: object

Generic Class for storing sweep specific PatchClampSeries Data

Methods

add_acquisition	
add_stimulus	
get_acquisition	
get_stimulus	
num_acquisition	
num_stimulus	

add_acquisition(*self*, *key*, *pcs*)

add_stimulus(*self*, *key*, *pcs*)

get_acquisition(*self*)

get_stimulus(*self*)

num_acquisition(*self*)

num_stimulus(*self*)

class ipfx.bin.nwb_to_pdf.**SweepCollection**
Bases: object

Class for grouping sweep related PatchClampSeries

Methods

get	

get (*self*, *id*)

ipfx.bin.nwb_to_pdf.**check_stimset_reconstruction** (*nwbfile*, *outfile*)
From a specially crafted NWB file this routine creates a PDF for checking the stimset reconstruction.

The NWB file must have data acquired on two headstages/electrodes where the second just reads back the stimulus set.

ipfx.bin.nwb_to_pdf.**create_regular_pdf** (*nwbfile*, *outfile*)
convert a NeurodataWithoutBorders file to a PortableDocumentFile

ipfx.bin.nwb_to_pdf.**gather_sweeps** (*nwbfile*)
sort PatchClampSeries according to cycle_id

ipfx.bin.nwb_to_pdf.**main** ()

ipfx.bin.nwb_to_pdf.**physical** (*number*, *unit*)

ipfx.bin.nwb_to_pdf.**plot_patchClampSeries** (*axis*, *pcs_data_plot*, *length*)

plot a PatchClampSeries against the axis pcs_data_plot: class PatchClampSeriesPlotData axis: plt.axis length: number of points to plot

ipfx.bin.nwb_to_pdf.**plot_sweepdata** (*sweepdata*, *axes*, *length*, *addXTicks=False*)

plot the given sweep data (stimulus or acquisition) on the given axes

sweepdata: dict(class PatchClampSeriesPlotData) (either acquisition or stimulus)

Definition list ends without a blank line; unexpected unindent.

axes: np.ndarray(plt.axis) length: number of points to plot addXTicks: Add ticks and a label to the X axis at the bottom

ipfx.bin.nwb_to_pdf.**to_si** (*d*, *sep=' '*)
taken from https://stackoverflow.com/a/15734251/7809404

Convert number to string with SI prefix

Example

```
>>> to_si(2500.0)
'2.5 k'
```

```
>>> to_si(2.3E6)
'2.3 M'
```

```
>>> to_si(2.3E-6)
'2.3 µ'
```

```
>>> to_si(-2500.0)
'-2.5 k'
```

```
>>> to_si(0)
'0'
```

ipfx.bin.pipeline_from_specimen_id module

ipfx.bin.pipeline_from_specimen_id.**main**()
Runs pipeline from the specimen_id Usage: python pipeline_from_nwb_file.py SPECIMEN_ID

User must specify the OUTPUT_DIR

ipfx.bin.plot_ephys_nwb module

ipfx.bin.plot_ephys_nwb.**axes_ratios**(*sweep_table*)

ipfx.bin.plot_ephys_nwb.**customize_axis**(*ax*)

ipfx.bin.plot_ephys_nwb.**get_vertical_offset**(*data*)

ipfx.bin.plot_ephys_nwb.**main**()
Plot sweeps of a given ephys nwb file # Usage: $ python plot_ephys_nwb_file.py NWB_FILE_NAME

ipfx.bin.plot_ephys_nwb.**plot_data_set**(*data_set*, *clamp_mode:Union[str, NoneType]=None*, *stimuli:Union[Collection[str], NoneType]=None*, *stimuli_exclude:Union[Collection[str], NoneType]=None*, *show_amps:Union[bool, NoneType]=True*, *qc_sweeps:Union[bool, NoneType]=True*, *figsize=(15, 7)*)

ipfx.bin.plot_ephys_nwb.**plot_waveforms**(*ax*, *ys*, *rs*, *annotations=None*)

ipfx.bin.run_chirp_fv_extraction module

class ipfx.bin.run_chirp_fv_extraction.**CollectChirpFeatureVectorParameters**(*only=None*, *exclude=()*, *many=False*, *context=None*, *load_only=()*, *dump_only=()*, *partial=False*, *unknown=None*)

Bases: argschema.schemas.ArgSchema

Attributes

dict_class

set_class

Methods

toctree references unknown document 'ipfx.bin.run_chirp_fv_extraction.CollectChirpFeatureVectorParameters.Meta'

toctree references unknown document 'ipfx.bin.run_chirp_fv_extraction.CollectChirpFeatureVectorParameters.OPTIONS_CLAS

toctree references unknown document 'ipfx.bin.run_chirp_fv_extraction.CollectChirpFeatureVectorParameters.dump'

toctree references unknown document 'ipfx.bin.run_chirp_fv_extraction.CollectChirpFeatureVectorParameters.dumps'

toctree references unknown document 'ipfx.bin.run_chirp_fv_extraction.CollectChirpFeatureVectorParameters.get_attribute'

toctree references unknown document 'ipfx.bin.run_chirp_fv_extraction.CollectChirpFeatureVectorParameters.handle_error'

toctree references unknown document 'ipfx.bin.run_chirp_fv_extraction.CollectChirpFeatureVectorParameters.load'

toctree references unknown document 'ipfx.bin.run_chirp_fv_extraction.CollectChirpFeatureVectorParameters.loads'

toctree references unknown document 'ipfx.bin.run_chirp_fv_extraction.CollectChirpFeatureVectorParameters.make_object'

toctree references unknown document 'ipfx.bin.run_chirp_fv_extraction.CollectChirpFeatureVectorParameters.on_bind_field'

toctree references unknown document 'ipfx.bin.run_chirp_fv_extraction.CollectChirpFeatureVectorParameters.validate'

`Meta`	Options object for a Schema.
`OPTIONS_CLASS`	alias of `marshmallow.schema.SchemaOpts`
`dump`(self, obj[, many])	Serialize an object to native Python data types according to this Schema's fields.
`dumps`(self, obj[, many])	Same as `dump()`, except return a JSON-encoded string.
`get_attribute`(self, obj, attr, default)	Defines how to pull values from an object to serialize.
`handle_error`(self, error, data)	Custom error handler function for the schema.
`load`(self, data[, many, partial, unknown])	Deserialize a data structure to an object defined by this Schema's fields.
`loads`(self, json_data[, many, partial, unknown])	Same as `load()`, except it takes a JSON string as input.
`make_object`(self, in_data)	marshmallow.pre_load decorated function for applying defaults on deserialation
`on_bind_field`(self, field_name, field_obj)	Hook to modify a field when it is bound to the *Schema*.
`validate`(self, data[, many, partial])	Validate *data* against the schema, returning a dictionary of validation errors.

opts = <marshmallow.schema.SchemaOpts object>

ipfx.bin.run_chirp_fv_extraction.**data_for_specimen_id**(*specimen_id*, *data_source*, *ontology*)

ipfx.bin.run_chirp_fv_extraction.**edit_ontology_data**(*original_ontology_data*, *codes_to_rename*, *new_name_tag*, *new_core_tag*)

ipfx.bin.run_chirp_fv_extraction.**main**(*output_dir*, *output_code*, *input_file*, *include_failed_cells*, *run_parallel*, *data_source*, *chirp_stimulus_codes*, ***kwargs*)

ipfx.bin.run_chirp_fv_extraction.**run_chirp_feature_vector_extraction**(*output_dir*, *output_code*, *include_failed_cells*, *specimen_ids*, *chirp_stimulus_codes*, *data_source='lims'*, *run_parallel=True*)

ipfx.bin.run_feature_collection module

class ipfx.bin.run_feature_collection.**CollectFeatureParameters**(*only=None*, *exclude=()*, *many=False*, *context=None*, *load_only=()*, *dump_only=()*, *partial=False*, *unknown=None*)

Bases: argschema.schemas.ArgSchema

Attributes

dict_class

set_class

Methods

toctree references unknown document 'ipfx.bin.run_feature_collection.CollectFeatureParameters.Meta'

toctree references unknown document 'ipfx.bin.run_feature_collection.CollectFeatureParameters.OPTIONS_CLASS'

toctree references unknown document 'ipfx.bin.run_feature_collection.CollectFeatureParameters.dump'

toctree references unknown document 'ipfx.bin.run_feature_collection.CollectFeatureParameters.dumps'

toctree references unknown document 'ipfx.bin.run_feature_collection.CollectFeatureParameters.get_attribute'

toctree references unknown document 'ipfx.bin.run_feature_collection.CollectFeatureParameters.handle_error'

toctree references unknown document 'ipfx.bin.run_feature_collection.CollectFeatureParameters.load'

toctree references unknown document 'ipfx.bin.run_feature_collection.CollectFeatureParameters.loads'

toctree references unknown document 'ipfx.bin.run_feature_collection.CollectFeatureParameters.make_object'

toctree references unknown document 'ipfx.bin.run_feature_collection.CollectFeatureParameters.on_bind_field'

toctree references unknown document 'ipfx.bin.run_feature_collection.CollectFeatureParameters.validate'

Meta	Options object for a Schema.
OPTIONS_CLASS	alias of marshmallow.schema.SchemaOpts
dump(self, obj[, many])	Serialize an object to native Python data types according to this Schema's fields.
dumps(self, obj[, many])	Same as dump(), except return a JSON-encoded string.
get_attribute(self, obj, attr, default)	Defines how to pull values from an object to serialize.
handle_error(self, error, data)	Custom error handler function for the schema.
load(self, data[, many, partial, unknown])	Deserialize a data structure to an object defined by this Schema's fields.
loads(self, json_data[, many, partial, unknown])	Same as load(), except it takes a JSON string as input.
make_object(self, in_data)	marshmallow.pre_load decorated function for applying defaults on deserialation

Continued on next page

Table 9 – continued from previous page

on_bind_field(self, field_name, field_obj)	Hook to modify a field when it is bound to the *Schema*.
validate(self, data[, many, partial])	Validate *data* against the schema, returning a dictionary of validation errors.

opts = <marshmallow.schema.SchemaOpts object>

ipfx.bin.run_feature_collection.**data_for_specimen_id**(*specimen_id*, *passed_only*, *data_source*, *ontology*, *file_list=None*)

ipfx.bin.run_feature_collection.**extract_features**(*data_set*, *ramp_sweep_numbers*, *ssq_sweep_numbers*, *lsq_sweep_numbers*, *amp_interval=20*, *max_above_rheo=100*)

ipfx.bin.run_feature_collection.**fi_curve_fit**(*amps*, *rates*)

ipfx.bin.run_feature_collection.**first_spike_lsq**(*spike_data, feature_list=['threshold_v', 'peak_v', 'upstroke', 'downstroke', 'upstroke_downstroke_ratio', 'width', 'fast_trough_v']*)

ipfx.bin.run_feature_collection.**first_spike_ramp**(*ramp_analyzer, feature_list=['threshold_v', 'peak_v', 'upstroke', 'downstroke', 'upstroke_downstroke_ratio', 'width', 'fast_trough_v']*)

ipfx.bin.run_feature_collection.**first_spike_ssq**(*ssq_analyzer, feature_list=['threshold_v', 'peak_v', 'upstroke', 'downstroke', 'upstroke_downstroke_ratio', 'width', 'fast_trough_v']*)

ipfx.bin.run_feature_collection.**lin_sqrt_fit**(*x*, *y*)

ipfx.bin.run_feature_collection.**main**()

ipfx.bin.run_feature_collection.**mean_spike_lsq**(*spike_data, feature_list=['threshold_v', 'peak_v', 'upstroke', 'downstroke', 'upstroke_downstroke_ratio', 'width', 'fast_trough_v']*)

ipfx.bin.run_feature_collection.**run_feature_collection**(*ids=None*, *project='T301'*, *include_failed_sweeps=True*, *include_failed_cells=False*, *output_file=''*, *run_parallel=True*, *data_source='lims'*, *file_list=None*, ***kwargs*)

ipfx.bin.run_feature_collection.**sqrt_curve**(*x*, *A*, *Ic*)

ipfx.bin.run_feature_extraction module

ipfx.bin.run_feature_extraction.**collect_spike_times**(*sweep_features*)

ipfx.bin.run_feature_extraction.**main**()
Usage: python run_feature_extraction.py –input_json INPUT_JSON –output_json OUTPUT_JSON

ipfx.bin.run_feature_extraction.**run_feature_extraction**(*input_nwb_file*, *stimulus_ontology_file*, *output_nwb_file*, *qc_fig_dir*, *sweep_info*, *cell_info*, *write_spikes=True*)

ipfx.bin.run_feature_vector_extraction module

class ipfx.bin.run_feature_vector_extraction.**CollectFeatureVectorParameters**(*only=None*, *exclude=()*, *many=False*, *context=None*, *load_only=()*, *dump_only=()*, *partial=False*, *unknown=None*)

Bases: argschema.schemas.ArgSchema

Attributes

dict_class

set_class

Methods

toctree references unknown document 'ipfx.bin.run_feature_vector_extraction.CollectFeatureVectorParameters.Meta'

toctree references unknown document 'ipfx.bin.run_feature_vector_extraction.CollectFeatureVectorParameters.OPTIONS_CLAS

toctree references unknown document 'ipfx.bin.run_feature_vector_extraction.CollectFeatureVectorParameters.dump'

toctree references unknown document 'ipfx.bin.run_feature_vector_extraction.CollectFeatureVectorParameters.dumps'

toctree references unknown document 'ipfx.bin.run_feature_vector_extraction.CollectFeatureVectorParameters.get_attribute'

toctree references unknown document 'ipfx.bin.run_feature_vector_extraction.CollectFeatureVectorParameters.handle_error'

toctree references unknown document 'ipfx.bin.run_feature_vector_extraction.CollectFeatureVectorParameters.load'

toctree references unknown document 'ipfx.bin.run_feature_vector_extraction.CollectFeatureVectorParameters.loads'

toctree references unknown document 'ipfx.bin.run_feature_vector_extraction.CollectFeatureVectorParameters.make_object'

toctree references unknown document 'ipfx.bin.run_feature_vector_extraction.CollectFeatureVectorParameters.on_bind_field'

toctree references unknown document 'ipfx.bin.run_feature_vector_extraction.CollectFeatureVectorParameters.validate'

Meta	Options object for a Schema.
OPTIONS_CLASS	alias of marshmallow.schema.SchemaOpts
dump(self, obj[, many])	Serialize an object to native Python data types according to this Schema's fields.
dumps(self, obj[, many])	Same as dump(), except return a JSON-encoded string.
get_attribute(self, obj, attr, default)	Defines how to pull values from an object to serialize.
handle_error(self, error, data)	Custom error handler function for the schema.
load(self, data[, many, partial, unknown])	Deserialize a data structure to an object defined by this Schema's fields.
loads(self, json_data[, many, partial, unknown])	Same as load(), except it takes a JSON string as input.
make_object(self, in_data)	marshmallow.pre_load decorated function for applying defaults on deserialation
on_bind_field(self, field_name, field_obj)	Hook to modify a field when it is bound to the *Schema*.
validate(self, data[, many, partial])	Validate *data* against the schema, returning a dictionary of validation errors.

opts = <marshmallow.schema.SchemaOpts object>

ipfx.bin.run_feature_vector_extraction.**data_for_specimen_id**(*specimen_id*, *sweep_qc_option*, *data_source*, *ontology*, *ap_window_length=0.005*, *target_sampling_rate=50000*, *file_list=None*)

Extract feature vector from given cell identified by the specimen_id Parameters ———- specimen_id : int

Unexpected indentation.

cell identified

Block quote ends without a blank line; unexpected unindent.

sweep_qc_option [str] see CollectFeatureVectorParameters input schema for details

data_source: str see CollectFeatureVectorParameters input schema for details

ontology [stimulus.StimulusOntology] mapping of stimuli names to stimulus codes

ap_window_length [float] see CollectFeatureVectorParameters input schema for details

target_sampling_rate [float] sampling rate

file_list [list of str] nwbfile names

Definition list ends without a blank line; unexpected unindent.

Unexpected section title.

```
Returns
-------
```

dict : features for a given cell specimen_id

ipfx.bin.run_feature_vector_extraction.**main**()

ipfx.bin.run_feature_vector_extraction.**run_feature_vector_extraction**(*output_dir*, *data_source*, *output_code*, *project*, *output_file_type*, *sweep_qc_option*, *include_failed_cells*, *run_parallel*, *ap_window_length*, *ids=None*, *file_list=None*, ***kwargs*)

Extract feature vector from a list of cells and save result to the output file(s)

Parameters

output_dir [str] see CollectFeatureVectorParameters input schema for details

data_source [str] see CollectFeatureVectorParameters input schema for details

output_code: str see CollectFeatureVectorParameters input schema for details

project [str] see CollectFeatureVectorParameters input schema for details

output_file_type [str] see CollectFeatureVectorParameters input schema for details

sweep_qc_option: str see CollectFeatureVectorParameters input schema for details

include_failed_cells: bool see CollectFeatureVectorParameters input schema for details

run_parallel: bool see CollectFeatureVectorParameters input schema for details

ap_window_length: float see CollectFeatureVectorParameters input schema for details

ids: int ids associated to each cell.

file_list: list of str nwbfile names

kwargs

ipfx.bin.run_pipeline module

ipfx.bin.run_pipeline.**main**()

Usage: python run_pipeline.py –input_json INPUT_JSON –output_json OUTPUT_JSON

ipfx.bin.run_pipeline.**run_pipeline**(*input_nwb_file*, *output_nwb_file*, *stimulus_ontology_file*, *qc_fig_dir*, *qc_criteria*, *manual_sweep_states*, *write_spikes=True*)

ipfx.bin.run_pipeline_from_nwb_file module

ipfx.bin.run_pipeline_from_nwb_file.**main**()

Convenience script for running ephys pipeline from a given nwb file It generates the pipeline input and then

calls the run_pipeline executable

Usage: python run_pipeline_from_nwb_file.py <input_nwb_file> <output_dir>

ipfx.bin.run_qc module

ipfx.bin.run_qc.**main**()
Usage: python run_qc.py –input_json INPUT_JSON –output_json OUTPUT_JSON

ipfx.bin.run_qc.**qc_summary**(*sweep_features*, *sweep_states*, *cell_features*, *cell_state*)
Output QC summary

Parameters

sweep_features: list of dicts

sweep_states: list of dict

cell_features: list of dicts

cell_state: dict

ipfx.bin.run_qc.**run_qc**(*stimulus_ontology_file*, *cell_features*, *sweep_features*, *qc_criteria*)

Parameters

stimulus_ontology_file [str] ontology file name

cell_features: dict cell features

sweep_features [list of dicts] sweep features

qc_criteria: dict qc criteria

Returns

dict containing state of the cell and sweeps

ipfx.bin.run_sweep_extraction module

ipfx.bin.run_sweep_extraction.**main**()
Usage: python run_sweep_extraction.py

Unexpected indentation.

–input_json INPUT_JSON –output_json OUTPUT_JSON

ipfx.bin.run_sweep_extraction.**run_sweep_extraction**(*input_nwb_file*, *stimulus_ontology_file*, *input_manual_values=None*)

Parameters

input_nwb_file

stimulus_ontology_file

input_manual_values

ipfx.bin.run_synphys_feature_vector_extraction module

ipfx.bin.run_x_to_nwb_conversion module

ipfx.bin.run_x_to_nwb_conversion.**convert**(*inFileOrFolder*, *overwrite=False*, *fileType=None*, *outputMetadata=False*, *outputFeedbackChannel=False*, *multipleGroupsPerFile=False*, *compression=True*)

Convert the given file to a NeuroDataWithoutBorders file using pynwb

Supported fileformats:

- ABF v2 files created by Clampex
- DAT files created by Patchmaster v2x90

Parameters

- **inFileOrFolder** – path to a file or folder
- **overwrite** – overwrite output file, defaults to *False*
- **fileType** – file type to be converted, must be passed iff *inFileOrFolder* refers to a folder
- **outputMetadata** – output metadata of the file, helpful for debugging
- **outputFeedbackChannel** – Output ADC data which stems from stimulus feedback channels (ignored for DAT files)
- **multipleGroupsPerFile** – Write all Groups in the DAT file into one NWB file. By default we create one NWB per Group (ignored for ABF files).
- **compression** – Toggle compression for HDF5 datasets

Returns path of the created NWB file

ipfx.bin.run_x_to_nwb_conversion.**main**()

ipfx.bin.validate_experiment module

ipfx.bin.validate_experiment.**get_pipeline_output_json**(*storage_dir*, *err_id*)

Return the name of the output file giving preference to the newer version Parameters ———- storage_dir: str of storage directory err_id: str err_id Returns ——- pipeline_output_json: str filename

ipfx.bin.validate_experiment.**main**()

ipfx.bin.validate_experiment.**nullisclose**(*a*, *b*)

ipfx.bin.validate_experiment.**validate_cell_features**(*err*, *ephys_features*, *cell_record*)

ipfx.bin.validate_experiment.**validate_feature_set**(*features*, *d1*, *d2*)

ipfx.bin.validate_experiment.**validate_fx**(*test_output_json='test/fx_output.json'*)

ipfx.bin.validate_experiment.**validate_pipeline**(*input_json*, *output_json*)

ipfx.bin.validate_experiment.**validate_run_completion**(*pipeline_input_json*, *pipeline_output_json*)

Check if the pipeline output was generated as way of confirming that the run completed Parameters ———- pipeline_input_json pipeline_output_json Returns ——-

ipfx.bin.validate_experiment.**validate_se**(*test_output_json='test/sweep_extraction_output.json'*)

Module contents

8.1.3 ipfx.dataset package

Submodules

ipfx.dataset.create module

ipfx.dataset.create.**create_ephys_data_set**(*nwb_file:str, sweep_info:Union[Dict[str, Any], NoneType]=None, ontology:Union[str, NoneType]=None*) → ipfx.dataset.ephys_data_set.EphysDataSet

Create an ephys data set with the appropriate nwbdata reader class

Parameters

nwb_file

sweep_info

ontology

Returns

EphysDataSet

ipfx.dataset.create.**get_nwb_version**(*nwb_file:str*) → Dict[str, Any]

Find version of the nwb file

Parameters

nwb_file

Returns

dict in the format:

{ *major*: str *full* str.

}

ipfx.dataset.create.**get_scalar_value**(*dataset_from_nwb*)

Some values in NWB are stored as scalar whereas others as np.ndarrays with dimension 1. Use this function to retrieve the scalar value itself.

ipfx.dataset.create.**is_file_mies**(*path:str*) → bool

ipfx.dataset.ephys_data_interface module

class ipfx.dataset.ephys_data_interface.**EphysDataInterface**(*ontology: ipfx.stimulus.StimulusOntology, validate_stim: bool = True*)

Bases: abc.ABC

The interface that any child class providing data to the EphysDataSet must implement

Attributes

sweep_numbers A time-ordered sequence of each sweep's integer identifier

Methods

toctree references unknown document 'ipfx.dataset.ephys_data_interface.EphysDataInterface.get_clamp_mode'

toctree references unknown document 'ipfx.dataset.ephys_data_interface.EphysDataInterface.get_full_recording_date'

toctree references unknown document 'ipfx.dataset.ephys_data_interface.EphysDataInterface.get_stimulus_code'

toctree references unknown document 'ipfx.dataset.ephys_data_interface.EphysDataInterface.get_stimulus_unit'

toctree references unknown document 'ipfx.dataset.ephys_data_interface.EphysDataInterface.get_sweep_attrs'

toctree references unknown document 'ipfx.dataset.ephys_data_interface.EphysDataInterface.get_sweep_data'

toctree references unknown document 'ipfx.dataset.ephys_data_interface.EphysDataInterface.get_sweep_metadata'

get_clamp_mode(self, sweep_number)	Extract clamp mode from the class of Time Series Parameters ————- sweep_number
get_full_recording_date(self)	Obtain the full date and time at which recording began.
get_stimulus_code(self, sweep_number)	Obtain the code of the stimulus presented on a particular sweep.
get_stimulus_unit(self, sweep_number)	Extract unit of a stimulus
get_sweep_attrs(self, sweep_number)	Extract sweep attributes
get_sweep_data(self, sweep_number)	Extract sweep data
get_sweep_metadata(self, sweep_number)	Returns metadata about a sweep

get_stimulus_name	

get_clamp_mode (*self*, *sweep_number*) → str
Extract clamp mode from the class of Time Series Parameters ————- sweep_number

get_full_recording_date (*self*) → datetime.datetime
Obtain the full date and time at which recording began.

Returns

A datetime object, with timezone, reporting the start of recording

get_stimulus_code (*self*, *sweep_number:int*) → str
Obtain the code of the stimulus presented on a particular sweep.

Parameters

sweep_number [unique identifier for the sweep]

Returns

The codified name of the stimulus presented on the

identified sweep

get_stimulus_name (*self*, *stim_code*)

get_stimulus_unit (*self*, *sweep_number:int*) → str
Extract unit of a stimulus

Parameters

sweep_number

Returns

stimulus unit

get_sweep_attrs (*self*, *sweep_number*) → Dict[str, Any]
Extract sweep attributes

Parameters

sweep_number

Returns

sweep attributes

get_sweep_data (*self*, *sweep_number:int*) → Dict[str, Any]
Extract sweep data

Parameters

sweep_number

Returns

dict in the format:

{ 'stimulus': np.ndarray, 'response': np.ndarray, 'stimulus_unit': string, 'sampling_rate': float

}

get_sweep_metadata (*self*, *sweep_number:int*) → Dict[str, Any]
Returns metadata about a sweep

Parameters

sweep_number [identifier of the sweep whose metadata will be returned]

Returns

dict in the format:

{ "sweep_number": int, "stimulus_units": str, "bridge_balance_mohm": float, "leak_pa": float, "stimulus_scale_factor": float, "stimulus_code": str, "stimulus_code_ext": str, "stimulus_name": str, "clamp_mode": str

}

sweep_numbers
A time-ordered sequence of each sweep's integer identifier

ipfx.dataset.ephys_data_set module

class ipfx.dataset.ephys_data_set.**EphysDataSet** (*data: ipfx.dataset.ephys_data_interface.EphysDataInterface*, *sweep_info: Optional[List[Dict]] = None*)

Bases: object

Attributes

ontology The stimulus ontology maps codified description of the stimulus type to the human-readable descriptions.

sweep_info

sweep_table Each row of the sweep table contains the metadata for a single sweep.

Methods

toctree references unknown document 'ipfx.dataset.ephys_data_set.EphysDataSet.filtered_sweep_table'

toctree references unknown document 'ipfx.dataset.ephys_data_set.EphysDataSet.get_clamp_mode'

toctree references unknown document 'ipfx.dataset.ephys_data_set.EphysDataSet.get_recording_date'

toctree references unknown document 'ipfx.dataset.ephys_data_set.EphysDataSet.get_stimulus_code'

toctree references unknown document 'ipfx.dataset.ephys_data_set.EphysDataSet.get_stimulus_code_ext'

toctree references unknown document 'ipfx.dataset.ephys_data_set.EphysDataSet.get_stimulus_units'

toctree references unknown document 'ipfx.dataset.ephys_data_set.EphysDataSet.get_sweep_data'

toctree references unknown document 'ipfx.dataset.ephys_data_set.EphysDataSet.get_sweep_number'

toctree references unknown document 'ipfx.dataset.ephys_data_set.EphysDataSet.get_sweep_numbers'

toctree references unknown document 'ipfx.dataset.ephys_data_set.EphysDataSet.sweep'

toctree references unknown document 'ipfx.dataset.ephys_data_set.EphysDataSet.sweep_set'

filtered_sweep_table(self, clamp_mode, ...)	Utility for filtering the sweep table
get_clamp_mode(self, sweep_number)	Obtain the clamp mode of a given sweep.
get_recording_date(self)	Return the date and time at which recording began.
get_stimulus_code(self, sweep_number)	Return the (short form) stimulus code for a particular sweep.
get_stimulus_code_ext(self, sweep_number)	Obtain the extended stimulus code for a sweep.
get_stimulus_units(self, sweep_number)	Report the SI unit of measurement for a sweep's stimulus data
get_sweep_data(self, sweep_number)	Obtain the recorded data for a given sweep.
get_sweep_number(self, stimuli, clamp_mode, ...)	Convenience for getting the integer identifier of the temporally latest sweep matching argued criteria.
get_sweep_numbers(self, stimuli, clamp_mode, ...)	Return the integer identifier of all sweeps matching argued criteria
sweep(self, sweep_number)	Create an instance of the Sweep class with the data loaded from the from a file
sweep_set(self, sweep_numbers, int, ...)	Construct a SweepSet object, which offers convenient access to an ordered collection of sweeps.

```
CLAMP_MODE = 'clamp_mode'
COLUMN_NAMES = ['stimulus_units', 'stimulus_code', 'stimulus_amplitude', 'stimulus_nam
CURRENT_CLAMP = 'CurrentClamp'
STIMULUS_AMPLITUDE = 'stimulus_amplitude'
STIMULUS_CODE = 'stimulus_code'
STIMULUS_NAME = 'stimulus_name'
STIMULUS_UNITS = 'stimulus_units'
SWEEP_NUMBER = 'sweep_number'
```

VOLTAGE_CLAMP = 'VoltageClamp'

filtered_sweep_table(*self, clamp_mode:Union[str, NoneType]=None, stimuli:Union[Collection[str], NoneType]=None, stimuli_exclude:Union[Collection[str], NoneType]=None*) → pandas.core.frame.DataFrame

Utility for filtering the sweep table

Parameters

clamp_mode: filter to one of self.VOLTAGE_CLAMP or self.CURRENT_CLAMP

stimuli: filter to sweeps presenting these stimuli (codes)

stimuli_exclude: filter to sweeps not presenting these stimuli

Returns

filtered sweep table

get_clamp_mode(*self*, *sweep_number:int*) → str

Obtain the clamp mode of a given sweep. Should be one of EphysDataSet.VOLTAGE_CLAMP or EphysDataSet.CURRENT_CLAMP

Parameters

sweep_number [identifier for the sweep whose clamp mode will be] returned

Returns

The clamp mode of the identified sweep

get_recording_date(*self*) → str

Return the date and time at which recording began.

Returns

a string, formatted like: "%Y-%m-%d %H:%M:%S" in local time

get_stimulus_code(*self*, *sweep_number:int*) → str

Return the (short form) stimulus code for a particular sweep.

Parameters

sweep_number [identifier for the sweep whose stimulus code will be] returned

Returns

code defining the stimulus presented on the identified sweep

get_stimulus_code_ext(*self*, *sweep_number:int*) → str

Obtain the extended stimulus code for a sweep. This is the stimulus code for that sweep augmented with an integer counter describing the number of presentations of that stimulus up to and including the requested sweep.

Parameters

sweep_number [identifies the sweep whose extended stimulus code will]

be returned

Returns

A string of the form "{stimulus_code}[{counter}]"

get_stimulus_units(*self*, *sweep_number:int*) → str

Report the SI unit of measurement for a sweep's stimulus data

Parameters

sweep_number [identifies the sweep whose stimulus unit will be] returned

Returns

An SI (or derived) unit's name

get_sweep_data(*self*, *sweep_number:int*) → Dict
Obtain the recorded data for a given sweep.

Parameters

sweep_number [identifier for the sweep whose data will be returned]

Returns

A dictionary containing at least:

{ 'stimulus': np.ndarray, 'response': np.ndarray, 'stimulus_unit': string, 'sampling_rate': float

Definition list ends without a blank line; unexpected unindent.

}

get_sweep_number(*self, stimuli:Collection[str], clamp_mode:Union[str, NoneType]=None*) → int
Convenience for getting the integer identifier of the temporally latest sweep matching argued criteria.

Parameters

stimuli [filter to sweeps presentingthese stimuli]

clamp_mode [filter to sweeps of this clamp mode]

Returns

The identifier of the last sweep matching argued criteria

get_sweep_numbers(*self*, *stimuli:Collection[str]=None*, *clamp_mode:Union[str*, *None-Type]=None*) → List[int]
Return the integer identifier of all sweeps matching argued criteria

Parameters

stimuli [filter to sweeps presenting these stimuli (codes)]

clamp_mode [filter to sweeps of this clamp mode]

Returns

A list of sweep numbers matching these criteria

ontology
The stimulus ontology maps codified description of the stimulus type to the human-readable descriptions.

sweep(*self*, *sweep_number:int*) → ipfx.sweep.Sweep
Create an instance of the Sweep class with the data loaded from the from a file

Parameters

sweep_number: int

Returns

sweep: Sweep object

sweep_info

sweep_set(*self, sweep_numbers:Union[Sequence[int], int, NoneType]=None*) → ipfx.sweep.SweepSet

Construct a SweepSet object, which offers convenient access to an ordered collection of sweeps.

Parameters

sweep_numbers [Identifiers for the sweeps which will make up this set.] If None, use all available sweeps.

Returns

A SweepSet constructed from the requested sweeps

sweep_table

Each row of the sweep table contains the metadata for a single sweep. In particular details of the stimulus presented and the clamp mode. See EphysDataInterface.get_sweep_metadata for more information.

ipfx.dataset.ephys_nwb_data module

class ipfx.dataset.ephys_nwb_data.**EphysNWBData**(*nwb_file: None, ontology: ipfx.stimulus.StimulusOntology, load_into_memory: bool = True, validate_stim: bool = True*)

Bases: *ipfx.dataset.ephys_data_interface.EphysDataInterface*

Abstract base class for implementing an EphysDataInterface with an NWB file

Provides common NWB2 reading and writing functionality

Attributes

sweep_numbers A time-ordered sequence of each sweep's integer identifier

Methods

toctree references unknown document 'ipfx.dataset.ephys_nwb_data.EphysNWBData.get_clamp_mode'

toctree references unknown document 'ipfx.dataset.ephys_nwb_data.EphysNWBData.get_full_recording_date'

toctree references unknown document 'ipfx.dataset.ephys_nwb_data.EphysNWBData.get_stimulus_code'

toctree references unknown document 'ipfx.dataset.ephys_nwb_data.EphysNWBData.get_stimulus_unit'

toctree references unknown document 'ipfx.dataset.ephys_nwb_data.EphysNWBData.get_sweep_attrs'

toctree references unknown document 'ipfx.dataset.ephys_nwb_data.EphysNWBData.get_sweep_data'

toctree references unknown document 'ipfx.dataset.ephys_nwb_data.EphysNWBData.get_sweep_metadata'

toctree references unknown document 'ipfx.dataset.ephys_nwb_data.EphysNWBData.load_nwb'

get_clamp_mode(self, sweep_number)	Extract clamp mode from the class of Time Series Parameters ———- sweep_number
get_full_recording_date(self)	Extract session_start_time in nwb Use last value if more than one is present
get_stimulus_code(self, sweep_number)	Obtain the code of the stimulus presented on a particular sweep.
get_stimulus_unit(self, sweep_number)	Extract unit of a stimulus
get_sweep_attrs(self, sweep_number)	Extract sweep attributes

Continued on next page

Table 13 – continued from previous page

get_sweep_data(self, sweep_number)	Extracts numpy arrays, stimulus unit, and sampling rate from sweep.
get_sweep_metadata(self, sweep_number)	Returns metadata about a sweep
load_nwb(self, nwb_file, load_into_memory)	Load NWB to self.nwb

get_long_unit_name	
get_spike_times	
get_stimulus_name	
validate_SI_unit	

RESPONSE = (<class 'pynwb.icephys.VoltageClampSeries'>, <class 'pynwb.icephys.CurrentC

STIMULUS = (<class 'pynwb.icephys.VoltageClampStimulusSeries'>, <class 'pynwb.icephys.

get_clamp_mode(*self*, *sweep_number*)
Extract clamp mode from the class of Time Series Parameters ————- sweep_number

get_full_recording_date(*self*)
Extract session_start_time in nwb Use last value if more than one is present

Returns

recording_date: str use date format "%Y-%m-%d %H:%M:%S", drop timezone info

static get_long_unit_name(*unit*)

get_spike_times(*self*, *sweep_number*)

get_stimulus_code(*self*, *sweep_number*)
Obtain the code of the stimulus presented on a particular sweep.

Parameters

sweep_number [unique identifier for the sweep]

Returns

The codified name of the stimulus presented on the

identified sweep

get_stimulus_unit(*self*, *sweep_number*)
Extract unit of a stimulus

Parameters

sweep_number

Returns

stimulus unit

get_sweep_attrs(*self*, *sweep_number*)
Extract sweep attributes

Parameters

sweep_number

Returns

sweep attributes

get_sweep_data(*self*, *sweep_number:int*) → Dict[str, Union[numpy.ndarray, str, float]]
Extracts numpy arrays, stimulus unit, and sampling rate from sweep.

Grabs stimulus and response PatchClampSeries and extracts numpy arrays for the stimulus and response time series data as well as the stimulus unit and sampling rate

Parameters

sweep_number: int Integer specifying the sweep to extract data from

Returns

sweep_data [Dict[str, Union[np.ndarray, str, float]]] Dictionary of sweep data, which includes stimulus and response numpy arrays as well as stimulus units and sampling rate.

get_sweep_metadata(*self*, *sweep_number:int*)
Returns metadata about a sweep

Parameters

sweep_number [identifier of the sweep whose metadata will be returned]

Returns

dict in the format:

{ "sweep_number": int, "stimulus_units": str, "bridge_balance_mohm": float, "leak_pa": float, "stimulus_scale_factor": float, "stimulus_code": str, "stimulus_code_ext": str, "stimulus_name": str, "clamp_mode": str

}

load_nwb(*self*, *nwb_file:None*, *load_into_memory:bool=True*)
Load NWB to self.nwb

Parameters

nwb_file: NWB file path or hdf5 obj

load_into_memory: whether using load_into_memory approach to load NWB

sweep_numbers
A time-ordered sequence of each sweep's integer identifier

static validate_SI_unit(*unit*)

class ipfx.dataset.ephys_nwb_data.**NWBHDF5IO**(**args*, ***kwargs*)
Bases: pynwb.NWBHDF5IO

Simple alias to suppress 'ignoring namespace' errors in NWBHDF5IO

Attributes

comm The MPI communicator to use for parallel I/O.

driver

manager The BuildManager this instance is using

mode Return the HDF5 file mode.

source The source of the container being read/written i.e.

Methods

toctree references unknown document 'ipfx.dataset.ephys_nwb_data.NWBHDF5IO.close'

toctree references unknown document 'ipfx.dataset.ephys_nwb_data.NWBHDF5IO.close_linked_files'

toctree references unknown document 'ipfx.dataset.ephys_nwb_data.NWBHDF5IO.copy_file'

toctree references unknown document 'ipfx.dataset.ephys_nwb_data.NWBHDF5IO.export'

toctree references unknown document 'ipfx.dataset.ephys_nwb_data.NWBHDF5IO.export_io'

toctree references unknown document 'ipfx.dataset.ephys_nwb_data.NWBHDF5IO.get_builder'

toctree references unknown document 'ipfx.dataset.ephys_nwb_data.NWBHDF5IO.get_container'

toctree references unknown document 'ipfx.dataset.ephys_nwb_data.NWBHDF5IO.get_written'

toctree references unknown document 'ipfx.dataset.ephys_nwb_data.NWBHDF5IO.load_namespaces'

toctree references unknown document 'ipfx.dataset.ephys_nwb_data.NWBHDF5IO.open'

toctree references unknown document 'ipfx.dataset.ephys_nwb_data.NWBHDF5IO.read'

toctree references unknown document 'ipfx.dataset.ephys_nwb_data.NWBHDF5IO.read_builder'

toctree references unknown document 'ipfx.dataset.ephys_nwb_data.NWBHDF5IO.set_attributes'

toctree references unknown document 'ipfx.dataset.ephys_nwb_data.NWBHDF5IO.set_dataio'

toctree references unknown document 'ipfx.dataset.ephys_nwb_data.NWBHDF5IO.write'

toctree references unknown document 'ipfx.dataset.ephys_nwb_data.NWBHDF5IO.write_builder'

toctree references unknown document 'ipfx.dataset.ephys_nwb_data.NWBHDF5IO.write_dataset'

toctree references unknown document 'ipfx.dataset.ephys_nwb_data.NWBHDF5IO.write_group'

toctree references unknown document 'ipfx.dataset.ephys_nwb_data.NWBHDF5IO.write_link'

`close`(self)	Close this HDMFIO object to further reading/writing
`close_linked_files`(self)	Close all opened, linked-to files.
`copy_file`(source_filename, dest_filename[, ...])	Convenience function to copy an HDF5 file while allowing external links to be resolved.
`export`(src_io[, nwbfile, write_args])	Args:
`export_io`(path, src_io[, comm, container, ...])	Export from one backend to HDF5 (class method).
`get_builder`(h5obj)	Get the builder for the corresponding h5py Group or Dataset
`get_container`(h5obj)	Get the container for the corresponding h5py Group or Dataset
`get_written`(self, builder)	Return True if this builder has been written to (or read from) disk by this IO object, False otherwise.
`load_namespaces`(namespace_catalog[, path, ...])	Load cached namespaces from a file.
`open`(self)	Open this HDMFIO object for writing of the builder
`read`()	Read a container from the IO source.
`read_builder`()	Returns:
`set_attributes`(obj, attributes)	Args:
`set_dataio`([data, maxshape, chunks, ...])	Wrap the given Data object with an H5DataIO.
`write`(container[, cache_spec, link_data, ...])	Write the container to an HDF5 file.
`write_builder`(builder[, link_data, ...])	Args:

Continued on next page

Table 14 – continued from previous page

write_dataset(parent, builder[, link_data, ...])	Write a dataset to HDF5
write_group(parent, builder[, link_data, ...])	Args:
write_link(parent, builder)	Args:

get_type

ipfx.dataset.ephys_nwb_data.**get_finite_or_none**(*d*, *key*)

ipfx.dataset.ephys_nwb_data.**get_scalar_value**(*dataset_from_nwb*)

Some values in NWB are stored as scalar whereas others as np.ndarrays with dimension 1. Use this function to retrieve the scalar value itself.

ipfx.dataset.hbg_nwb_data module

class ipfx.dataset.hbg_nwb_data.**HBGNWBData**(*nwb_file: str*, *ontology: ipfx.stimulus.StimulusOntology*, *load_into_memory: bool = True*, *validate_stim: bool = True*)

Bases: *ipfx.dataset.ephys_nwb_data.EphysNWBData*

Provides an Ephys Data Interface to an HBG generated NWB file

Attributes

sweep_numbers A time-ordered sequence of each sweep's integer identifier

Methods

toctree references unknown document 'ipfx.dataset.hbg_nwb_data.HBGNWBData.get_clamp_mode'

toctree references unknown document 'ipfx.dataset.hbg_nwb_data.HBGNWBData.get_full_recording_date'

toctree references unknown document 'ipfx.dataset.hbg_nwb_data.HBGNWBData.get_stimulus_code'

toctree references unknown document 'ipfx.dataset.hbg_nwb_data.HBGNWBData.get_stimulus_unit'

toctree references unknown document 'ipfx.dataset.hbg_nwb_data.HBGNWBData.get_sweep_attrs'

toctree references unknown document 'ipfx.dataset.hbg_nwb_data.HBGNWBData.get_sweep_data'

toctree references unknown document 'ipfx.dataset.hbg_nwb_data.HBGNWBData.get_sweep_metadata'

toctree references unknown document 'ipfx.dataset.hbg_nwb_data.HBGNWBData.load_nwb'

get_clamp_mode(self, sweep_number)	Extract clamp mode from the class of Time Series Parameters ———- sweep_number
get_full_recording_date(self)	Extract session_start_time in nwb Use last value if more than one is present
get_stimulus_code(self, sweep_number)	Obtain the code of the stimulus presented on a particular sweep.
get_stimulus_unit(self, sweep_number)	Extract unit of a stimulus
get_sweep_attrs(self, sweep_number)	Extract sweep attributes
get_sweep_data(self, sweep_number)	Extracts numpy arrays, stimulus unit, and sampling rate from sweep.
get_sweep_metadata(self, sweep_number)	Returns metadata about a sweep

Continued on next page

Table 15 – continued from previous page

load_nwb(self, nwb_file, load_into_memory)	Load NWB to self.nwb

get_long_unit_name	
get_spike_times	
get_stimulus_code_ext	
get_stimulus_name	
validate_SI_unit	

get_stimulus_code_ext (*self*, *sweep_number*)

get_sweep_metadata (*self*, *sweep_number:int*) → Dict[str, Any]

Returns metadata about a sweep

Parameters

sweep_number [identifier of the sweep whose metadata will be returned]

Returns

dict in the format:

{ "sweep_number": int, "stimulus_units": str, "bridge_balance_mohm": float, "leak_pa": float, "stimulus_scale_factor": float, "stimulus_code": str, "stimulus_code_ext": str, "stimulus_name": str, "clamp_mode": str

}

ipfx.dataset.labnotebook module

class ipfx.dataset.labnotebook.**LabNotebookReader**

Bases: object

The Lab Notebook Reader class

This class reads two sections: numeric data and text data

Methods

toctree references unknown document 'ipfx.dataset.labnotebook.LabNotebookReader.get_numeric_value'

toctree references unknown document 'ipfx.dataset.labnotebook.LabNotebookReader.get_text_value'

toctree references unknown document 'ipfx.dataset.labnotebook.LabNotebookReader.get_value'

toctree references unknown document 'ipfx.dataset.labnotebook.LabNotebookReader.register_enabled_names'

get_numeric_value(self, name, data_col, ...)	fetch numeric data
get_text_value(self, name, data_col, ...)	fetch text data
get_value(self, name, sweep_num, default_val)	looks for key in lab notebook and returns the value associated with the specified sweep, or the default value if no value is found (NaN and empty strings are considered to be non-values)

Continued on next page

Table 16 – continued from previous page

register_enabled_names(self)	register_enabled_names: mapping of notebook keys to keys representing if that value is enabled move this to subclasses if/when key names diverge

get_numeric_value(*self*, *name*, *data_col*, *sweep_col*, *enable_col*, *sweep_num*, *default_val*)
fetch numeric data

val_number has 3 dimensions – the first has a shape of (#fields * 9). there are many hundreds of elements in this dimension. they look to represent the full array of values (for each field for each multipatch) for a given point in time, and thus given sweep according to Thomas Braun (igor nwb dev), the first 8 pages are for headstage data, and the 9th is for headstage-independent data

Parameters

name: str

data_col: int

sweep_col: int

enable_col: int

sweep_num: int

default_val: dict

Returns

last non-empty entry in specified column

for specified sweep number

get_text_value(*self*, *name*, *data_col*, *sweep_col*, *enable_col*, *sweep_num*, *default_val*)
fetch text data

Parameters

name: str

data_col: int

sweep_col: int

enable_col: int

sweep_num: int

default_val: dict

Returns

last non-empty entry in specified column

for specified sweep number

get_value(*self*, *name*, *sweep_num*, *default_val*)
looks for key in lab notebook and returns the value associated with the specified sweep, or the default value if no value is found (NaN and empty strings are considered to be non-values)

name_number has 3 dimensions – the first has shape (#fields * 9) and stores the key names. the second looks to store units for those keys. The third is numeric text but it's role isn't clear

val_number has 3 dimensions – the first has a shape of (#fields * 9). there are many hundreds of elements in this dimension. they look to represent the full array of values (for each field for each multipatch) for a given point in time, and thus given sweep

Parameters

name: str

sweep_num: int

default_val: dict

Returns

values obtained from lab notebook

register_enabled_names (*self*)
register_enabled_names: mapping of notebook keys to keys representing if that value is enabled move this to subclasses if/when key names diverge

class ipfx.dataset.labnotebook.**LabNotebookReaderIgorNwb** (*nwb_file*)
Bases: *ipfx.dataset.labnotebook.LabNotebookReader*

LabNotebookReaderIgorNwb: Loads lab notebook data out of an Igor-generated NWB file. Module input is the name of the nwb file. Notebook data can be read through get_value() function

Methods

toctree references unknown document 'ipfx.dataset.labnotebook.LabNotebookReaderIgorNwb.get_numeric_value'

toctree references unknown document 'ipfx.dataset.labnotebook.LabNotebookReaderIgorNwb.get_text_value'

toctree references unknown document 'ipfx.dataset.labnotebook.LabNotebookReaderIgorNwb.get_value'

toctree references unknown document 'ipfx.dataset.labnotebook.LabNotebookReaderIgorNwb.register_enabled_names'

get_numeric_value(self, name, data_col, ...)	fetch numeric data
get_text_value(self, name, data_col, ...)	fetch text data
get_value(self, name, sweep_num, default_val)	looks for key in lab notebook and returns the value associated with the specified sweep, or the default value if no value is found (NaN and empty strings are considered to be non-values)
register_enabled_names(self)	register_enabled_names: mapping of notebook keys to keys representing if that value is enabled move this to subclasses if/when key names diverge

get_numerical_keys	
get_numerical_values	
get_textual_keys	
get_textual_values	

get_numerical_keys (*self*)

get_numerical_values (*self*)

get_textual_keys (*self*)

get_textual_values (*self*)

ipfx.dataset.mies_nwb_data module

class ipfx.dataset.mies_nwb_data.**MIESNWBData**(*nwb_file: str*, *notebook: ipfx.dataset.labnotebook.LabNotebookReader*, *ontology: ipfx.stimulus.StimulusOntology*, *load_into_memory: bool = True*, *validate_stim: bool = True*)

Bases: *ipfx.dataset.ephys_nwb_data.EphysNWBData*

Provides an Ephys Data Interface to a MIES generated NWB file

Attributes

sweep_numbers A time-ordered sequence of each sweep's integer identifier

Methods

toctree references unknown document 'ipfx.dataset.mies_nwb_data.MIESNWBData.get_clamp_mode'

toctree references unknown document 'ipfx.dataset.mies_nwb_data.MIESNWBData.get_full_recording_date'

toctree references unknown document 'ipfx.dataset.mies_nwb_data.MIESNWBData.get_stimulus_code'

toctree references unknown document 'ipfx.dataset.mies_nwb_data.MIESNWBData.get_stimulus_unit'

toctree references unknown document 'ipfx.dataset.mies_nwb_data.MIESNWBData.get_sweep_attrs'

toctree references unknown document 'ipfx.dataset.mies_nwb_data.MIESNWBData.get_sweep_data'

toctree references unknown document 'ipfx.dataset.mies_nwb_data.MIESNWBData.get_sweep_metadata'

toctree references unknown document 'ipfx.dataset.mies_nwb_data.MIESNWBData.load_nwb'

get_clamp_mode(self, sweep_number)	Extract clamp mode from the class of Time Series Parameters ———- sweep_number
get_full_recording_date(self)	Extract session_start_time in nwb Use last value if more than one is present
get_stimulus_code(self, sweep_number)	Obtain the code of the stimulus presented on a particular sweep.
get_stimulus_unit(self, sweep_number)	Extract unit of a stimulus
get_sweep_attrs(self, sweep_number)	Extract sweep attributes
get_sweep_data(self, sweep_number)	Extracts numpy arrays, stimulus unit, and sampling rate from sweep.
get_sweep_metadata(self, sweep_number)	Returns metadata about a sweep
load_nwb(self, nwb_file, load_into_memory)	Load NWB to self.nwb

get_long_unit_name	
get_spike_times	
get_stim_code_ext	
get_stimulus_name	
validate_SI_unit	

get_stim_code_ext(*self*, *sweep_number*)

get_sweep_metadata(*self*, *sweep_number:int*) → Dict[str, Any]

Returns metadata about a sweep

Parameters

sweep_number [identifier of the sweep whose metadata will be returned]

Returns

dict in the format:

{ "sweep_number": int, "stimulus_units": str, "bridge_balance_mohm": float, "leak_pa": float, "stimulus_scale_factor": float, "stimulus_code": str, "stimulus_code_ext": str, "stimulus_name": str, "clamp_mode": str

}

Module contents

8.1.4 ipfx.x_to_nwb package

Submodules

ipfx.x_to_nwb.ABFConverter module

Convert ABF files, created by PClamp/Clampex, to NWB v2 files.

class ipfx.x_to_nwb.ABFConverter.**ABFConverter**(*inFileOrFolder*, *outFile*, *outputFeedbackChannel*, *compression=True*)

Bases: object

Attributes

protocolStorageDir

Methods

outputMetadata	

adcNamesWithRealData = ['IN 0', 'IN 1', 'IN 2', 'IN 3']

static outputMetadata(*inFile*)

protocolStorageDir = None

ipfx.x_to_nwb.DatConverter module

class ipfx.x_to_nwb.DatConverter.**DatConverter**(*inFile*, *outFile*, *multipleGroupsPerFile=False*, *compression=True*)

Bases: object

Methods

toctree references unknown document 'ipfx.x_to_nwb.DatConverter.DatConverter.outputMetadata'

outputMetadata(inFile)	Create a file with metadata of the given DAT file.

static outputMetadata(*inFile*)

Create a file with metadata of the given DAT file.

Parameters

inFile: DAT file

Returns

None

ipfx.x_to_nwb.conversion_utils module

Miscellaneous helper routines for the ABF/DAT to NWB v2 (aka X to NWB) conversion functionality.

ipfx.x_to_nwb.conversion_utils.**clampModeToString**(*clampMode*)
Return the given clamp mode as human readable string. Useful for error messages.

ipfx.x_to_nwb.conversion_utils.**convertDataset**(*array*, *compression*)
Convert to FP32 and optionally request compression for the given array and return it wrapped.

ipfx.x_to_nwb.conversion_utils.**createCycleID**(*numbers*, *total*)
Create an integer from all numbers which is unique for that combination.

Param numbers: Iterable holding non-negative integer numbers

Param total: Total number of TimeSeries written to the NWB file

ipfx.x_to_nwb.conversion_utils.**createSeriesName**(*prefix*, *number*, *total*)
Format a unique series group name of the form *prefix_XXX* where *XXX* is the formatted *number* long enough for *total* number of groups.

ipfx.x_to_nwb.conversion_utils.**getAcquiredSeriesClass**(*clampMode*)
Return the appropriate pynwb acquisition class for the given clamp mode.

ipfx.x_to_nwb.conversion_utils.**getChannelRecordIndex**(*pgf*, *sweep*, *trace*)
Given a pgf node, a SweepRecord and TraceRecord this returns the corresponding *ChannelRecordStimulus* node as index.

ipfx.x_to_nwb.conversion_utils.**getPackageInfo**()
Return a dictionary with version information for the allensdk package

ipfx.x_to_nwb.conversion_utils.**getStimulusRecordIndex**(*sweep*)

ipfx.x_to_nwb.conversion_utils.**getStimulusSeriesClass**(*clampMode*)
Return the appropriate pynwb stimulus class for the given clamp mode.

ipfx.x_to_nwb.conversion_utils.**parseUnit**(*unitString*)
Split a SI unit string with prefix into the base unit and the prefix (as number).

ipfx.x_to_nwb.hr_bundle module

class ipfx.x_to_nwb.hr_bundle.**Bundle**(*file_name*)
Bases: object

Represent a PATCHMASTER tree file in memory

Attributes

amp The Amplifier object from this bundle.

data The Data object from this bundle.

mrk The Markers object from this bundle.

mth The ProtocolMethod object from this bundle.

onl The Online Analysis object from this bundle.

pgf The Stimulus Template object from this bundle.

pul The Pulsed object from this bundle.

sol The Solutions object from this bundle.

amp
The Amplifier object from this bundle.

data
The Data object from this bundle.

item_classes = {'.amp': <class 'ipfx.x_to_nwb.hr_nodes.AmplifierFile'>, '.dat': <cla

mrk
The Markers object from this bundle.

mth
The ProtocolMethod object from this bundle.

onl
The Online Analysis object from this bundle.

pgf
The Stimulus Template object from this bundle.

pul
The Pulsed object from this bundle.

sol
The Solutions object from this bundle.

ipfx.x_to_nwb.hr_nodes module

All supported nodes are listed here. The root node of each bundle calls the TreeNode constructor explicitly the other are plain children of the root nodes.

Documentation:

- https://github.com/neurodroid/stimfit/blob/master/src/libstfio/heka/hekalib.cpp
- ftp://server.hekahome.de/pub/FileFormat/Patchmasterv9/
- For the field_info type parameters see https://docs.python.org/3/library/struct.html#format-characters
- The CARD types are unsigned, see e.g. https://www.common-lisp.net/project/cmucl/doc/clx/1_6_Data_Types.html

The nodes are tailored for patchmaster version 2x90.x.

class ipfx.x_to_nwb.hr_nodes.**AmplifierFile**(*fh*, *endianess*)
Bases: *ipfx.x_to_nwb.hr_treenode.TreeNode*

Methods

toctree references unknown document 'ipfx.x_to_nwb.hr_nodes.AmplifierFile.array'

toctree references unknown document 'ipfx.x_to_nwb.hr_nodes.AmplifierFile.get_fields'

array(x)	Return a new StructArray class of length *x* and using this struct as the array item type.
get_fields(self)	Recursively convert struct fields+values to nested dictionaries.

size

field_info = [('Version', 'i'), ('Mark', 'i'), ('VersionName', '32s', <function cstr>)

rectypes = [None, <class 'ipfx.x_to_nwb.hr_nodes.AmplifierSeriesRecord'>, <class 'ipfx

required_size = 80

class ipfx.x_to_nwb.hr_nodes.**AmplifierSeriesRecord**(*fh*, *endianess*, *record_types*, *level_sizes*, *level=0*)

Bases: *ipfx.x_to_nwb.hr_treenode.TreeNode*

Methods

toctree references unknown document 'ipfx.x_to_nwb.hr_nodes.AmplifierSeriesRecord.array'

toctree references unknown document 'ipfx.x_to_nwb.hr_nodes.AmplifierSeriesRecord.get_fields'

array(x)	Return a new StructArray class of length *x* and using this struct as the array item type.
get_fields(self)	Recursively convert struct fields+values to nested dictionaries.

size

field_info = [('Mark', 'i'), ('SeriesCount', 'i'), ('Filler1', 'i', None), ('CRC', 'I'

required_size = 16

class ipfx.x_to_nwb.hr_nodes.**AmplifierState**(*data*, *endian='<'*)

Bases: *ipfx.x_to_nwb.hr_struct.Struct*

Methods

toctree references unknown document 'ipfx.x_to_nwb.hr_nodes.AmplifierState.array'

toctree references unknown document 'ipfx.x_to_nwb.hr_nodes.AmplifierState.get_fields'

array(x)	Return a new StructArray class of length *x* and using this struct as the array item type.
get_fields(self)	Recursively convert struct fields+values to nested dictionaries.

size

field_info = [('StateVersion', '8s', <function cstr>), ('RealCurrentGain', 'd'), ('Rea

required_size = 400

class ipfx.x_to_nwb.hr_nodes.**AmplifierStateRecord**(*fh*, *endianess*, *record_types*, *level_sizes*, *level=0*)

Bases: *ipfx.x_to_nwb.hr_treenode.TreeNode*

Methods

toctree references unknown document 'ipfx.x_to_nwb.hr_nodes.AmplifierStateRecord.array'

toctree references unknown document 'ipfx.x_to_nwb.hr_nodes.AmplifierStateRecord.get_fields'

array(x)	Return a new StructArray class of length *x* and using this struct as the array item type.
get_fields(self)	Recursively convert struct fields+values to nested dictionaries.

size	

field_info = [('Mark', 'i'), ('StateCount', 'i'), ('StateVersion', 'b'), ('Filler1', '

required_size = 560

class ipfx.x_to_nwb.hr_nodes.**Analysis**(*fh*, *endianess*)

Bases: *ipfx.x_to_nwb.hr_treenode.TreeNode*

Methods

toctree references unknown document 'ipfx.x_to_nwb.hr_nodes.Analysis.array'

toctree references unknown document 'ipfx.x_to_nwb.hr_nodes.Analysis.get_fields'

array(x)	Return a new StructArray class of length *x* and using this struct as the array item type.
get_fields(self)	Recursively convert struct fields+values to nested dictionaries.

size	

field_info = [('Version', 'i'), ('Mark', 'i'), ('VersionName', '32s', <function cstr>)

rectypes = [None, <class 'ipfx.x_to_nwb.hr_nodes.AnalysisMethodRecord'>, <class 'ipfx.

required_size = 64

class ipfx.x_to_nwb.hr_nodes.**AnalysisEntryRecord**(*data*, *endian='<'*)

Bases: *ipfx.x_to_nwb.hr_struct.Struct*

Methods

toctree references unknown document 'ipfx.x_to_nwb.hr_nodes.AnalysisEntryRecord.array'

toctree references unknown document 'ipfx.x_to_nwb.hr_nodes.AnalysisEntryRecord.get_fields'

array(x)	Return a new StructArray class of length *x* and using this struct as the array item type.
get_fields(self)	Recursively convert struct fields+values to nested dictionaries.

size	

field_info = [('XWave', 'h'), ('YWave', 'h'), ('MarkerSize', 'h'), ('MarkerColorRed',

required_size = 16

class ipfx.x_to_nwb.hr_nodes.**AnalysisFunctionRecord**(*fh*, *endianess*, *record_types*, *level_sizes*, *level=0*)

Bases: *ipfx.x_to_nwb.hr_treenode.TreeNode*

Methods

toctree references unknown document 'ipfx.x_to_nwb.hr_nodes.AnalysisFunctionRecord.array'

toctree references unknown document 'ipfx.x_to_nwb.hr_nodes.AnalysisFunctionRecord.get_fields'

array(x)	Return a new StructArray class of length *x* and using this struct as the array item type.
get_fields(self)	Recursively convert struct fields+values to nested dictionaries.

size	

field_info = [('Mark', 'i'), ('Name', '32s', <function cstr>), ('Unit', '8s', <functio

required_size = 168

class ipfx.x_to_nwb.hr_nodes.**AnalysisGraphRecord**(*data*, *endian='<'*)

Bases: *ipfx.x_to_nwb.hr_struct.Struct*

Methods

toctree references unknown document 'ipfx.x_to_nwb.hr_nodes.AnalysisGraphRecord.array'

toctree references unknown document 'ipfx.x_to_nwb.hr_nodes.AnalysisGraphRecord.get_fields'

array(x)	Return a new StructArray class of length *x* and using this struct as the array item type.
get_fields(self)	Recursively convert struct fields+values to nested dictionaries.

size	

field_info = [('GActive', '?'), ('Overlay', '?'), ('Wrap', 'b'), ('OvrlSwp', '?'), ('N

required_size = 152

class ipfx.x_to_nwb.hr_nodes.**AnalysisMethodRecord**(*fh*, *endianess*, *record_types*, *level_sizes*, *level=0*)

Bases: *ipfx.x_to_nwb.hr_treenode.TreeNode*

Methods

toctree references unknown document 'ipfx.x_to_nwb.hr_nodes.AnalysisMethodRecord.array'

toctree references unknown document 'ipfx.x_to_nwb.hr_nodes.AnalysisMethodRecord.get_fields'

array(x)	Return a new StructArray class of length *x* and using this struct as the array item type.
get_fields(self)	Recursively convert struct fields+values to nested dictionaries.

size	

field_info = [('Mark', 'i'), ('EntryName', '32s', <function cstr>), ('SharedXWin1', '?

required_size = 2640

class ipfx.x_to_nwb.hr_nodes.**AnalysisScalingRecord**(*data*, *endian='<'*)

Bases: *ipfx.x_to_nwb.hr_struct.Struct*

Methods

toctree references unknown document 'ipfx.x_to_nwb.hr_nodes.AnalysisScalingRecord.array'

toctree references unknown document 'ipfx.x_to_nwb.hr_nodes.AnalysisScalingRecord.get_fields'

array(x)	Return a new StructArray class of length *x* and using this struct as the array item type.
get_fields(self)	Recursively convert struct fields+values to nested dictionaries.

size	

field_info = [('MinValue', 'd'), ('MaxValue', 'd'), ('GridFactor', 'd'), ('TicLength',

required_size = 40

class ipfx.x_to_nwb.hr_nodes.**BundleHeader**(*data*, *endian='<'*)

Bases: *ipfx.x_to_nwb.hr_struct.Struct*

Methods

toctree references unknown document 'ipfx.x_to_nwb.hr_nodes.BundleHeader.array'

toctree references unknown document 'ipfx.x_to_nwb.hr_nodes.BundleHeader.get_fields'

array(x)	Return a new StructArray class of length *x* and using this struct as the array item type.
get_fields(self)	Recursively convert struct fields+values to nested dictionaries.

size

field_info = [('Signature', '8s', <function cstr>), ('Version', '32s', <function cstr>

required_size = 256

class ipfx.x_to_nwb.hr_nodes.**BundleItem**(*data*, *endian='<'*)

Bases: *ipfx.x_to_nwb.hr_struct.Struct*

Methods

toctree references unknown document 'ipfx.x_to_nwb.hr_nodes.BundleItem.array'

toctree references unknown document 'ipfx.x_to_nwb.hr_nodes.BundleItem.get_fields'

array(x)	Return a new StructArray class of length *x* and using this struct as the array item type.
get_fields(self)	Recursively convert struct fields+values to nested dictionaries.

size

field_info = [('Start', 'i'), ('Length', 'i'), ('Extension', '8s', <function cstr>)]

required_size = 16

class ipfx.x_to_nwb.hr_nodes.**ChannelRecordStimulus**(*fh*, *endianess*, *record_types*, *level_sizes*, *level=0*)

Bases: *ipfx.x_to_nwb.hr_treenode.TreeNode*

Methods

toctree references unknown document 'ipfx.x_to_nwb.hr_nodes.ChannelRecordStimulus.array'

toctree references unknown document 'ipfx.x_to_nwb.hr_nodes.ChannelRecordStimulus.get_fields'

array(x)	Return a new StructArray class of length *x* and using this struct as the array item type.
get_fields(self)	Recursively convert struct fields+values to nested dictionaries.

size

field_info = [('Mark', 'i'), ('LinkedChannel', 'i'), ('CompressionFactor', 'i'), ('YUn

required_size = 401

class ipfx.x_to_nwb.hr_nodes.**GroupRecord**(*fh*, *endianess*, *record_types*, *level_sizes*, *level=0*)
Bases: *ipfx.x_to_nwb.hr_treenode.TreeNode*

Methods

toctree references unknown document 'ipfx.x_to_nwb.hr_nodes.GroupRecord.array'

toctree references unknown document 'ipfx.x_to_nwb.hr_nodes.GroupRecord.get_fields'

array(x)	Return a new StructArray class of length *x* and using this struct as the array item type.
get_fields(self)	Recursively convert struct fields+values to nested dictionaries.

size	

field_info = [('Mark', 'i'), ('Label', '32s', <function cstr>), ('Text', '80s', <funct

required_size = 144

class ipfx.x_to_nwb.hr_nodes.**LockInParams**(*data*, *endian='<'*)
Bases: *ipfx.x_to_nwb.hr_struct.Struct*

Methods

toctree references unknown document 'ipfx.x_to_nwb.hr_nodes.LockInParams.array'

toctree references unknown document 'ipfx.x_to_nwb.hr_nodes.LockInParams.get_fields'

array(x)	Return a new StructArray class of length *x* and using this struct as the array item type.
get_fields(self)	Recursively convert struct fields+values to nested dictionaries.

size	

field_info = [('ExtCalPhase', 'd'), ('ExtCalAtten', 'd'), ('PLPhase', 'd'), ('PLPhaseY

required_size = 96

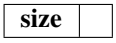
class ipfx.x_to_nwb.hr_nodes.**Marker**(*fh*, *endianess*)
Bases: *ipfx.x_to_nwb.hr_treenode.TreeNode*

Methods

toctree references unknown document 'ipfx.x_to_nwb.hr_nodes.Marker.array'

toctree references unknown document 'ipfx.x_to_nwb.hr_nodes.Marker.get_fields'

array(x)	Return a new StructArray class of length *x* and using this struct as the array item type.
get_fields(self)	Recursively convert struct fields+values to nested dictionaries.



field_info = [('Version', 'i'), ('CRC', 'I')]

rectypes = [None]

required_size = 8

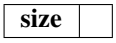
class ipfx.x_to_nwb.hr_nodes.**ProtocolMethod**(*fh*, *endianess*)
Bases: *ipfx.x_to_nwb.hr_treenode.TreeNode*

Methods

toctree references unknown document 'ipfx.x_to_nwb.hr_nodes.ProtocolMethod.array'

toctree references unknown document 'ipfx.x_to_nwb.hr_nodes.ProtocolMethod.get_fields'

array(x)	Return a new StructArray class of length *x* and using this struct as the array item type.
get_fields(self)	Recursively convert struct fields+values to nested dictionaries.



field_info = [('Version', 'i'), ('Mark', 'i'), ('VersionName', '32s', <function cstr>)

rectypes = [None]

required_size = 514

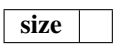
class ipfx.x_to_nwb.hr_nodes.**Pulsed**(*fh*, *endianess*)
Bases: *ipfx.x_to_nwb.hr_treenode.TreeNode*

Methods

toctree references unknown document 'ipfx.x_to_nwb.hr_nodes.Pulsed.array'

toctree references unknown document 'ipfx.x_to_nwb.hr_nodes.Pulsed.get_fields'

array(x)	Return a new StructArray class of length *x* and using this struct as the array item type.
get_fields(self)	Recursively convert struct fields+values to nested dictionaries.



field_info = [('Version', 'i'), ('Mark', 'i'), ('VersionName', '32s', <function cstr>)

rectypes = [None, <class 'ipfx.x_to_nwb.hr_nodes.GroupRecord'>, <class 'ipfx.x_to_nwb.

required_size = 640

class ipfx.x_to_nwb.hr_nodes.**RawData**(*bundle*)
Bases: object

class ipfx.x_to_nwb.hr_nodes.**SeriesRecord**(*fh*, *endianess*, *record_types*, *level_sizes*, *level=0*)
Bases: *ipfx.x_to_nwb.hr_treenode.TreeNode*

Methods

toctree references unknown document 'ipfx.x_to_nwb.hr_nodes.SeriesRecord.array'

toctree references unknown document 'ipfx.x_to_nwb.hr_nodes.SeriesRecord.get_fields'

array(x)	Return a new StructArray class of length *x* and using this struct as the array item type.
get_fields(self)	Recursively convert struct fields+values to nested dictionaries.

size	

field_info = [('Mark', 'i'), ('Label', '32s', <function cstr>), ('Comment', '80s', <fu

required_size = 1408

class ipfx.x_to_nwb.hr_nodes.**Solutions**(*fh*, *endianess*)
Bases: *ipfx.x_to_nwb.hr_treenode.TreeNode*

Methods

toctree references unknown document 'ipfx.x_to_nwb.hr_nodes.Solutions.array'

toctree references unknown document 'ipfx.x_to_nwb.hr_nodes.Solutions.get_fields'

array(x)	Return a new StructArray class of length *x* and using this struct as the array item type.
get_fields(self)	Recursively convert struct fields+values to nested dictionaries.

size	

field_info = [('RoVersion', 'H'), ('RoDataBaseName', '80s', <function cstr>), ('RoSpar

rectypes = [None]

required_size = 88

class ipfx.x_to_nwb.hr_nodes.**StimSegmentRecord**(*fh*, *endianess*, *record_types*, *level_sizes*, *level=0*)
Bases: *ipfx.x_to_nwb.hr_treenode.TreeNode*

Methods

toctree references unknown document 'ipfx.x_to_nwb.hr_nodes.StimSegmentRecord.array'

toctree references unknown document 'ipfx.x_to_nwb.hr_nodes.StimSegmentRecord.get_fields'

array(x)	Return a new StructArray class of length *x* and using this struct as the array item type.
get_fields(self)	Recursively convert struct fields+values to nested dictionaries.

size	

field_info = [('Mark', 'i'), ('Class', 'b', <function getSegmentClass>), ('StoreKind',

required_size = 80

class ipfx.x_to_nwb.hr_nodes.**StimulationRecord**(*fh*, *endianess*, *record_types*, *level_sizes*, *level=0*)

Bases: *ipfx.x_to_nwb.hr_treenode.TreeNode*

Methods

toctree references unknown document 'ipfx.x_to_nwb.hr_nodes.StimulationRecord.array'

toctree references unknown document 'ipfx.x_to_nwb.hr_nodes.StimulationRecord.get_fields'

array(x)	Return a new StructArray class of length *x* and using this struct as the array item type.
get_fields(self)	Recursively convert struct fields+values to nested dictionaries.

size	

field_info = [('Mark', 'i'), ('EntryName', '32s', <function cstr>), ('FileName', '32s'

required_size = 248

class ipfx.x_to_nwb.hr_nodes.**StimulusTemplate**(*fh*, *endianess*)

Bases: *ipfx.x_to_nwb.hr_treenode.TreeNode*

Methods

toctree references unknown document 'ipfx.x_to_nwb.hr_nodes.StimulusTemplate.array'

toctree references unknown document 'ipfx.x_to_nwb.hr_nodes.StimulusTemplate.get_fields'

array(x)	Return a new StructArray class of length *x* and using this struct as the array item type.
get_fields(self)	Recursively convert struct fields+values to nested dictionaries.

size	

field_info = [('Version', 'i'), ('Mark', 'i'), ('VersionName', '32s', <function cstr>)

rectypes = [None, <class 'ipfx.x_to_nwb.hr_nodes.StimulationRecord'>, <class 'ipfx.x_t

required_size = 584

class ipfx.x_to_nwb.hr_nodes.**SweepRecord**(*fh*, *endianess*, *record_types*, *level_sizes*, *level=0*)
Bases: *ipfx.x_to_nwb.hr_treenode.TreeNode*

Methods

toctree references unknown document 'ipfx.x_to_nwb.hr_nodes.SweepRecord.array'

toctree references unknown document 'ipfx.x_to_nwb.hr_nodes.SweepRecord.get_fields'

array(x)	Return a new StructArray class of length *x* and using this struct as the array item type.
get_fields(self)	Recursively convert struct fields+values to nested dictionaries.

size	

field_info = [('Mark', 'i'), ('Label', '32s', <function cstr>), ('AuxDataFileOffset',

required_size = 288

class ipfx.x_to_nwb.hr_nodes.**TraceRecord**(*fh*, *endianess*, *record_types*, *level_sizes*, *level=0*)
Bases: *ipfx.x_to_nwb.hr_treenode.TreeNode*

Methods

toctree references unknown document 'ipfx.x_to_nwb.hr_nodes.TraceRecord.array'

toctree references unknown document 'ipfx.x_to_nwb.hr_nodes.TraceRecord.get_fields'

array(x)	Return a new StructArray class of length *x* and using this struct as the array item type.
get_fields(self)	Recursively convert struct fields+values to nested dictionaries.

size	

field_info = [('Mark', 'i'), ('Label', '32s', <function cstr>), ('TraceCount', 'i'), (

required_size = 512

class ipfx.x_to_nwb.hr_nodes.**UserParamDescrType**(*data*, *endian='<'*)
Bases: *ipfx.x_to_nwb.hr_struct.Struct*

Methods

toctree references unknown document 'ipfx.x_to_nwb.hr_nodes.UserParamDescrType.array'

toctree references unknown document 'ipfx.x_to_nwb.hr_nodes.UserParamDescrType.get_fields'

`array`(x)	Return a new StructArray class of length *x* and using this struct as the array item type.
`get_fields`(self)	Recursively convert struct fields+values to nested dictionaries.

size	

field_info = [('Name', '32s', <function cstr>), ('Unit', '8s', <function cstr>)]

required_size = 40

ipfx.x_to_nwb.hr_nodes.**convertDataFormatToNP**(*dataFormat*)

ipfx.x_to_nwb.hr_nodes.**convertDataKind**(*byte*)

ipfx.x_to_nwb.hr_nodes.**convertStimToDacID**(*byte*)

ipfx.x_to_nwb.hr_nodes.**cstr**(*byte*)
Convert C string bytes to python string.

ipfx.x_to_nwb.hr_nodes.**getADBoard**(*byte*)

ipfx.x_to_nwb.hr_nodes.**getADCMode**(*byte*)

ipfx.x_to_nwb.hr_nodes.**getAmplMode**(*byte*)

ipfx.x_to_nwb.hr_nodes.**getAmplifierGain**(*byte*)
Units: V/A

ipfx.x_to_nwb.hr_nodes.**getAmplifierType**(*byte*)

ipfx.x_to_nwb.hr_nodes.**getChirpKind**(*byte*)

ipfx.x_to_nwb.hr_nodes.**getClampMode**(*byte*)

ipfx.x_to_nwb.hr_nodes.**getDataFormat**(*byte*)

ipfx.x_to_nwb.hr_nodes.**getFromList**(*lst*, *index*)

ipfx.x_to_nwb.hr_nodes.**getIncrementMode**(*byte*)

ipfx.x_to_nwb.hr_nodes.**getRecordingMode**(*byte*)

ipfx.x_to_nwb.hr_nodes.**getSegmentClass**(*byte*)

ipfx.x_to_nwb.hr_nodes.**getSourceType**(*byte*)

ipfx.x_to_nwb.hr_nodes.**getSquareKind**(*byte*)

ipfx.x_to_nwb.hr_nodes.**getStoreType**(*byte*)

ipfx.x_to_nwb.hr_segments module

Create a numpy array with the stimulus set data from the stored parameters.

class ipfx.x_to_nwb.hr_segments.**ChirpSegment**(*stimRec*, *channelRec*, *segmentRec*)
Bases: *ipfx.x_to_nwb.hr_segments.Segment*

Methods

toctree references unknown document 'ipfx.x_to_nwb.hr_segments.ChirpSegment.applyAmplitudeScale'

toctree references unknown document 'ipfx.x_to_nwb.hr_segments.ChirpSegment.calculateNumberOfPoints'

toctree references unknown document 'ipfx.x_to_nwb.hr_segments.ChirpSegment.createArray'

toctree references unknown document 'ipfx.x_to_nwb.hr_segments.ChirpSegment.doStepping'

toctree references unknown document 'ipfx.x_to_nwb.hr_segments.ChirpSegment.getAmplitude'

toctree references unknown document 'ipfx.x_to_nwb.hr_segments.ChirpSegment.hasXDelta'

toctree references unknown document 'ipfx.x_to_nwb.hr_segments.ChirpSegment.hasYDelta'

applyAmplitudeScale(self, amplitude)	Scale the amplitude so that we get the correct units.
calculateNumberOfPoints(self, duration)	Return the number of points of this segment.
createArray(self, sweep)	Return a numpy array with the stimset data.
doStepping(self, sweepNo)	Apply the delta modes the given number of times (once per sweep)
getAmplitude(self, channelRec, segmentRec)	Extract the value of the amplitude from the ChannelRecordStimulus and the StimSegmentRecord.
hasXDelta(self)	Return true if delta mode is active for the x dimension.
hasYDelta(self)	Return true if delta mode is active for the y dimension.

createArray(*self*, *sweep*)
Return a numpy array with the stimset data.

Units are [mV] for voltage clamp and [pA] for current clamp.

getAmplitude(*self*, *channelRec*, *segmentRec*)
Extract the value of the amplitude from the ChannelRecordStimulus and the StimSegmentRecord.

class ipfx.x_to_nwb.hr_segments.**ConstantSegment**(*stimRec*, *channelRec*, *segmentRec*)
Bases: *ipfx.x_to_nwb.hr_segments.Segment*

Methods

toctree references unknown document 'ipfx.x_to_nwb.hr_segments.ConstantSegment.applyAmplitudeScale'

toctree references unknown document 'ipfx.x_to_nwb.hr_segments.ConstantSegment.calculateNumberOfPoints'

toctree references unknown document 'ipfx.x_to_nwb.hr_segments.ConstantSegment.createArray'

toctree references unknown document 'ipfx.x_to_nwb.hr_segments.ConstantSegment.doStepping'

toctree references unknown document 'ipfx.x_to_nwb.hr_segments.ConstantSegment.getAmplitude'

toctree references unknown document 'ipfx.x_to_nwb.hr_segments.ConstantSegment.hasXDelta'

toctree references unknown document 'ipfx.x_to_nwb.hr_segments.ConstantSegment.hasYDelta'

applyAmplitudeScale(self, amplitude)	Scale the amplitude so that we get the correct units.
calculateNumberOfPoints(self, duration)	Return the number of points of this segment.
createArray(self, sweep)	Return a numpy array with the stimset data.

Continued on next page

Table 47 – continued from previous page

doStepping(self, sweepNo)	Apply the delta modes the given number of times (once per sweep)
getAmplitude(self, channelRec, segmentRec)	Extract the value of the amplitude from the ChannelRecordStimulus and the StimSegmentRecord.
hasXDelta(self)	Return true if delta mode is active for the x dimension.
hasYDelta(self)	Return true if delta mode is active for the y dimension.

createArray(*self*, *sweep*)

Return a numpy array with the stimset data.

Units are [mV] for voltage clamp and [pA] for current clamp.

class ipfx.x_to_nwb.hr_segments.**RampSegment**(*stimRec*, *channelRec*, *segmentRec*)

Bases: *ipfx.x_to_nwb.hr_segments.Segment*

Methods

toctree references unknown document 'ipfx.x_to_nwb.hr_segments.RampSegment.applyAmplitudeScale'

toctree references unknown document 'ipfx.x_to_nwb.hr_segments.RampSegment.calculateNumberOfPoints'

toctree references unknown document 'ipfx.x_to_nwb.hr_segments.RampSegment.createArray'

toctree references unknown document 'ipfx.x_to_nwb.hr_segments.RampSegment.doStepping'

toctree references unknown document 'ipfx.x_to_nwb.hr_segments.RampSegment.getAmplitude'

toctree references unknown document 'ipfx.x_to_nwb.hr_segments.RampSegment.hasXDelta'

toctree references unknown document 'ipfx.x_to_nwb.hr_segments.RampSegment.hasYDelta'

applyAmplitudeScale(self, amplitude)	Scale the amplitude so that we get the correct units.
calculateNumberOfPoints(self, duration)	Return the number of points of this segment.
createArray(self, sweep)	Return a numpy array with the stimset data.
doStepping(self, sweepNo)	Apply the delta modes the given number of times (once per sweep)
getAmplitude(self, channelRec, segmentRec)	Extract the value of the amplitude from the ChannelRecordStimulus and the StimSegmentRecord.
hasXDelta(self)	Return true if delta mode is active for the x dimension.
hasYDelta(self)	Return true if delta mode is active for the y dimension.

createArray(*self*, *sweep*)

Return a numpy array with the stimset data.

Units are [mV] for voltage clamp and [pA] for current clamp.

class ipfx.x_to_nwb.hr_segments.**Segment**(*stimRec*, *channelRec*, *segmentRec*)

Bases: abc.ABC

Base class for all segment types. Derived class must implement *createArray* only.

The following segment types are supported: - Constant - Ramp - Square - Chirp

Support for the following types is missing: - Continous - Sine

Note: Only currently used segment options/modes/specialities are implemented.

Methods

toctree references unknown document 'ipfx.x_to_nwb.hr_segments.Segment.applyAmplitudeScale'

toctree references unknown document 'ipfx.x_to_nwb.hr_segments.Segment.calculateNumberOfPoints'

toctree references unknown document 'ipfx.x_to_nwb.hr_segments.Segment.createArray'

toctree references unknown document 'ipfx.x_to_nwb.hr_segments.Segment.doStepping'

toctree references unknown document 'ipfx.x_to_nwb.hr_segments.Segment.getAmplitude'

toctree references unknown document 'ipfx.x_to_nwb.hr_segments.Segment.hasXDelta'

toctree references unknown document 'ipfx.x_to_nwb.hr_segments.Segment.hasYDelta'

applyAmplitudeScale(self, amplitude)	Scale the amplitude so that we get the correct units.
calculateNumberOfPoints(self, duration)	Return the number of points of this segment.
createArray(self, sweep)	Return a numpy array with the stimset data.
doStepping(self, sweepNo)	Apply the delta modes the given number of times (once per sweep)
getAmplitude(self, channelRec, segmentRec)	Extract the value of the amplitude from the ChannelRecordStimulus and the StimSegmentRecord.
hasXDelta(self)	Return true if delta mode is active for the x dimension.
hasYDelta(self)	Return true if delta mode is active for the y dimension.

applyAmplitudeScale(*self*, *amplitude*)
Scale the amplitude so that we get the correct units. See also Segment.createArray.

calculateNumberOfPoints(*self*, *duration*)
Return the number of points of this segment.

createArray(*self*, *sweep*)
Return a numpy array with the stimset data.

Units are [mV] for voltage clamp and [pA] for current clamp.

doStepping(*self*, *sweepNo*)
Apply the delta modes the given number of times (once per sweep)

getAmplitude(*self*, *channelRec*, *segmentRec*)
Extract the value of the amplitude from the ChannelRecordStimulus and the StimSegmentRecord.

hasXDelta(*self*)
Return true if delta mode is active for the x dimension.

hasYDelta(*self*)
Return true if delta mode is active for the y dimension.

class ipfx.x_to_nwb.hr_segments.**SquareSegment**(*stimRec*, *channelRec*, *segmentRec*)
Bases: *ipfx.x_to_nwb.hr_segments.Segment*

Methods

toctree references unknown document 'ipfx.x_to_nwb.hr_segments.SquareSegment.applyAmplitudeScale'

toctree references unknown document 'ipfx.x_to_nwb.hr_segments.SquareSegment.calculateNumberOfPoints'

toctree references unknown document 'ipfx.x_to_nwb.hr_segments.SquareSegment.createArray'

toctree references unknown document 'ipfx.x_to_nwb.hr_segments.SquareSegment.doStepping'

toctree references unknown document 'ipfx.x_to_nwb.hr_segments.SquareSegment.getAmplitude'

toctree references unknown document 'ipfx.x_to_nwb.hr_segments.SquareSegment.hasXDelta'

toctree references unknown document 'ipfx.x_to_nwb.hr_segments.SquareSegment.hasYDelta'

applyAmplitudeScale(self, amplitude)	Scale the amplitude so that we get the correct units.
calculateNumberOfPoints(self, duration)	Return the number of points of this segment.
createArray(self, sweep)	Return a numpy array with the stimset data.
doStepping(self, sweepNo)	Apply the delta modes the given number of times (once per sweep)
getAmplitude(self, channelRec, segmentRec)	Extract the value of the amplitude from the ChannelRecordStimulus and the StimSegmentRecord.
hasXDelta(self)	Return true if delta mode is active for the x dimension.
hasYDelta(self)	Return true if delta mode is active for the y dimension.

createArray (*self*, *sweep*)
Return a numpy array with the stimset data.

Units are [mV] for voltage clamp and [pA] for current clamp.

getAmplitude (*self*, *channelRec*, *segmentRec*)
Extract the value of the amplitude from the ChannelRecordStimulus and the StimSegmentRecord.

ipfx.x_to_nwb.hr_segments.**getSegmentClass** (*stimRec*, *channelRec*, *segmentRec*)
Return the correct derived class instance of Segment for the given records.

ipfx.x_to_nwb.hr_stimsetgenerator module

class ipfx.x_to_nwb.hr_stimsetgenerator.**StimSetGenerator** (*bundle*)
Bases: object

High level class for creating stimsets

Methods

toctree references unknown document 'ipfx.x_to_nwb.hr_stimsetgenerator.StimSetGenerator.fetch'

fetch(self, sweep, trace)	Fetch a stimulus set from the cache, generate it if it is not present.

fetch (*self*, *sweep*, *trace*)
Fetch a stimulus set from the cache, generate it if it is not present.

Parameters

- **sweep** – SweepRecord node
- **trace** – TraceRecord node

Return: python list of numpy arrays, one array per sweep

ipfx.x_to_nwb.hr_struct module

class ipfx.x_to_nwb.hr_struct.**Struct**(*data*, *endian='<'*)

Bases: object

High-level wrapper around struct.Struct that makes it a bit easier to unpack large, nested structures.

- Unpacks to dictionary allowing fields to be retrieved by name
- Optionally massages field data on read
- Handles arrays and nested structures

fields must be a list of tuples like (name, format) or (name, format, function) where *format* must be a simple struct format string like 'i', 'd', '32s', or '4d'; or another Struct instance.

function may be either a function that filters the data for that field or None to exclude the field altogether.

If *size* is given, then an exception will be raised if the final struct size does not match the given size.

Example:

```
class MyStruct(Struct):
    field_info = [
        ('char_field', 'c'),            # single char
        ('char_array', '8c'),           # list of 8 chars
        ('str_field',  '8s', cstr),     # C string of len 8
        ('sub_struct', MyOtherStruct),  # dict generated by s2.unpack
        ('filler', '32s', None),        # ignored field
    ]
    required_size = 300

fh = open(fname, 'rb')
data = MyStruct(fh)
```

Attributes

field_info

required_size

Methods

toctree references unknown document 'ipfx.x_to_nwb.hr_struct.Struct.array'

toctree references unknown document 'ipfx.x_to_nwb.hr_struct.Struct.get_fields'

array(x)	Return a new StructArray class of length *x* and using this struct as the array item type.
get_fields(self)	Recursively convert struct fields+values to nested dictionaries.

size	

classmethod array(*x*)

Return a new StructArray class of length *x* and using this struct as the array item type.

field_info = None

get_fields(*self*)
Recursively convert struct fields+values to nested dictionaries.

required_size = None

classmethod size()

class ipfx.x_to_nwb.hr_struct.**StructArray**(*data*, *endian='<'*)
Bases: *ipfx.x_to_nwb.hr_struct.Struct*

Attributes

array_size

field_info

item_struct

required_size

Methods

toctree references unknown document 'ipfx.x_to_nwb.hr_struct.StructArray.array'

toctree references unknown document 'ipfx.x_to_nwb.hr_struct.StructArray.get_fields'

array(x)	Return a new StructArray class of length *x* and using this struct as the array item type.
get_fields(self)	Recursively convert struct fields+values to nested dictionaries.

size	

array_size = None

item_struct = None

classmethod size()

ipfx.x_to_nwb.hr_treenode module

Wrapper class around the native struct

class ipfx.x_to_nwb.hr_treenode.**TreeNode**(*fh*, *endianess*, *record_types*, *level_sizes*, *level=0*)
Bases: *ipfx.x_to_nwb.hr_struct.Struct*

Struct that also represents a node in the tree.

Attributes

field_info

required_size

Methods

toctree references unknown document 'ipfx.x_to_nwb.hr_treenode.TreeNode.array'

toctree references unknown document 'ipfx.x_to_nwb.hr_treenode.TreeNode.get_fields'

`array`(x)	Return a new StructArray class of length *x* and using this struct as the array item type.
`get_fields`(self)	Recursively convert struct fields+values to nested dictionaries.

size	

Module contents

This package allows to create NeuroDataWithoutBorders v2 files from ABF and DAT files.

See Readme.md for more information.

8.2 Submodules

8.3 ipfx.chirp module

`ipfx.chirp.`**`chirp_amp_phase`**(*sweep_set*, *start=0.6*, *end=20.6*, *down_rate=2000*, *min_freq=0.2*, *max_freq=40.0*)

Calculate amplitude and phase of chirp responses

Parameters

sweep_set: SweepSet Set of chirp sweeps

start: float (optional, default 0.6) Start of chirp stimulus in seconds

end: float (optional, default 20.6) End of chirp stimulus in seconds

down_rate: int (optional, default 2000) Sampling rate for downsampling before FFT

min_freq: float (optional, default 0.2) Minimum frequency for output to contain

max_freq: float (optional, default 40) Maximum frequency for output to contain

Returns

amplitude: array Aka resistance

phase: array Aka reactance

freq: array Frequencies for amplitude and phase results

`ipfx.chirp.`**`divide_chirps_by_stimulus`**(*sweep_set*)

`ipfx.chirp.`**`extract_chirp_feature_vector`**(*data_set*, *chirp_sweep_numbers*)

`ipfx.chirp.`**`feature_vectors_chirp`**(*chirp_sweeps*, *min_freq=0.2*, *max_freq=40.0*)

8.4 ipfx.data_access module

```
get_archive_info(dataset:str, archive_info:pandas.core.frame.DataFrame=         size (GB)
dataset              ...
human            12  ...  /home/docs/checkouts/readthedocs.org/user_buil...
mouse           114  ...  /home/docs/checkouts/readthedocs.org/user_buil...

[2 rows x 4 columns]) -> Tuple[str, pandas.core.frame.DataFrame, pandas.core.frame.DataFra
```

Provide information about released archive

Parameters

dataset [name of the dataset to query. Currently supported options are:]

- human
- mouse

archive_info [dataframe of metadata and manifest files for each supported] dataset. Dataset name is the index.

Returns

Information about the archive

8.5 ipfx.data_set_features module

ipfx.data_set_features.**detection_parameters**(*stimulus_type*)

ipfx.data_set_features.**detection_parameters_from_stimulus_name**(*stimulus_name*)

ipfx.data_set_features.**extract_cell_long_square_features**(*data_set*, *subthresh_min_amp=None*)

ipfx.data_set_features.**extract_cell_ramp_features**(*data_set*)

ipfx.data_set_features.**extract_cell_short_square_features**(*data_set*)

ipfx.data_set_features.**extract_data_set_features**(*data_set*, *subthresh_min_amp=None*)

Parameters

data_set [EphysDataSet] data set

subthresh_min_amp

Returns

cell_features :

sweep_features :

cell_record :

sweep_records :

ipfx.data_set_features.**extract_sweep_features**(*data_set*, *sweep_table*)

ipfx.data_set_features.**extractors_for_sweeps**(*sweep_set*, *dv_cutoff=20.0*, *thresh_frac=0.05*, *reject_at_stim_start_interval=0*, *min_peak=-30*, *filter_frequency=10.0*, *thresh_frac_floor=None*, *est_window=None*, *start=None*, *end=None*)

Extract data from sweeps

Parameters

sweep_set [SweepSet object]

dv_cutoff [float]

thresh_frac :

reject_at_stim_start_interval :

thresh_frac_floor

est_window :

start :

end :

Returns

spx [SpikeExtractor object]

spfx [SpikeTrainFeatureExtractor object]

ipfx.data_set_features.**fallback_on_error**(*fallback_value=None*, *catch_errors=<class 'Exception'>*)

ipfx.data_set_features.**record_errors**(*fn*)

ipfx.data_set_features.**select_subthreshold_min_amplitude**(*stim_amps*, *decimals=0*)

Find the min delta between amplitudes of coarse long square sweeps. Includes failed sweeps.

Parameters

stim_amps: list of stimulus amplitudes

decimals: int of decimals to keep

Returns

subthresh_min_amp: float min amplitude

min_amp_delta: min increment in the stimulus amplitude

8.6 ipfx.data_set_utils module

A shim for backwards compatible imports of create_data_set

8.7 ipfx.ephys_data_set module

A shim for backwards compatible imports of EphysDataSet

8.8 ipfx.epochs module

ipfx.epochs.**get_experiment_epoch**(*i*, *hz*, *test_pulse=True*)
Find index range for the experiment epoch. The start index of the experiment epoch is defined as stim_start_idx - PRESTIM_DURATION*sampling_rate The end index of the experiment epoch is defined as stim_end_idx + POSTSTIM_DURATION*sampling_rate

Parameters

i [float np.array of current]

hz [float sampling rate]

test_pulse: bool True if present, False otherwise

Returns

(expt_start_idx,expt_end_idx): int tuple with start, end indices of the epoch

ipfx.epochs.**get_first_noise_epoch**(*idx*, *hz*)

ipfx.epochs.**get_first_stability_epoch**(*stim_start_idx*, *hz*)

ipfx.epochs.**get_last_noise_epoch**(*idx1*, *hz*)

ipfx.epochs.**get_last_stability_epoch**(*idx1*, *hz*)
Get epoch lasting LAST_STABILITY_EPOCH before idx1

Parameters

idx1 [int last index of the epoch]

hz [float sampling rate]

Returns

(idx0,idx1) [int tuple of epoch indices]

ipfx.epochs.**get_recording_epoch**(*response*)
Detect response epoch defined as interval from start to the last non-nan value of the response

Parameters

response: float np.array

Returns

start,end: int indices of the epoch

ipfx.epochs.**get_stim_epoch**(*i*, *test_pulse=True*)
Determine the start index, and end index of a general stimulus.

Parameters

i [numpy array] current

test_pulse: bool True if test pulse is assumed

Returns

start,end: int tuple

ipfx.epochs.**get_sweep_epoch**(*response*)
Defined as interval including entire sweep

Parameters

response: float np.array

Returns

(start_index,end_index): int tuple with start,end indices of the epoch

ipfx.epochs.**get_test_epoch**(*i*, *hz*)

Find index range of the test epoch

Parameters

i [float np.array] current trace

Returns

start_idx,end_idx: int tuple start,end indices of the epoch

hz: float sampling rate

8.9 ipfx.error module

exception ipfx.error.**FeatureError**

Bases: Exception

Generic Python-exception-derived object raised by feature detection functions.

8.10 ipfx.feature_extractor module

class ipfx.feature_extractor.**SpikeFeatureExtractor**(*start=None*, *end=None*, *filter=10.0*, *dv_cutoff=20.0*, *max_interval=0.005*, *min_height=2.0*, *min_peak=-30.0*, *thresh_frac=0.05*, *reject_at_stim_start_interval=0*)

Bases: object

Methods

toctree references unknown document 'ipfx.feature_extractor.SpikeFeatureExtractor.spike_feature'

toctree references unknown document 'ipfx.feature_extractor.SpikeFeatureExtractor.spike_feature_keys'

toctree references unknown document 'ipfx.feature_extractor.SpikeFeatureExtractor.spikes'

spike_feature(self, spikes_df, key[, ...])	Get specified feature for every spike.
spike_feature_keys(self, spikes_df)	Get list of every available spike feature.
spikes(self, spikes_df)	Get all features for each spike as a list of records.

is_spike_feature_affected_by_clipping	
process	

AFFECTED_BY_CLIPPING = ['trough_t', 'trough_v', 'trough_i', 'trough_index', 'downstrok

Feature calculation for a sweep (voltage and/or current time series).

is_spike_feature_affected_by_clipping(*self*, *key*)

process(*self*, *t*, *v*, *i*)

spike_feature(*self*, *spikes_df*, *key*, *include_clipped=False*, *force_exclude_clipped=False*)
Get specified feature for every spike.

Parameters

key [feature name]

include_clipped: return values for every identified spike, even when clipping means they will be incorrect/un

Returns

spike_feature_values [ndarray of features for each spike]

spike_feature_keys(*self*, *spikes_df*)
Get list of every available spike feature.

spikes(*self*, *spikes_df*)
Get all features for each spike as a list of records.

class ipfx.feature_extractor.**SpikeTrainFeatureExtractor**(*start*, *end*, *burst_tol=0.5*, *pause_cost=1.0*, *deflect_type=None*, *stim_amp_fn=None*, *baseline_interval=0.1*, *filter_frequency=1.0*, *sag_baseline_interval=0.03*, *peak_width=0.005*)

Bases: object

Methods

process	

process(*self*, *t*, *v*, *i*, *spikes_df*, *extra_features=None*, *exclude_clipped=False*)

8.11 ipfx.feature_record module

ipfx.feature_record.**add_features_to_record**(*feature_names*, *feature_data*, *cell_record*, *postfix=None*)

ipfx.feature_record.**build_cell_feature_record**(*cell_features*)

ipfx.feature_record.**build_sweep_feature_record**(*sweep_table*, *sweep_features*)

ipfx.feature_record.**convert_units**(*cell_record*)

ipfx.feature_record.**nan_get**(*obj*, *key*)
Return a value from a dictionary. If it does not exist, return None. If it is NaN, return None

8.12 ipfx.feature_vectors module

ipfx.feature_vectors.**first_ap_vectors**(*sweeps_list*, *spike_info_list*, *target_sampling_rate=50000*, *window_length=0.003*, *skip_clipped=False*)

Average waveforms of first APs from sweeps

Parameters

sweeps_list: list List of Sweep objects

spike_info_list: list List of spike info DataFrames

target_sampling_rate: float (optional, default 50000) Desired sampling rate of output (Hz)

window_length: float (optional, default 0.003) Length of AP waveform (seconds)

Returns

ap_v: array of shape (target_sampling_rate * window_length) Waveform of average AP

ap_dv: array of shape (target_sampling_rate * window_length - 1) Waveform of first derivative of ap_v

ipfx.feature_vectors.**first_ap_waveform**(*sweep*, *spikes*, *length_in_points*)

Waveform of first AP with *length_in_points* time samples

Parameters

sweep: Sweep

Sweep object with spikes

Block quote ends without a blank line; unexpected unindent.

spikes: DataFrame Spike info dataframe with "threshold_index" column

length_in_points: int Length of returned AP waveform

Returns

first_ap_v: array of shape (length_in_points) The waveform of the first AP in *sweep*

ipfx.feature_vectors.**identify_subthreshold_depol_with_amplitudes**(*features*, *sweeps*)

Identify subthreshold responses from depolarizing steps

Parameters

features: dict Output of LongSquareAnalysis.analyze()

sweeps: SweepSet Long square sweeps

Returns

amp_sweep_dict: dict Amplitude-sweep pairs

deflect_dict: dict Dictionary of (base, deflect) tuples with amplitudes as keys

ipfx.feature_vectors.**identify_subthreshold_hyperpol_with_amplitudes**(*features*, *sweeps*)

Identify subthreshold responses from hyperpolarizing steps

Parameters

features: dict Output of LongSquareAnalysis.analyze()

sweeps: SweepSet Long square sweeps

Returns

amp_sweep_dict: dict Amplitude-sweep pairs

deflect_dict: dict Dictionary of (base, deflect) tuples with amplitudes as keys

ipfx.feature_vectors.**identify_suprathreshold_spike_info**(*features*, *target_amplitudes*, *shift=None*, *amp_tolerance=0*)

Find spike information for sweeps matching desired amplitudes relative to rheobase

Parameters

features: dict Output of LongSquareAnalysis.analyze()

target_amplitudes: array Amplitudes (relative to rheobase) for each desired step

shift: float (optional, default None) Amount to consider shifting "rheobase" to identify more matching sweeps if only a single sweep matches. A value of None means that no shift is attempted.

amp_tolerance: float (optional, default 0) Tolerance for matching amplitude (pA)

Returns

info_list: list Spike info in order of desired amplitudes. If a given amplitude cannot be found, the list has *None* at that location

ipfx.feature_vectors.**identify_suprathreshold_sweeps**(*sweeps*, *features*, *target_amplitudes*, *shift=None*, *amp_tolerance=0*)

Find spike information for sweeps matching desired amplitudes relative to rheobase

Parameters

sweeps: Sweep set Long square sweeps

features: dict Output of LongSquareAnalysis.analyze()

target_amplitudes: array Amplitudes (relative to rheobase) for each desired step

shift: float (optional, default None) Amount to consider shifting "rheobase" to identify more matching sweeps if only a single sweep matches. A value of None means that no shift is attempted.

amp_tolerance: float (optional, default 0) Tolerance for matching amplitude (pA)

Returns

sweeps: list Sweeps in order of desired amplitudes. If a given amplitude cannot be found, the list has *None* at that location

ipfx.feature_vectors.**identify_sweep_for_isi_shape**(*sweeps*, *features*, *duration*, *min_spike=5*)

Find lowest-amplitude spiking sweep that has at least min_spike or else sweep with most spikes

Parameters

sweeps: SweepSet Sweeps to consider for ISI shape calculation

features: dict Output of LongSquareAnalysis.analyze()

duration: float Length of stimulus interval (seconds)

min_spike: int (optional, default 5) Minimum number of spikes for first preference sweep (default 5)

Returns

selected_sweep: Sweep Sweep object for ISI shape calculation

selected_spike_info: DataFrame Spike info for selected sweep

ipfx.feature_vectors.**inst_freq_vector**(*spike_info_list*, *start*, *end*, *width=20*)

Create binned instantaneous frequency feature vector, concatenated across sweeps

Parameters

spike_info_list: list Spike info DataFrames for each sweep

start: float Start of stimulus interval (seconds)

end: float End of stimulus interval (seconds)

width: float (optional, default 20) Bin width in ms

Returns

output: array Concatenated vector of binned instantaneous firing rates (spikes/s)

ipfx.feature_vectors.**isi_shape**(*sweep*, *spike_info*, *end*, *n_points=100*, *steady_state_interval=0.1*, *single_return_tolerance=1.0*, *single_max_duration=0.1*)

Average interspike voltage trajectory with normalized duration, aligned to threshold

Parameters

sweep: Sweep Sweep object with at least one action potential

spike_info: DataFrame Spike info for sweep

end: float End of stimulus interval (seconds)

n_points: int (optional, default 100) Number of points in output

steady_state_interval: float (optional, default 0.1) Interval for calculating steady-state for sweeps with only one spike (seconds)

single_return_tolerance: float (optional, default 1) Allowable difference from steady-state for determining end of "ISI" if only one spike is in sweep (mV)

single_max_duration: float (optional, default 0.1) Allowable max duration for finding end of "ISI" if only one spike is in sweep (seconds)

Returns

isi_norm: array of shape (n_points) Averaged, threshold-aligned, duration-normalized voltage trace

ipfx.feature_vectors.**noise_ap_features**(*noise_sweeps, stim_interval_list=[(2.02, 5.02), (10.02, 13.02), (18.02, 21.02)], target_sampling_rate=50000, window_length=0.003, skip_first_n=1*)

Average AP waveforms in noise sweeps

Parameters

noise_sweeps: SweepSet Noise sweeps

stim_interval_list: list Tuples of start and end times (in seconds) of analysis intervals

target_sampling_rate: float (optional, default 50000) Desired sampling rate of output (Hz)

window_length: float (optional, default 0.003) Length of AP waveform (seconds)

skip_first_n: int (optional, default 1) Number of initial APs to exclude from average (default 1)

Returns

ap_v: array of shape (target_sampling_rate * window_length) Waveform of average AP

ap_dv: array of shape (target_sampling_rate * window_length - 1) Waveform of first derivative of ap_v

ipfx.feature_vectors.**psth_vector**(*spike_info_list*, *start*, *end*, *width=50*)

Create binned "PSTH"-like feature vector based on spike times, concatenated across sweeps

Parameters

spike_info_list: list Spike info DataFrames for each sweep

start: float Start of stimulus interval (seconds)

end: float End of stimulus interval (seconds)

width: float (optional, default 50) Bin width in ms

Returns

output: array Concatenated vector of binned spike rates (spikes/s)

ipfx.feature_vectors.**spike_feature_vector**(*feature*, *spike_info_list*, *start*, *end*, *width=20*)

Create binned feature vector for specified features, concatenated across sweeps

Parameters

feature: string Name of feature found in members of spike_info_list

spike_info_list: list Spike info DataFrames for each sweep

start: float Start of stimulus interval (seconds)

end: float End of stimulus interval (seconds)

width: float (optional, default 20) Bin width in ms

Returns

output: array Concatenated vector of binned spike features

ipfx.feature_vectors.**step_subthreshold**(*amp_sweep_dict*, *target_amps*, *start*, *end*, *extend_duration=0.2*, *subsample_interval=0.01*, *amp_tolerance=0.0*)

Subsample set of subthreshold step responses including regions before and after step

Parameters

amp_sweep_dict [dict] Amplitude-sweep pairs

target_amps: list Desired amplitudes for output vector

start: float start stimulus interval (seconds)

end: float end of stimulus interval (seconds)

extend_duration: float (optional, default 0.2) Duration to extend sweep before and after stimulus interval (seconds)

subsample_interval: float (optional, default 0.01) Size of subsampled bins (seconds)

amp_tolerance: float (optional, default 0) Tolerance for finding matching amplitudes

Returns

output_vector: subsampled, concatenated voltage trace

ipfx.feature_vectors.**subthresh_depol_norm**(*amp_sweep_dict*, *deflect_dict*, *start*, *end*, *extend_duration=0.2*, *subsample_interval=0.01*, *steady_state_interval=0.1*)

Largest positive-going subthreshold step response that does not evoke spikes, normalized to baseline and steady-state at end of step

Parameters

amp_sweep_dict: dict Amplitude-sweep pairs

deflect_dict: Dictionary of (baseline, deflect) tuples with amplitude keys

start: float start stimulus interval (seconds)

end: float end of stimulus interval (seconds)

extend_duration: float (optional, default 0.2) Duration to extend sweep on each side of stimulus interval (seconds)

subsample_interval: float (optional, default 0.01) Size of subsampled bins (seconds)

steady_state_interval: float (optional, default 0.1) Interval before end for normalization (seconds)

Returns

subsampled_v: array Subsampled, normalized voltage trace

ipfx.feature_vectors.**subthresh_norm**(*amp_sweep_dict*, *deflect_dict*, *start*, *end*, *target_amp=-101.0*, *extend_duration=0.2*, *subsample_interval=0.01*)

Subthreshold step response closest to target amplitude normalized to baseline and peak deflection

Parameters

amp_sweep_dict: dict Amplitude-sweep pairs

deflect_dict: Dictionary of (baseline, deflect) tuples with amplitude keys

start: float start stimulus interval (seconds)

end: float end of stimulus interval (seconds)

target_amp: float (optional, default=-101) Search target for amplitude (pA)

extend_duration: float (optional, default 0.2) Duration to extend sweep on each side of stimulus interval (seconds)

subsample_interval: float (optional, default 0.01) Size of subsampled bins (seconds)

Returns

subsampled_v: array Subsampled, normalized voltage trace

8.13 ipfx.lab_notebook_reader module

A shim for backwards compatible imports of lab_notebook_reader

8.14 ipfx.lims_queries module

ipfx.lims_queries.**able_to_connect_to_lims**()

ipfx.lims_queries.**get_igorh5_path_from_lims**(*ephys_roi_result*)

ipfx.lims_queries.**get_input_h5_file**(*specimen_id*)

ipfx.lims_queries.**get_input_nwb_file**(*specimen_id*)

ipfx.lims_queries.**get_nwb_path_from_lims**(*ephys_roi_result*)

Try to find NWBIgor file preferentially If not found, look for a processed NWB file

well known file type ID for NWB files is 475137571 well known file type ID for NWBIgor files is 570280085

Parameters

ephys_roi_result: int

Returns

full path of the nwb file

ipfx.lims_queries.**get_specimen_info_from_lims_by_id**(*specimen_id*)

ipfx.lims_queries.**get_stimuli_description**()

ipfx.lims_queries.**get_sweep_states**(*specimen_id*)

ipfx.lims_queries.**project_specimen_ids**(*project*, *passed_only=True*)

ipfx.lims_queries.**query**(*query*, *parameters=None*)

8.15 ipfx.logging_utils module

ipfx.logging_utils.**configure_logger**(*cell_dir*)

ipfx.logging_utils.**log_pretty_header**(*header*, *level=1*, *top_line_break=True*, *bottom_line_break=True*)

Decorate logging message to make logging output more human readable

Parameters

header: str header message

level: int 1 or 2 as in markdown

top_line_break: bool (True) add a blank line at the top

bottom_line_break: bool (True) add a blank line at the bottom

8.16 ipfx.nwb_append module

ipfx.nwb_append.**append_spike_times**(*input_nwb_path:Union[str, pathlib.Path], sweep_spike_times:Dict[int, List[float]], output_nwb_path:Union[str, pathlib.Path, NoneType]=None*)

Appends spiketimes to an nwb2 file

8.17 ipfx.plot_qc_figures module

ipfx.plot_qc_figures.**display_features**(*qc_fig_dir*, *data_set*, *feature_data*)

Parameters

qc_fig_dir: str directory name for storing html pages

data_set: NWB data set

feature_data: dict cell and sweep features

ipfx.plot_qc_figures.**exp_curve**(*x*, *a*, *inv_tau*, *y0*)

Function used for tau curve fitting

ipfx.plot_qc_figures.**get_features**(*sweep_features*, *sweep_number*)

ipfx.plot_qc_figures.**get_spikes**(*sweep_features*, *sweep_number*)

ipfx.plot_qc_figures.**get_time_string**()

ipfx.plot_qc_figures.**load_sweep**(*data_set*, *sweep_number*)

ipfx.plot_qc_figures.**make_cell_html**(*image_files*, *metadata*, *file_name*, *img_sub_dir*, *required_fields=('electrode_0_pa', 'seal_gohm', 'initial_access_resistance_mohm', 'input_resistance_mohm')*)

ipfx.plot_qc_figures.**make_cell_page**(*data_set*, *feature_data*, *working_dir*, *save_cell_plots=True*)

ipfx.plot_qc_figures.**make_sweep_html**(*sweep_files*, *file_name*, *img_sub_dir*)

ipfx.plot_qc_figures.**make_sweep_page**(*data_set*, *working_dir*)

ipfx.plot_qc_figures.**mask_nulls**(*data*)

ipfx.plot_qc_figures.**plot_cell_figures**(*data_set*, *figure_data*, *image_dir*, *sizes*)

ipfx.plot_qc_figures.**plot_fi_curve_figures**(*data_set*, *cell_features*, *lims_features*, *sweep_features*, *image_dir*, *sizes*, *cell_image_files*)

ipfx.plot_qc_figures.**plot_hero_figures**(*data_set*, *cell_features*, *lims_features*, *sweep_features*, *image_dir*, *sizes*, *cell_image_files*)

ipfx.plot_qc_figures.**plot_images**(*image_dir*, *sizes*, *image_sets*)

ipfx.plot_qc_figures.**plot_instantaneous_threshold_thumbnail**(*data_set*, *sweep_numbers*, *cell_features*, *lims_features*, *sweep_features*, *color='red'*)

ipfx.plot_qc_figures.**plot_long_square_summary**(*data_set*, *cell_features*, *lims_features*)

ipfx.plot_qc_figures.**plot_ramp_figures**(*data_set*, *cell_features*, *lims_features*, *sweep_features*, *image_dir*, *sizes*, *cell_image_files*)

ipfx.plot_qc_figures.**plot_rheo_figures**(*data_set*, *cell_features*, *lims_features*, *sweep_features*, *image_dir*, *sizes*, *cell_image_files*)

ipfx.plot_qc_figures.**plot_sag_figures**(*data_set*, *cell_features*, *lims_features*, *sweep_features*, *image_dir*, *sizes*, *cell_image_files*)

ipfx.plot_qc_figures.**plot_short_square_figures**(*data_set*, *cell_features*, *lims_features*, *sweep_features*, *image_dir*, *sizes*, *cell_image_files*)

ipfx.plot_qc_figures.**plot_single_ap_values**(*data_set*, *sweep_numbers*, *lims_features*, *sweep_features*, *cell_features*, *type_name*)

ipfx.plot_qc_figures.**plot_subthreshold_long_square_figures**(*data_set*, *cell_features*, *lims_features*, *sweep_features*, *image_dir*, *sizes*, *cell_image_files*)

ipfx.plot_qc_figures.**plot_sweep_figures**(*data_set*, *image_dir*, *sizes*)

ipfx.plot_qc_figures.**plot_sweep_set_summary**(*data_set*, *highlight_sweep_number*, *sweep_numbers*, *highlight_color='#0779BE'*, *background_color='#dddddd'*)

ipfx.plot_qc_figures.**plot_sweep_value_figures**(*sweeps*, *image_dir*, *sizes*, *cell_image_files*)

ipfx.plot_qc_figures.**save_figure**(*fig*, *image_name*, *image_set_name*, *image_dir*, *sizes*, *image_sets*, *scalew=1*, *scaleh=1*, *ext='png'*)

8.18 ipfx.qc_feature_evaluator module

ipfx.qc_feature_evaluator.**evaluate_blowout**(*blowout_mv*, *blowout_mv_min*, *blowout_mv_max*, *fail_tags*)

ipfx.qc_feature_evaluator.**evaluate_electrode_0**(*electrode_0_pa*, *electrode_0_pa_max*, *fail_tags*)

ipfx.qc_feature_evaluator.**evaluate_input_and_access_resistance**(*input_access_resistance_ratio*, *input_vs_access_resistance_max*, *initial_access_resistance_mohm*, *access_resistance_mohm_min*, *access_resistance_mohm_max*, *fail_tags*)

ipfx.qc_feature_evaluator.**evaluate_seal**(*seal_gohm*, *seal_gohm_min*, *fail_tags*)

ipfx.qc_feature_evaluator.**load_default_qc_criteria**()

ipfx.qc_feature_evaluator.**qc_cell**(*cell_data*, *qc_criteria=None*)

Evaluate cell state across different types of stimuli

Parameters

cell_data [dict] cell features

qc_criteria [dict] qc criteria

Returns

cell_state [dict] cell state including qc features

ipfx.qc_feature_evaluator.**qc_current_clamp_sweep**(*sweep*, *is_ramp*, *qc_criteria=None*)

QC for the current-clamp sweeps

Parameters

is_ramp: bool True for Ramp otherwise False

sweep [dict] features of a sweep

qc_criteria [dict] qc criteria

Returns

fails [int] number of fails

fail_tags [list of str] tags of the failed sweeps

ipfx.qc_feature_evaluator.**qc_experiment**(*ontology*, *cell_features*, *sweep_features*, *qc_criteria=None*)

Parameters

ontology: StimulusOntology object stimulus ontology

cell_features [dict] cell features

sweep_features: list of dicts sweep features

qc_criteria [dict] qc criteria

Returns

cell_state [list] sweep_states : list

ipfx.qc_feature_evaluator.**qc_sweeps**(*ontology*, *sweep_features*, *qc_criteria*)

8.19 ipfx.qc_feature_extractor module

ipfx.qc_feature_extractor.**cell_qc_features**(*data_set*, *manual_values=None*)

Parameters

data_set [EphysDataSet] dataset

manual_values [dict] default (manual) values that can be passed in through input.json.

Returns

features [dict] cell qc features

tags [list] warning tags

ipfx.qc_feature_extractor.**check_sweep_integrity**(*sweep*, *is_ramp*)

ipfx.qc_feature_extractor.**compute_input_access_resistance_ratio**(*ir*, *sr*)

ipfx.qc_feature_extractor.**current_clamp_sweep_qc_features**(*sweep*, *is_ramp*)

ipfx.qc_feature_extractor.**current_clamp_sweep_stim_features**(*sweep*)

ipfx.qc_feature_extractor.**extract_blowout**(*data_set*, *tags*)
Measure blowout voltage

Parameters

data_set: EphysDataSet

tags: list warning tags

Returns

blowout_mv: float blowout voltage in mV

ipfx.qc_feature_extractor.**extract_clamp_seal**(*data_set*, *tags*, *manual_values=None*)

Parameters

data_set: EphysDataSet

tags: list warning tags

manual_values

ipfx.qc_feature_extractor.**extract_electrode_0**(*data_set*, *tags*)
Measure electrode zero

Parameters

data_set: EphysDataSet

tags: list warning tags

Returns

e0: float

ipfx.qc_feature_extractor.**extract_initial_access_resistance**(*breakin_sweep*, *tags*, *manual_values*)

ipfx.qc_feature_extractor.**extract_input_and_access_resistance**(*data_set*, *tags*, *manual_values=None*)

Measure input and series (access) resistance in two steps:

1. finding the breakin sweep
2. and then analyzing it

if the value is unavailable then check to see if it was set manually

Parameters

data_set: EphysDataSet

tags: list warning tags

manual_values: dict manual/default values

Returns

ir: float input resistance

sr: float access resistance

ipfx.qc_feature_extractor.**extract_input_resistance**(*breakin_sweep*, *tags*, *manual_values*)

ipfx.qc_feature_extractor.**extract_recording_date**(*data_set*, *tags*)

ipfx.qc_feature_extractor.**sweep_qc_features**(*data_set*)

8.20 ipfx.qc_features module

ipfx.qc_features.**get_r_from_peak_pulse_response**(*v*, *i*, *t*)

ipfx.qc_features.**get_r_from_stable_pulse_response**(*v*, *i*, *t*)

Compute input resistance from the stable pulse response

Parameters

v [float membrane voltage (V)]

i [float input current (A)]

t [time (s)]

Returns

ir: float input resistance

ipfx.qc_features.**get_square_pulse_idx**(*v*)

Get up and down indices of the square pulse(s). Skipping the very first pulse (test pulse)

Parameters

v: float pulse trace

Returns

up_idx, down_idx: list, list up, down indices

ipfx.qc_features.**measure_blowout**(*v*, *idx0*)

ipfx.qc_features.**measure_electrode_0**(*curr*, *hz*, *t=0.005*)

ipfx.qc_features.**measure_initial_access_resistance**(*v*, *curr*, *t*)

ipfx.qc_features.**measure_input_resistance**(*v*, *curr*, *t*)

ipfx.qc_features.**measure_seal**(*v*, *curr*, *t*)

ipfx.qc_features.**measure_vm**(*vals*)

ipfx.qc_features.**measure_vm_delta**(*mean_start*, *mean_end*)

8.21 ipfx.script_utils module

ipfx.script_utils.**categorize_iclamp_sweeps**(*data_set*, *stimuli_names*, *sweep_qc_option='none'*, *specimen_id=None*)

ipfx.script_utils.**dataset_for_specimen_id**(*specimen_id*, *data_source*, *ontology*, *file_list=None*)

ipfx.script_utils.**filter_results**(*specimen_ids*, *results*)

ipfx.script_utils.**lims_nwb_information**(*specimen_id*)

ipfx.script_utils.**organize_results**(*specimen_ids*, *results*)

Build dictionary of results, filling data from cells with appropriate-length nan arrays where needed

ipfx.script_utils.**preprocess_long_square_sweeps**(*data_set*, *sweep_numbers*, *extra_dur=0.2*, *subthresh_min_amp=-100.0*)

ipfx.script_utils.**preprocess_ramp_sweeps**(*data_set*, *sweep_numbers*)

ipfx.script_utils.**preprocess_short_square_sweeps**(*data_set*, *sweep_numbers*, *extra_dur=0.2*, *spike_window=0.05*)

ipfx.script_utils.**save_errors_to_json**(*error_set*, *output_dir*, *output_code*)

ipfx.script_utils.**save_results_to_h5**(*specimen_ids*, *results_dict*, *output_dir*, *output_code*)

ipfx.script_utils.**save_results_to_npy**(*specimen_ids*, *results_dict*, *output_dir*, *output_code*)

ipfx.script_utils.**sdk_nwb_information**(*specimen_id*)

ipfx.script_utils.**validate_sweeps**(*data_set*, *sweep_numbers*, *extra_dur=0.2*)

8.22 ipfx.spike_detector module

ipfx.spike_detector.**check_thresholds_and_peaks**(*v*, *t*, *spike_indexes*, *peak_indexes*, *upstroke_indexes*, *start=None*, *end=None*, *max_interval=0.005*, *thresh_frac=0.05*, *filter=10.0*, *dvdt=None*, *tol=1.0*, *reject_at_stim_start_interval=0.0*)

Validate thresholds and peaks for set of spikes

Check that peaks and thresholds for consecutive spikes do not overlap Spikes with overlapping thresholds and peaks will be merged.

Check that peaks and thresholds for a given spike are not too far apart.

Parameters

v [numpy array of voltage time series in mV]

t [numpy array of times in seconds]

spike_indexes [numpy array of spike indexes]

peak_indexes [numpy array of indexes of spike peaks]

upstroke_indexes [numpy array of indexes of spike upstrokes]

start [start of time window for feature analysis (optional)]

end [end of time window for feature analysis (optional)]

max_interval [maximum allowed time between start of spike and time of peak in sec (default 0.005)]

thresh_frac [fraction of average upstroke for threshold calculation (optional, default 0.05)]

filter [cutoff frequency for 4-pole low-pass Bessel filter in kHz (optional, default 10)]

dvdt [pre-calculated time-derivative of voltage (optional)]

tol [tolerance for returning to threshold in mV (optional, default 1)]

reject_at_stim_start_interval [duration of window after start to reject potential spikes (optional, default 0)]

Returns

spike_indexes [numpy array of modified spike indexes]

peak_indexes [numpy array of modified spike peak indexes]

upstroke_indexes [numpy array of modified spike upstroke indexes]

clipped [numpy array of clipped status of spikes]

ipfx.spike_detector.**detect_putative_spikes**(*v*, *t*, *start=None*, *end=None*, *filter=10.0*, *dv_cutoff=20.0*, *dvdt=None*)

Perform initial detection of spikes and return their indexes.

Parameters

v [numpy array of voltage time series in mV]

t [numpy array of times in seconds]

start [start of time window for spike detection (optional)]

end [end of time window for spike detection (optional)]

filter [cutoff frequency for 4-pole low-pass Bessel filter in kHz (optional, default 10)]

dv_cutoff [minimum dV/dt to qualify as a spike in V/s (optional, default 20)]

dvdt [pre-calculated time-derivative of voltage (optional)]

Returns

putative_spikes [numpy array of preliminary spike indexes]

ipfx.spike_detector.**filter_putative_spikes**(*v*, *t*, *spike_indexes*, *peak_indexes*, *min_height=2.0*, *min_peak=-30.0*, *filter=10.0*, *dvdt=None*)

Filter out events that are unlikely to be spikes based on:

- Height (threshold to peak)
- Absolute peak level

Parameters

v [numpy array of voltage time series in mV]

t [numpy array of times in seconds]

spike_indexes [numpy array of preliminary spike indexes]

peak_indexes [numpy array of indexes of spike peaks]

min_height [minimum acceptable height from threshold to peak in mV (optional, default 2)]

min_peak [minimum acceptable absolute peak level in mV (optional, default -30)]

filter [cutoff frequency for 4-pole low-pass Bessel filter in kHz (optional, default 10)]

dvdt [pre-calculated time-derivative of voltage (optional)]

Returns

spike_indexes [numpy array of threshold indexes]

peak_indexes [numpy array of peak indexes]

ipfx.spike_detector.**find_clipped_spikes**(*v*, *t*, *spike_indexes*, *peak_indexes*, *end_index*, *tol*)

Check that last spike was not cut off too early by end of stimulus by checking that the membrane potential returned to at least the threshold voltage - otherwise, drop it

Parameters

v [numpy array of voltage time series in mV]

t [numpy array of times in seconds]

spike_indexes [numpy array of spike indexes]

peak_indexes [numpy array of indexes of spike peaks]

end_index: int index of the end of time window for feature analysis

tol: float tolerance to returning to threshold

Returns

clipped: Boolean np.array

ipfx.spike_detector.**find_downstroke_indexes**(*v*, *t*, *peak_indexes*, *trough_indexes*, *clipped=None*, *filter=10.0*, *dvdt=None*)

Find indexes of minimum voltage (troughs) between spikes.

Parameters

v [numpy array of voltage time series in mV]

t [numpy array of times in seconds]

peak_indexes [numpy array of spike peak indexes]

trough_indexes [numpy array of threshold indexes]

clipped: boolean array - False if spike not clipped by edge of window

filter [cutoff frequency for 4-pole low-pass Bessel filter in kHz (optional, default 10)]

dvdt [pre-calculated time-derivative of voltage (optional)]

Returns

downstroke_indexes [numpy array of downstroke indexes]

ipfx.spike_detector.**find_peak_indexes**(*v*, *t*, *spike_indexes*, *end=None*)

Find indexes of spike peaks.

Parameters

v [numpy array of voltage time series in mV]

t [numpy array of times in seconds]

spike_indexes [numpy array of preliminary spike indexes]

end [end of time window for spike detection (optional)]

ipfx.spike_detector.**find_trough_indexes**(*v*, *t*, *spike_indexes*, *peak_indexes*, *clipped=None*, *end=None*)

Find indexes of minimum voltage (trough) between spikes.

Parameters

v [numpy array of voltage time series in mV]

t [numpy array of times in seconds]

spike_indexes [numpy array of spike indexes]

peak_indexes [numpy array of spike peak indexes]

end [end of time window (optional)]

Returns

trough_indexes [numpy array of threshold indexes]

ipfx.spike_detector.**find_upstroke_indexes**(*v*, *t*, *spike_indexes*, *peak_indexes*, *filter=10.0*, *dvdt=None*)

Find indexes of maximum upstroke of spike.

Parameters

v [numpy array of voltage time series in mV]

t [numpy array of times in seconds]

spike_indexes [numpy array of preliminary spike indexes]

peak_indexes [numpy array of indexes of spike peaks]

filter [cutoff frequency for 4-pole low-pass Bessel filter in kHz (optional, default 10)]

dvdt [pre-calculated time-derivative of voltage (optional)]

Returns

upstroke_indexes [numpy array of upstroke indexes]

ipfx.spike_detector.**refine_threshold_indexes**(*v*, *t*, *upstroke_indexes*, *thresh_frac=0.05*, *filter=10.0*, *dvdt=None*)

Refine threshold detection of previously-found spikes.

Parameters

v [numpy array of voltage time series in mV]

t [numpy array of times in seconds]

upstroke_indexes [numpy array of indexes of spike upstrokes (for threshold target calculation)]

thresh_frac [fraction of average upstroke for threshold calculation (optional, default 0.05)]

filter [cutoff frequency for 4-pole low-pass Bessel filter in kHz (optional, default 10)]

dvdt [pre-calculated time-derivative of voltage (optional)]

Returns

threshold_indexes [numpy array of threshold indexes]

8.23 ipfx.spike_features module

ipfx.spike_features.**analyze_trough_details**(*v*, *t*, *spike_indexes*, *peak_indexes*, *clipped=None*, *end=None*, *filter=10.0*, *heavy_filter=1.0*, *term_frac=0.01*, *adp_thresh=0.5*, *tol=0.5*, *flat_interval=0.002*, *adp_max_delta_t=0.005*, *adp_max_delta_v=10.0*, *dvdt=None*)

Analyze trough to determine if an ADP exists and whether the reset is a 'detour' or 'direct'

Parameters

v [numpy array of voltage time series in mV]

t [numpy array of times in seconds]

spike_indexes [numpy array of spike indexes]

peak_indexes [numpy array of spike peak indexes]

end [end of time window (optional)]

filter [cutoff frequency for 4-pole low-pass Bessel filter in kHz (default 1)]

heavy_filter [lower cutoff frequency for 4-pole low-pass Bessel filter in kHz (default 1)]

thresh_frac [fraction of average upstroke for threshold calculation (optional, default 0.05)]

adp_thresh: minimum dV/dt in V/s to exceed to be considered to have an ADP (optional, default 1.5)

tol [tolerance for evaluating whether Vm drops appreciably further after end of spike (default 1.0 mV)]

flat_interval: if the trace is flat for this duration, stop looking for an ADP (default 0.002 s)

adp_max_delta_t: max possible ADP delta t (default 0.005 s)

adp_max_delta_v: max possible ADP delta v (default 10 mV)

dvdt [pre-calculated time-derivative of voltage (optional)]

Returns

isi_types [numpy array of isi reset types (direct or detour)]

fast_trough_indexes [numpy array of indexes at the start of the trough (i.e. end of the spike)]

adp_indexes [numpy array of adp indexes (np.nan if there was no ADP in that ISI]

slow_trough_indexes [numpy array of indexes at the minimum of the slow phase of the trough] (if there wasn't just a fast phase)

ipfx.spike_features.**estimate_adjusted_detection_parameters**(*v_set*, *t_set*, *interval_start*, *interval_end*, *filter=10*)

Estimate adjusted values for spike detection by analyzing a period when the voltage changes quickly but passively (due to strong current stimulation), which can result in spurious spike detection results.

Parameters

v_set [list of numpy arrays of voltage time series in mV]

t_set [list of numpy arrays of times in seconds]

interval_start [start of analysis interval (sec)]

interval_end [end of analysis interval (sec)]

Returns

new_dv_cutoff [adjusted dv/dt cutoff (V/s)]

new_thresh_frac [adjusted fraction of avg upstroke to find threshold]

ipfx.spike_features.**find_widths**(*v*, *t*, *spike_indexes*, *peak_indexes*, *trough_indexes*, *clipped=None*)

Find widths at half-height for spikes.

Widths are only returned when heights are defined

Parameters

v [numpy array of voltage time series in mV]

t [numpy array of times in seconds]

spike_indexes [numpy array of spike indexes]

peak_indexes [numpy array of spike peak indexes]

trough_indexes [numpy array of trough indexes]

Returns

widths [numpy array of spike widths in sec]

ipfx.spike_features.**fit_prespike_time_constant**(*t*, *v*, *start*, *spike_time*, *dv_limit=-0.001*, *tau_limit=0.3*)

Finds the dominant time constant of the pre-spike rise in voltage

Parameters

v [numpy array of voltage time series in mV]

t [numpy array of times in seconds]

start [start of voltage rise (seconds)]

spike_time [time of first spike (seconds)]

dv_limit [dV/dt cutoff (default -0.001)] Shortens fit window if rate of voltage drop exceeds this limit

tau_limit [upper bound for slow time constant (seconds, default 0.3)] If the slower time constant of a double-exponential fit is twice that of the faster and exceeds this limit, the faster one will be considered the dominant one

Returns

tau [dominant time constant (seconds)]

8.24 ipfx.spike_train_features module

ipfx.spike_train_features.**adaptation_index**(*isis*)

Calculate adaptation index of *isis*.

ipfx.spike_train_features.**average_rate**(*t*, *spikes*, *start*, *end*)

Calculate average firing rate during interval between *start* and *end*.

Parameters

t [numpy array of times in seconds]

spikes [numpy array of spike indexes]

start [start of time window for spike detection]

end [end of time window for spike detection]

Returns

avg_rate [average firing rate in spikes/sec]

ipfx.spike_train_features.**basic_spike_train_features**(*t*, *spikes_df*, *start*, *end*, *exclude_clipped=False*)

ipfx.spike_train_features.**burst**(*t*, *spikes_df*, *tol=0.5*, *pause_cost=1.0*)

Find bursts and return max "burstiness" index (normalized max rate in burst vs out).

Returns

max_burstiness_index [max "burstiness" index across detected bursts]

num_bursts [number of bursts detected]

ipfx.spike_train_features.**delay**(*t*, *v*, *spikes_df*, *start*, *end*)
Calculates ratio of latency to dominant time constant of rise before spike

Returns

delay_ratio [ratio of latency to tau (higher means more delay)]

tau [dominant time constant of rise before spike]

ipfx.spike_train_features.**detect_bursts**(*isis*, *isi_types*, *fast_tr_v*, *fast_tr_t*, *slow_tr_v*, *slow_tr_t*, *thr_v*, *tol=0.5*, *pause_cost=1.0*)
Detect bursts in spike train.

Parameters

isis [numpy array of n interspike intervals]

isi_types [numpy array of n interspike interval types]

fast_tr_v [numpy array of fast trough voltages for the n + 1 spikes of the train]

fast_tr_t [numpy array of fast trough times for the n + 1 spikes of the train]

slow_tr_v [numpy array of slow trough voltages for the n + 1 spikes of the train]

slow_tr_t [numpy array of slow trough times for the n + 1 spikes of the train]

thr_v [numpy array of threshold voltages for the n + 1 spikes of the train]

tol [tolerance for the difference in slow trough voltages and thresholds (default 0.5 mV)] Used to identify "delay" interspike intervals that occur within a burst

Returns

bursts [list of bursts] Each item in list is a tuple of the form (burst_index, start, end) where *burst_index* is a comparison index between the highest instantaneous rate within the burst vs the highest instantaneous rate outside the burst. *start* is the index of the first ISI of the burst, and *end* is the ISI index immediately following the burst.

ipfx.spike_train_features.**detect_pauses**(*isis*, *isi_types*, *cost_weight=1.0*)
Determine which ISIs are "pauses" in ongoing firing.

Pauses are unusually long ISIs with a "detour reset" among "direct resets".

Parameters

isis [numpy array of interspike intervals]

isi_types [numpy array of interspike interval types ('direct' or 'detour')]

cost_weight [weight for cost function for calling an ISI a pause] Higher cost weights lead to fewer ISIs identified as pauses. The cost function also depends on the difference between the duration of the "pause" ISIs and the average duration and standard deviation of "non-pause" ISIs.

Returns

pauses [numpy array of indices corresponding to pauses in *isis*]

ipfx.spike_train_features.**fit_fi_slope**(*stim_amps*, *avg_rates*)
Fit the rate and stimulus amplitude to a line and return the slope of the fit.

ipfx.spike_train_features.**get_isis**(*t*, *spikes*)
Find interspike intervals in sec between spikes (as indexes).

ipfx.spike_train_features.**latency**(*t*, *spikes*, *start*)
Calculate time to the first spike.

ipfx.spike_train_features.**norm_diff**(*a*)
Calculate average of (a[i] - a[i+1]) / (a[i] + a[i+1]).

ipfx.spike_train_features.**norm_sq_diff**(*a*)
Calculate average of (a[i] - a[i+1])^2 / (a[i] + a[i+1])^2.

ipfx.spike_train_features.**pause**(*t*, *spikes_df*, *start*, *end*, *cost_weight=1.0*)
Estimate average number of pauses and average fraction of time spent in a pause

Attempts to detect pauses with a variety of conditions and averages results together.

Pauses that are consistently detected contribute more to estimates.

Returns

avg_n_pauses [average number of pauses detected across conditions]

avg_pause_frac [average fraction of interval (between start and end) spent in a pause]

max_reliability [max fraction of times most reliable pause was detected given weights tested]

n_max_rel_pauses [number of pauses detected with *max_reliability*]

8.25 ipfx.stim_features module

ipfx.stim_features.**find_stim_interval**(*idx0*, *stim*, *hz*)

ipfx.stim_features.**get_stim_characteristics**(*i*, *t*, *test_pulse=True*)
Identify the start time, duration, amplitude, start index, and end index of a general stimulus.

8.26 ipfx.stimulus module

class ipfx.stimulus.**Stimulus**(*tag_sets*)
Bases: object

Methods

has_tag	
tags	

has_tag(*self*, *tag*, *tag_type=None*)

tags(*self*, *tag_type=None*, *flat=False*)

class ipfx.stimulus.**StimulusOntology**(*stim_ontology_tags=None*)
Bases: object

Methods

toctree references unknown document 'ipfx.stimulus.StimulusOntology.default'

toctree references unknown document 'ipfx.stimulus.StimulusOntology.find'

toctree references unknown document 'ipfx.stimulus.StimulusOntology.stimulus_has_any_tags'

default()	Construct an ontology object using default tags
find(self, tag[, tag_type])	Find stimuli matching a given tag Parameters ———- tag: str tag_type: str
stimulus_has_any_tags(self, stim, tags[, ...])	Find stimulus based on a tag stim and then check if it has any tags Parameters ———- stim: str tag to find stimulus tags: str tags to check in any belong to the stimulus tag_type

find_one	
stimulus_has_all_tags	

DEFAULT_STIMULUS_ONTOLOGY_FILE = '/home/docs/checkouts/readthedocs.org/user_builds/ipf

classmethod default()
Construct an ontology object using default tags

find(*self*, *tag*, *tag_type=None*)
Find stimuli matching a given tag Parameters ———- tag: str tag_type: str

Returns

matching_stims: list of Stimuli objects

find_one(*self*, *tag*, *tag_type=None*)

stimulus_has_all_tags(*self*, *stim*, *tags*, *tag_type=None*)

stimulus_has_any_tags(*self*, *stim*, *tags*, *tag_type=None*)
Find stimulus based on a tag stim and then check if it has any tags Parameters ———- stim: str

Unexpected indentation.

tag to find stimulus

Block quote ends without a blank line; unexpected unindent.

tags: str tags to check in any belong to the stimulus

Definition list ends without a blank line; unexpected unindent.

tag_type

Returns

bool: True if any tags match, otherwise False

class ipfx.stimulus.**StimulusType**
Bases: enum.Enum

An enumeration.

BATH = 'bath'

BLOWOUT = 'blowout'

BREAKIN = 'breakin'

CHIRP = 'chirp'

COARSE_LONG_SQUARE = 'coarse_long_square'

EXTP = 'extp'

LONG_SQUARE = 'long_square'

RAMP = 'ramp'

SEAL = 'seal'

SEARCH = 'search'

SHORT_SQUARE = 'short_square'

SHORT_SQUARE_TRIPLE = 'short_square_triple'

TEST = 'test'

ipfx.stimulus.**get_stimulus_type**(*stimulus_name*)

8.27 ipfx.stimulus_protocol_analysis module

class ipfx.stimulus_protocol_analysis.**LongSquareAnalysis**(*spx*, *sptx*, *subthresh_min_amp*, *tau_frac=0.1*, *require_subthreshold=True*, *require_suprathreshold=True*)

Bases: *ipfx.stimulus_protocol_analysis.StimulusProtocolAnalysis*

Methods

toctree references unknown document 'ipfx.stimulus_protocol_analysis.LongSquareAnalysis.mean_features_first_spike'

mean_features_first_spike(self, spikes_set)	Compute mean feature values for the first spike in list of extractors

all_sweep_features	
analyze	
analyze_basic_features	
analyze_subthreshold	
analyze_suprathreshold	
as_dict	
find_hero_sweep	
find_rheobase_sweep	
reset_basic_features	
subthreshold_sweep_features	
suprathreshold_sweep_features	

HERO_MAX_AMP_OFFSET = 61.0

HERO_MIN_AMP_OFFSET = 39.0

SAG_TARGET = -100.0

SUBTHRESH_MAX_AMP = 0

analyze(*self*, *sweep_set*)

analyze_subthreshold(*self*, *sweep_set*)

analyze_suprathreshold(*self*, *sweep_set*)

as_dict(*self*, *features*, *extra_params=None*)

find_hero_sweep(*self*, *rheo_amp*, *spiking_features*, *min_offset=39.0*, *max_offset=61.0*)

find_rheobase_sweep(*self*, *spiking_features*)

class ipfx.stimulus_protocol_analysis.**RampAnalysis**(*spx*, *sptx*)

Bases: *ipfx.stimulus_protocol_analysis.StimulusProtocolAnalysis*

Methods

toctree references unknown document 'ipfx.stimulus_protocol_analysis.RampAnalysis.mean_features_first_spike'

mean_features_first_spike(self, spikes_set)	Compute mean feature values for the first spike in list of extractors

all_sweep_features	
analyze	
analyze_basic_features	
as_dict	
reset_basic_features	
subthreshold_sweep_features	
suprathreshold_sweep_features	

analyze(*self*, *sweep_set*)

as_dict(*self*, *features*, *extra_params=None*)

class ipfx.stimulus_protocol_analysis.**ShortSquareAnalysis**(*spx*, *sptx*)

Bases: *ipfx.stimulus_protocol_analysis.StimulusProtocolAnalysis*

Methods

toctree references unknown document 'ipfx.stimulus_protocol_analysis.ShortSquareAnalysis.mean_features_first_spike'

mean_features_first_spike(self, spikes_set)	Compute mean feature values for the first spike in list of extractors

all_sweep_features	
analyze	
analyze_basic_features	
as_dict	
reset_basic_features	
subthreshold_sweep_features	
suprathreshold_sweep_features	

analyze (*self*, *sweep_set*)

as_dict (*self*, *features*, *extra_params=None*)

class ipfx.stimulus_protocol_analysis.**StimulusProtocolAnalysis** (*spx*, *sptx*)
Bases: object

Methods

toctree references unknown document 'ipfx.stimulus_protocol_analysis.StimulusProtocolAnalysis.mean_features_first_spike'

mean_features_first_spike(self, spikes_set)	Compute mean feature values for the first spike in list of extractors

all_sweep_features	
analyze	
analyze_basic_features	
as_dict	
reset_basic_features	
subthreshold_sweep_features	
suprathreshold_sweep_features	

MEAN_FEATURES = ['upstroke_downstroke_ratio', 'peak_v', 'peak_t', 'trough_v', 'trough_

all_sweep_features (*self*)

analyze (*self*, *sweep_set*, *extra_sweep_features=None*, *exclude_clipped=False*)

analyze_basic_features (*self*, *sweep_set*, *extra_sweep_features=None*, *exclude_clipped=False*)

as_dict (*self*, *features*, *extra_params=None*)

mean_features_first_spike (*self*, *spikes_set*, *features_list=None*)
Compute mean feature values for the first spike in list of extractors

reset_basic_features (*self*)

subthreshold_sweep_features (*self*, *sweep_features=None*)

suprathreshold_sweep_features (*self*, *sweep_features=None*)

8.28 ipfx.string_utils module

ipfx.string_utils.**to_str** (*s:Union[str, bytes]*) → str
Convert bytes to string if not string

Parameters

s (Union[str,bytes]): variable to convert

Returns

str

8.29 ipfx.subthresh_features module

ipfx.subthresh_features.**baseline_voltage**(*t*, *v*, *start*, *baseline_interval=0.1*, *baseline_detect_thresh=0.3*, *filter_frequency=1.0*)

ipfx.subthresh_features.**fit_membrane_time_constant**(*t*, *v*, *start*, *end*, *rmse_max_tol=1.0*)

Fit an exponential to estimate membrane time constant between start and end

Parameters

v [numpy array of voltages in mV]

t [numpy array of times in seconds]

start [start of time window for exponential fit]

end [end of time window for exponential fit]

rsme_max_tol: minimal acceptable root mean square error (default 1e-4)

Returns

a, inv_tau, y0 [Coefficients of equation y0 + a * exp(-inv_tau * x)]

returns np.nan for values if fit fails

ipfx.subthresh_features.**input_resistance**(*t_set*, *i_set*, *v_set*, *start*, *end*, *baseline_interval=0.1*)

Estimate input resistance in MOhms, assuming all sweeps in passed extractor are hyperpolarizing responses.

ipfx.subthresh_features.**sag**(*t*, *v*, *i*, *start*, *end*, *peak_width=0.005*, *baseline_interval=0.03*)

Calculate the sag in a hyperpolarizing voltage response.

Parameters

peak_width [window width to get more robust peak estimate in sec (default 0.005)]

Returns

sag [fraction that membrane potential relaxes back to baseline]

ipfx.subthresh_features.**time_constant**(*t*, *v*, *i*, *start*, *end*, *max_fit_end=None*, *frac=0.1*, *baseline_interval=0.1*, *min_snr=20.0*)

Calculate the membrane time constant by fitting the voltage response with a single exponential.

Parameters

v [numpy array of voltages in mV]

t [numpy array of times in seconds]

start [start of stimulus interval in seconds]

end [end of stimulus interval in seconds]

max_fit_end [maximum end of exponential fit window. If None, end of fit] window will always be the time of the peak hyperpolarizing deflection. If set, end of fit window will be max_fit_end if it is earlier than the time of peak deflection (default None)

frac [fraction of peak deflection (or deflection at *present_fit_end* if used)] to find to determine start of fit window. (default 0.1)

baseline_interval [duration before *start* for baseline Vm calculation]

min_snr [minimum signal-to-noise ratio (SNR) to allow calculation of time constant.] If SNR is too low, np.nan will be returned. (default 20)

Returns

tau [membrane time constant in seconds]

ipfx.subthresh_features.**voltage_deflection**(*t*, *v*, *i*, *start*, *end*, *deflect_type=None*)

Measure deflection (min or max, between start and end if specified).

Parameters

deflect_type [measure minimal ('min') or maximal ('max') voltage deflection] If not specified, it will check to see if the current (i) is positive or negative between start and end, then choose 'max' or 'min', respectively If the current is not defined, it will default to 'min'.

Returns

deflect_v [peak]

deflect_index [index of peak deflection]

8.30 ipfx.sweep module

class ipfx.sweep.**Sweep**(*t*, *v*, *i*, *clamp_mode*, *sampling_rate*, *sweep_number=None*, *epochs=None*)

Bases: object

Attributes

i

t

v

Methods

toctree references unknown document 'ipfx.sweep.Sweep.detect_epochs'

detect_epochs(self)	Detect epochs if they are not provided in the constructor

select_epoch	
set_time_zero_to_index	

detect_epochs(*self*)

Detect epochs if they are not provided in the constructor

i

select_epoch(*self*, *epoch_name*)

set_time_zero_to_index(*self*, *time_step*)

t

v

class ipfx.sweep.**SweepSet**(*sweeps*)

Bases: object

Attributes

i

sampling_rate

sweep_number

t

v

Methods

align_to_start_of_epoch	
select_epoch	

align_to_start_of_epoch(*self*, *epoch_name*)

i

sampling_rate

select_epoch(*self*, *epoch_name*)

sweep_number

t

v

8.31 ipfx.sweep_props module

ipfx.sweep_props.**assign_sweep_states**(*sweep_states*, *sweep_features*)

Assign sweep state to all sweeps

Parameters

sweep_states: dict of sweep states

sweep_features: list of dics of sweep features

ipfx.sweep_props.**count_sweep_states**(*sweep_states*)

Count passed and total sweeps

Parameters

sweep_states: list of dicts

Sweep state dict has keys: "reason": list of strings "sweep_number": int "passed": True/False

Returns

——-

num_passed_sweeps: int number of sweeps passed QC

num_sweeps: int number of sweeps QCed

ipfx.sweep_props.**create_sweep_state**(*sweep_number*, *fail_tags*)

ipfx.sweep_props.**drop_failed_sweeps**(*sweep_features*)

ipfx.sweep_props.**drop_tagged_sweeps**(*sweep_features*)

ipfx.sweep_props.**extract_sweep_features_subset**(*feature_names*, *sweep_features*)

Parameters

sweep_features: list of dicts of sweep features

feature_names: list of features to select

Returns

sweep_features_subset: list of dicts including only a subset of features from feature_names

ipfx.sweep_props.**modify_sweep_info_keys**(*sweep_list*)

ipfx.sweep_props.**override_auto_sweep_states**(*manual_sweep_states*, *sweep_states*)

ipfx.sweep_props.**remove_sweep_feature**(*feature_name*, *sweep_features*)

8.32 ipfx.time_series_utils module

ipfx.time_series_utils.**average_voltage**(*v*, *t*, *start=None*, *end=None*)

Calculate average voltage between start and end.

Parameters

v [numpy array of voltage time series in mV]

t [numpy array of times in seconds]

start [start of time window for spike detection (optional, default None)]

end [end of time window for spike detection (optional, default None)]

Returns

v_avg [average voltage]

ipfx.time_series_utils.**calculate_dvdt**(*v*, *t*, *filter=None*)

Low-pass filters (if requested) and differentiates voltage by time.

Parameters

v [numpy array of voltage time series in mV]

t [numpy array of times in seconds]

filter [cutoff frequency for 4-pole low-pass Bessel filter in kHz (optional, default None)]

Returns

dvdt [numpy array of time-derivative of voltage (V/s = mV/ms)]

ipfx.time_series_utils.**find_time_index**(*t*, *t_0*)
Find the index value of a given time (t_0) in a time series (t).

Parameters

t [time array]

t_0 [time point to find an index]

Returns

idx: index of t closest to t_0

ipfx.time_series_utils.**flatnotnan**(*a*)
Returns indices that are non nan in a flattened version of a

Parameters

a: np.array

Returns

res: np.array Output array containing indices of an array that are not nan

ipfx.time_series_utils.**has_fixed_dt**(*t*)
Check that all time intervals are identical.

8.33 ipfx.utilities module

ipfx.utilities.**drop_failed_sweeps**(*dataset:ipfx.dataset.ephys_data_set.EphysDataSet*, *stimulus_ontology:Union[ipfx.stimulus.StimulusOntology*, *NoneType]=None*, *qc_criteria:Union[Dict*, *NoneType]=None*) → List[Dict]
A convenience which extracts and QCs sweeps in preparation for dataset feature extraction. This function: 1. extracts sweep qc features 2. removes sweeps tagged with failure messages 3. sets sweep states based on qc results

Parameters

dataset [dataset from which to draw sweeps]

Returns

sweep_features [a list of dictionaries, each describing a sweep]

8.34 Module contents

Top-level package for ipfx.

CHAPTER 9

Credits

- Matthew Aitken @matthewaitken
- Thomas Braun @t-b
- Nicholas Cain @nicain
- Tom Chartrand @tmchartrand
- Ben Dichter @bendichter
- David Feng @dyf
- Nathan Gouwens @gouwens
- Nile Graddis @nilegraddis
- Sergey Gratiy @sgratiy
- Yang Yu @gnayuy

CHAPTER 10

Welcome to Intrinsic Physiology Feature Extractor (IPFX)

IPFX is a Python package for computing intrinsic cell features from electrophysiology data. With this package you can:

- Perform cell data quality control (e.g. resting potential stability)
- Detect action potentials and their features (e.g. threshold time and voltage)
- Calculate features of spike trains (e.g., adaptation index)
- Calculate stimulus-specific cell features

For a full list of features refer to the WHOLE-CELL ELECTROPHYSIOLOGY FEATURE ANALYSIS Section in the Electrophysiology Overview

You can use `IPFX` for tasks varying in scale from analyzing individual sweeps, to datasets of single cells saved in the NWB file format, to constructing a standalone data processing pipeline alike the Allen Institute electrophysiology *QC and Analysis Pipeline* to process thousands of datasets

To get started check out the quick tutorial *Tutorial* or dive into complete examples *Gallery of Examples*.

Python Module Index

i

Index

A

B

C

D

E

F

G

H

I

L

M

N

O

P

Q

R

S

T

U

V

W

Z